
eFEL Documentation

Release 5.7.16+1.g01bfce4

BBP, EPFL

Apr 23, 2025

CONTENTS

1	Installation	3
1.1	Requirements	3
1.2	Installation using pip	3
1.3	Installing the C++ standalone library	3
2	Examples	5
2.1	Quick start	5
2.2	How to set the settings	6
2.3	Parallel efeature extraction using multiprocessing/scoop	10
2.4	DEAP optimisation	11
2.4.1	Introduction	12
2.4.2	Evaluation function	12
2.4.3	Setting up the algorithm	13
2.4.4	Running the code	16
2.5	Reading different file formats	18
2.5.1	Stimulus information within the file	18
2.6	Loading NWB files using Neo	19
2.7	Use of eFEL on the models downloaded from the Neocortical Microcircuit Portal	22
2.8	Extracting features from SONATA Network simulations	29
2.9	Extracellular Features Extraction for MEA Data	37
2.10	Voltage clamp trace and eFEL	47
2.10.1	Activation traces	47
2.10.2	Deactivation traces	55
2.10.3	Inactivation traces	58
3	eFeature descriptions	65
3.1	Implemented eFeatures	65
3.1.1	Spike event features	65
3.1.2	Spike shape features	86
3.1.3	Subthreshold features	103
3.1.4	Extracellular features	114
4	Python API	121
4.1	Submodules	121
4.1.1	efel.api	121
4.1.2	efel.io	127
4.1.3	efel.pyfeatures.cppfeature_access	129
4.1.4	efel.pyfeatures.isi	129
4.1.5	efel.pyfeatures.extrafeats	132
4.1.6	efel.pyfeatures.multitrace	135

4.1.7	efel.pyfeatures.pyfeatures	135
4.1.8	efel.pyfeatures.validation	138
4.1.9	efel.units	138
4.1.10	efel.settings	139
5	Changelog	145
5.1	5.7.11 - 2024-11	145
5.2	5.7.9 - 2024-09	146
5.3	5.6.7 - 2024-08	146
5.4	5.7.1 - 2024-07	146
5.5	5.7.0 - 2024-07	146
5.6	5.6.29 - 2024-06	146
5.7	5.6.27 - 2024-05	146
5.8	5.6.20 - 2024-05	147
5.9	5.6.6 - 2024-04	147
5.10	5.6.0 - 2024-02	147
5.11	5.5.5 - 2024-01	147
5.12	[5.5.4] - 2024-01	147
5.13	5.5.3 - 2024-01	147
5.14	5.5.0 - 2024-01	147
	5.14.1 C++ changes	147
5.15	5.4.0 - 2024-01	148
	5.15.1 C++ changes	148
	5.15.2 Python changes	148
6	Developer's Guide	149
6.1	Requirements	149
6.2	Forking and cloning the git repository	149
6.3	Makefile	150
6.4	Adding a new eFeature	150
	6.4.1 Picking a name	150
	6.4.2 Creating a branch	150
	6.4.3 Implementation	150
	6.4.4 Updating relevant files	150
	6.4.5 Adding a test	151
	6.4.6 Add documentation	151
	6.4.7 Pull request	151
7	Indices and tables	153
	Python Module Index	155
	Index	157

The Electrophys Feature Extract Library (eFEL) allows neuroscientists to automatically extract eFeatures from time series data recorded from neurons (both in vitro and in silico). Examples are the action potential width and amplitude in voltage traces recorded during whole-cell patch clamp experiments. The user of the library provides a set of traces and selects the eFeatures to be calculated. The library will then extract the requested eFeatures and return the values to the user.

The core of the library is written in C++, and a Python wrapper is included. You can automatically compile and install the library as a Python module.

The source code of the eFEL is located on github: [BlueBrain/eFEL](#)

INSTALLATION

1.1 Requirements

- [Python](#)
- [Pip](#) (installed by default in newer versions of Python)
- [Numpy](#) (will be installed automatically by pip)
- The instruction below are written assuming you have access to a command shell on Linux / UNIX / MacOSX / Cygwin

1.2 Installation using pip

The easiest way to install eFEL is to use [pip](#):

```
pip install efel
```

In case you don't have administrator access this command might fail with a permission error. In that case you could install eFEL in your home directory:

```
pip install efel --user
```

Or you could use a [python virtual environment](#):

```
virtualenv pythonenv  
./pythonenv/bin/activate  
pip install git+git://github.com/BlueBrain/eFEL
```

1.3 Installing the C++ standalone library

If your system doesn't have it, install [CMake](#).

Make a new build directory:

```
mkdir build_cmake
```

Configure the build, replace YOURINSTALLDIR with the directory in which you want to install the efel library (e.g. /usr/local):

```
cd build_cmake  
cmake .. -DCMAKE_INSTALL_PREFIX=YOURINSTALLDIR
```

Run the compilation and installation:

```
make install
```

This will have installed a static and shared library as:

```
YOURINSTALLDIR/lib/libefel.a  
YOURINSTALLDIR/lib/libefel.so
```


EXAMPLES

2.1 Quick start

First you need to import the module:

```
import efel
```

To get a list with all the available eFeature names:

```
efel.get_feature_names()
```

Note that the extra-cellular features, the bpap_attenuation feature and the check_ais_initiation feature are not listed above because they have to be used in a special way, as described [here](#) for extra-cellular features, [here](#) for bpap_attenuation feature and [here](#) for check_ais_initiation feature.

The python function to extract eFeatures is get_feature_values(...). Below is a short example on how to use this function.

The code and example trace are available [here](#):

```
"""Basic example 1 for eFEL"""

import efel
import numpy

def main():
    """Main"""

    # Use numpy to read the trace data from the txt file
    data = numpy.loadtxt('example_trace1.txt')

    # Time is the first column
    time = data[:, 0]
    # Voltage is the second column
    voltage = data[:, 1]

    # Now we will construct the datastructure that will be passed to eFEL

    # A 'trace' is a dictionary
    trace1 = {}

    # Set the 'T' (=time) key of the trace
```

(continues on next page)

(continued from previous page)

```

trace1['T'] = time

# Set the 'V' (=voltage) key of the trace
trace1['V'] = voltage

# Set the 'stim_start' (time at which a stimulus starts, in ms)
# key of the trace
# Warning: this need to be a list (with one element)
trace1['stim_start'] = [700]

# Set the 'stim_end' (time at which a stimulus end) key of the trace
# Warning: this need to be a list (with one element)
trace1['stim_end'] = [2700]

# Multiple traces can be passed to the eFEL at the same time, so the
# argument should be a list
traces = [trace1]

# Now we pass 'traces' to the efel and ask it to calculate the feature
# values
traces_results = efel.get_feature_values(traces,
                                         ['AP_amplitude', 'voltage_base'])

# The return value is a list of trace_results, every trace_results
# corresponds to one trace in the 'traces' list above (in same order)
for trace_results in traces_results:
    # trace_result is a dictionary, with as keys the requested eFeatures
    for feature_name, feature_values in trace_results.items():
        print("Feature %s has the following values: %s" % \
              (feature_name, ', '.join([str(x) for x in feature_values])))

if __name__ == '__main__':
    main()

```

The output of this example is:

```

Feature AP_amplitude has the following values: 72.5782441262, 46.3672552618, 41.
↪1546679158, 39.7631750953, 36.1614653031, 37.8489295737
Feature voltage_base has the following values: -75.446665721

```

This means that the eFEL found 5 action potentials in the voltage trace. The amplitudes of these APs are the result of the 'AP_amplitude' feature.

The voltage before the start of the stimulus is measured by 'voltage_base'.

Results are in mV.

2.2 How to set the settings

A lot of features in efel can have their behaviour changed with settings, in order to be more customisable, depending on the traces and the need of the user. Here, we are going to see how to change the settings with an example, and where to find the list of all the settings in efel with their default values.

For this example, we are going to use a trace of a cell model with a bAP (backpropagating action potential) stimulus, recorded in the dendrite.

```
import efel

import numpy
import json

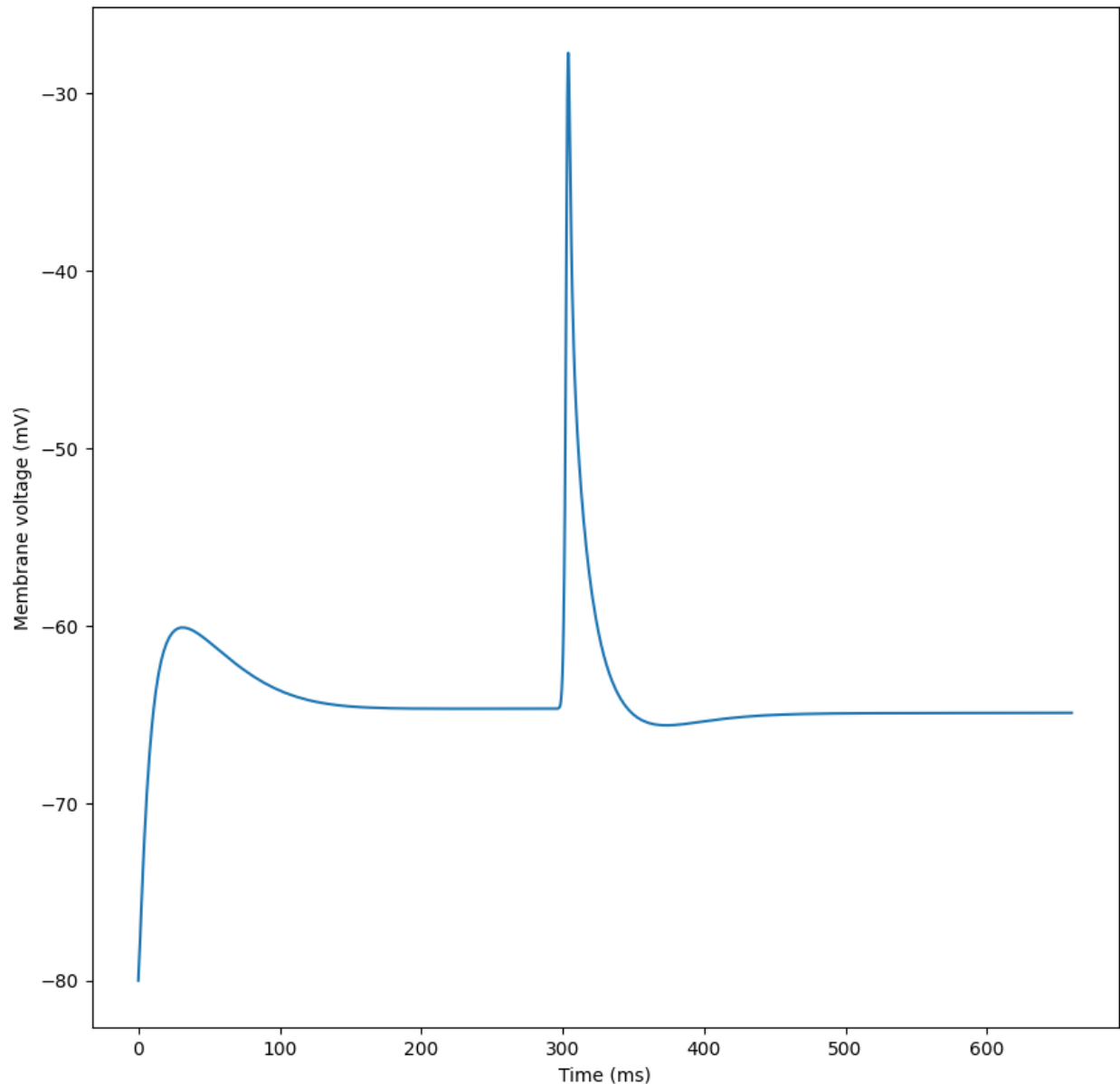
%matplotlib notebook
%matplotlib inline
from matplotlib import pyplot as plt
plt.rcParams['figure.figsize'] = 10, 10

test_url = '../tests/testdata/allfeatures/testbap2data.txt'
test_data = numpy.loadtxt(test_url)
time = test_data[:,0]
voltage = test_data[:, 1]
```

We can now plot this data

```
plt.rcParams['figure.figsize'] = 10, 10

fig1, ax1 = plt.subplots(1)
ax1.plot(time, voltage)
ax1.set_xlabel('Time (ms)')
ax1.set_ylabel('Membrane voltage (mV)');
```



Now, let us extract the peak time. We expect one value shortly after 300 ms.

```
stim_start = 295
stim_end = 500
trace = {'T': time, 'V': voltage, 'stim_start': [stim_start], 'stim_end': [stim_end]}

feature_values = efel.get_feature_values([trace], ['peak_time'])[0]
print(f'Spike detection threshold is {efel.get_settings().Threshold} mV.')
print(feature_values)
```

```
Spike detection threshold is -20.0 mV.
{'peak_time': None}
```

```
/path/to/efel/pyfeatures/cppfeature_access.py:14: RuntimeWarning: Error while_
```

(continues on next page)

(continued from previous page)

```

→calculating peak_time, An error occurred while computing the feature, feature is not_
→found. Voltage never goes below or above threshold in spike detection.
warnings.warn(

```

We can see that we don't have any value returned. The reason is that, by default, eFEL looks for spikes that go above -20 mV. This is fine for most cells when recorded in the soma, but here, we are recording a spike in the dendrite after a soma stimulus. This spike has a smaller amplitude than the one in the soma, and is thus below the spike-detecting threshold.

This can be solved by modifying the settings, using the easy to use `set_setting` function! By simply modifying the default threshold value of -20 to a lower value, e.g. -30, the spike gets detected!

```

efel.api.set_setting('Threshold', -30.)

feature_values = efel.get_feature_values([trace], ['peak_time'])[0]
print(feature_values)

```

```

{'peak_time': array([304.])}

```

If you want to see the current settings, you can do so by using the `get_settings` function:

```

print(efel.get_settings())

```

```

Threshold: -30.0
DerivativeThreshold: 10.0
DownDerivativeThreshold: -12.0
dependencyfile_path: /path/to/efel/DependencyV5.txt
spike_skipf: 0.1
max_spike_skip: 2
interp_step: 0.1
burst_factor: 1.5
strict_burst_factor: 2.0
voltage_base_start_perc: 0.9
voltage_base_end_perc: 1.0
current_base_start_perc: 0.9
current_base_end_perc: 1.0
rise_start_perc: 0.0
rise_end_perc: 1.0
initial_perc: 0.1
min_spike_height: 20.0
strict_stiminterval: False
initburst_freq_threshold: 50
initburst_sahp_start: 5
initburst_sahp_end: 100
DerivativeWindow: 3
voltage_base_mode: mean
current_base_mode: mean
precision_threshold: 1e-10
sahp_start: 5.0
ignore_first_ISI: True
impedance_max_freq: 50.0
AP_phaseslope_range: 2
inactivation_tc_end_skip: 10

```

You can reset the settings to their default value at any time using the `reset()` function. It is good practice to use it whenever you want to change the settings, in order to be sure that previously set settings are not interfering with your new settings.

```
efel.reset()
```

The settings can also be passed down to eFEL by passing them through the trace dictionary, inside a list:

```
trace['Threshold'] = [-30.]
feature_values = efel.get_feature_values([trace], ['peak_time'])[0]
print(feature_values)
```

```
{'peak_time': array([304.])}
```

The complete list of settings and their default value can be found in the documentation of [the Settings class here](#)

2.3 Parallel efeature extraction using multiprocessing/scoop

You can use multiprocessing or scoop (Scalable COncurrent Operations in Python) packages to extract features in parallel. This is a simple example of how to do it using multiprocessing:

```
import efel
import numpy
```

```
def main():
    """Main"""

    traces = []
    for filename in ['example_trace1.txt', 'example_trace2.txt']:
        # Use numpy to read the trace data from the txt file
        data = numpy.loadtxt(filename)

        # Time is the first column
        time = data[:, 0]
        # Voltage is the second column
        voltage = data[:, 1]

        # Now we will construct the datastructure that will be passed to eFEL

        # A 'trace' is a dictionary
        trace1 = {}

        # Set the 'T' (=time) key of the trace
        trace1['T'] = time

        # Set the 'V' (=voltage) key of the trace
        trace1['V'] = voltage

        # Set the 'stim_start' (time at which a stimulus starts, in ms)
        # key of the trace
        # Warning: this need to be a list (with one element)
        trace1['stim_start'] = [700]
```

(continues on next page)

(continued from previous page)

```

# Set the 'stim_end' (time at which a stimulus end) key of the trace
# Warning: this need to be a list (with one element)
trace1['stim_end'] = [2700]

# Multiple traces can be passed to the eFEL at the same time, so the
# argument should be a list
traces.append(trace1)

# Now we pass 'traces' to the efel and ask it to calculate the feature
# values

## Using multiprocessing
import multiprocessing
pool = multiprocessing.Pool()
traces_results = efel.get_feature_values(
    traces, [
        'AP_amplitude', 'voltage_base'], parallel_map=pool.map)

# The return value is a list of trace_results, every trace_results
# corresponds to one trace in the 'traces' list above (in same order)
for trace_number, trace_results in enumerate(traces_results):
    print("Results for trace %d" % (trace_number + 1))
    # trace_result is a dictionary, with as keys the requested features
    for feature_name, feature_values in trace_results.items():
        print("Feature %s has the following values: %s" % \
            (feature_name, ', '.join([str(x) for x in feature_values])))

```

```
main()
```

If you want to use scoop, it is advisable to use a python script e.g. the `scoop_example.py` (present in the `parallel` example folder) instead of the notebook. You can launch the script e.g. using

```
python -m scoop -n 4 scoop_example.py
```

where arguments are:

`-m scoop`: The `-m` flag tells Python to run a module as a script. In this case, the module is `scoop`, which is a library for distributed computing. Running SCOOP as a module initializes its environment, preparing it for distributed execution.

`-n 4`: This option specifies the number of worker processes that scoop should use for executing tasks. In this case, 4 workers are requested. scoop will distribute tasks among these workers, allowing for parallel execution.

2.4 DEAP optimisation

Contents

- *DEAP optimisation*
 - *Introduction*
 - *Evaluation function*
 - *Setting up the algorithm*

– *Running the code*

2.4.1 Introduction

Using the eFEL, pyNeuron and the DEAP optimisation library one can very easily set up a genetic algorithm to fit parameters of a neuron model.

We propose this setup because it leverages the power of the Python language to load several software tools in a compact script. The DEAP (Distributed Evolutionary Algorithms in Python) allows you to easily switch algorithms. Parallelising your evaluation function over cluster computers becomes a matter of only adding a couple of lines to your `code`, thanks to `pyScoop`.

In this example we will assume you have installed `eFEL`, `pyNeuron` and `DEAP`

The code of the example below can be downloaded from [here](#)

To keep the example simple, let's start from a passive single compartmental model. The parameters to fit will be the conductance and reversal potential of the leak channel. We will simulate the model for 1000 ms, and at 500 ms a step current of 1.0 nA is injected until the end of the simulation.

The objective values of the optimisation will be the voltage before the current injection (i.e. the 'voltage_base' feature), and the steady state voltage during the current injection at the end of the simulation ('steady_state_voltage').

2.4.2 Evaluation function

We now have to use pyNeuron to define the evaluation function to be optimised. The input arguments are the parameters:

```
g_pas, e_pas
```

and the return values:

```
abs(voltage_base - target_voltage1)
abs(steady_state_voltage - target_voltage2)
```

This translates into the following file (let's call it 'deap_efel_eval1.py'):

```
import neuron
neuron.h.load_file('stdrun.hoc')

import efel

# pylint: disable=W0212

def evaluate(individual, target_voltage1=-80, target_voltage2=-60):
    """
    Evaluate a neuron model with parameters e_pas and g_pas, extracts
    eFeatures from resulting traces and returns a tuple with
    abs(voltage_base-target_voltage1) and
    abs(steady_state_voltage-target_voltage2)
    """

    neuron.h.v_init = target_voltage1
```

(continues on next page)

(continued from previous page)

```

soma = neuron.h.Section()

soma.insert('pas')

soma.g_pas = individual[0]
soma.e_pas = individual[1]

clamp = neuron.h.IClamp(0.5, sec=soma)

stim_start = 500
stim_end = 1000

clamp.amp = 1.0
clamp.delay = stim_start
clamp.dur = 1000000

voltage = neuron.h.Vector()
voltage.record(soma(0.5)._ref_v)

time = neuron.h.Vector()
time.record(neuron.h._ref_t)

neuron.h.tstop = stim_end
neuron.h.run()

trace = {}
trace['T'] = time
trace['V'] = voltage
trace['stim_start'] = [stim_start]
trace['stim_end'] = [stim_end]
traces = [trace]

features = efel.get_feature_values(traces, ["voltage_base",
                                           "steady_state_voltage"])
voltage_base = features[0]["voltage_base"][0]
steady_state_voltage = features[0]["steady_state_voltage"][0]

return abs(target_voltage1 - voltage_base), \
        abs(target_voltage2 - steady_state_voltage)

```

2.4.3 Setting up the algorithm

Now that we have an evaluation function we just have to pass this to the DEAP optimisation library. DEAP allows you to easily set up a genetic algorithm to optimise your evaluation function. Let us first import all the necessary components:

```

import random
import numpy

import deap
import deap.gp
import deap.benchmarks

```

(continues on next page)

(continued from previous page)

```

from deap import base
from deap import creator
from deap import tools
from deap import algorithms
random.seed(1)

```

Next we define a number of constants that will be used as settings for DEAP later:

```

# Population size
POP_SIZE = 100
# Number of offspring in every generation
OFFSPRING_SIZE = 100

# Number of generations
NGEN = 300

# The parent and offspring population size are set the same
MU = OFFSPRING_SIZE
LAMBDA = OFFSPRING_SIZE
# Crossover probability
CXPB = 0.7
# Mutation probability, should sum to one together with CXPB
MUTPB = 0.3

# Eta parameter of cx and mut operators
ETA = 10.0

```

We have two parameters with the following bounds:

```

# The size of the individual is 2 (parameters g_pas and e_pas)
IND_SIZE = 2

LOWER = [1e-8, -100.0]
UPPER = [1e-4, -20.0]

```

As evolutionary algorithm we choose NSGA2:

```

SELECTOR = "NSGA2"

```

Let's create the DEAP individual and fitness. We set the weights of the fitness values to -1.0 so that the fitness function will be minimised instead of maximised:

```

creator.create("Fitness", base.Fitness, weights=[-1.0] * 2)

```

The individual will just be a list (of two parameters):

```

creator.create("Individual", list, fitness=creator.Fitness)

```

We want to start with individuals for which the parameters are picked from a uniform random distribution. Let's create a function that returns such a random list based on the bounds and the dimensions of the problem:

```

def uniform(lower_list, upper_list, dimensions):
    """Fill array """

```

(continues on next page)

(continued from previous page)

```

if hasattr(lower_list, '__iter__'):
    return [random.uniform(lower, upper) for lower, upper in
            zip(lower_list, upper_list)]
else:
    return [random.uniform(lower_list, upper_list)
            for _ in range(dimensions)]

```

DEAP works with the concept of ‘toolboxes’. The user defines genetic algorithm’s individuals, operators, etc by registering them in a toolbox.

We first create the toolbox:

```
toolbox = base.Toolbox()
```

Then we register the ‘uniform’ function we defined above:

```
toolbox.register("uniformparams", uniform, LOWER, UPPER, IND_SIZE)
```

The three last parameters of this register call will be passed on to the ‘uniform’ function call

Now we can also register an individual:

```

toolbox.register(
    "Individual",
    tools.initIterate,
    creator.Individual,
    toolbox.uniformparams)

```

And a population as list of individuals:

```
toolbox.register("population", tools.initRepeat, list, toolbox.Individual)
```

The function to evaluate we defined above. Assuming you saved that files as ‘deap_efel_eval1.py’, we can import it as a module, and register the function:

```

import deap_efel_eval1
toolbox.register("evaluate", deap_efel_eval1.evaluate)

```

For the mutation and crossover operator we use builtin operators that are typically used with NSGA2:

```

toolbox.register(
    "mate",
    deap.tools.cxSimulatedBinaryBounded,
    eta=ETA,
    low=LOWER,
    up=UPPER)
toolbox.register("mutate", deap.tools.mutPolynomialBounded, eta=ETA,
                 low=LOWER, up=UPPER, indpb=0.1)

```

And then we specify the selector to be used:

```

toolbox.register(
    "select",
    tools.selNSGA2)

```

We initialise the population with the size of the offspring:

```
pop = toolbox.population(n=MU)
```

And register some statistics we want to print during the run of the algorithm:

```
first_stats = tools.Statistics(key=lambda ind: ind.fitness.values[0])
second_stats = tools.Statistics(key=lambda ind: ind.fitness.values[1])
stats = tools.MultiStatistics(obj1=first_stats, obj2=second_stats)
stats.register("min", numpy.min, axis=0)
```

The only thing that is left now is to run the algorithm in 'main':

```
if __name__ == '__main__':
    pop, logbook = algorithms.eaMuPlusLambda(
        pop,
        toolbox,
        MU,
        LAMBDA,
        CXPB,
        MUTPB,
        NGEN,
        stats,
        halloffame=None)
```

For your convenience the full code is in a code block below. It should be saved as 'deap_efel.py'.

2.4.4 Running the code

Assuming that the necessary dependencies are installed correctly the optimisation can then be run with:

```
python deap_efel.py
```

The full code of 'deap_efel.py':

```
import random
import numpy

import deap
import deap.gp
import deap.benchmarks
from deap import base
from deap import creator
from deap import tools
from deap import algorithms

random.seed(1)
POP_SIZE = 100
OFFSPRING_SIZE = 100

NGEN = 300
ALPHA = POP_SIZE
MU = OFFSPRING_SIZE
LAMBDA = OFFSPRING_SIZE
```

(continues on next page)

(continued from previous page)

```

CXPB = 0.7
MUTPB = 0.3
ETA = 10.0

SELECTOR = "NSGA2"

IND_SIZE = 2
LOWER = [1e-8, -100.0]
UPPER = [1e-4, -20.0]

creator.create("Fitness", base.Fitness, weights=[-1.0] * 2)
creator.create("Individual", list, fitness=creator.Fitness)

def uniform(lower_list, upper_list, dimensions):
    """Fill array """

    if hasattr(lower_list, '__iter__'):
        return [random.uniform(lower, upper) for lower, upper in
                zip(lower_list, upper_list)]
    else:
        return [random.uniform(lower_list, upper_list)
                for _ in range(dimensions)]

toolbox = base.Toolbox()
toolbox.register("uniformparams", uniform, LOWER, UPPER, IND_SIZE)
toolbox.register(
    "Individual",
    tools.initIterate,
    creator.Individual,
    toolbox.uniformparams)
toolbox.register("population", tools.initRepeat, list, toolbox.Individual)

import deap_efel_eval1
toolbox.register("evaluate", deap_efel_eval1.evaluate)

toolbox.register(
    "mate",
    deap.tools.cxSimulatedBinaryBounded,
    eta=ETA,
    low=LOWER,
    up=UPPER)
toolbox.register("mutate", deap.tools.mutPolynomialBounded, eta=ETA,
                 low=LOWER, up=UPPER, indpb=0.1)

toolbox.register("variate", deap.algorithms.varAnd)

toolbox.register(
    "select",
    tools.selNSGA2)

```

(continues on next page)

(continued from previous page)

```

pop = toolbox.population(n=MU)

first_stats = tools.Statistics(key=lambda ind: ind.fitness.values[0])
second_stats = tools.Statistics(key=lambda ind: ind.fitness.values[1])
stats = tools.MultiStatistics(obj1=first_stats, obj2=second_stats)
stats.register("min", numpy.min, axis=0)

if __name__ == '__main__':
    pop, logbook = algorithms.eaMuPlusLambda(
        pop,
        toolbox,
        MU,
        LAMBDA,
        CXPB,
        MUTPB,
        NGEN,
        stats,
        halloffame=None)

```

2.5 Reading different file formats

Neo is a Python package which provides support for reading a wide range of neurophysiology file formats, including Spike2, NeuroExplorer, AlphaOmega, Axon, Blackrock, Plexon and Tdt.

The function `efel.io.load_neo_file()` reads data from any of the file formats supported by Neo and formats it for use in eFEL.

As an example, suppose we have an .abf file containing a single trace. Since eFEL requires information about the start and end times of the current injection stimulus, we provide these times as well as the filename:

```

import efel

data = efel.io.load_neo_file("path/first_file.abf", stim_start=200, stim_end=700)

```

Since some file formats can contain multiple recording episodes (e.g. trials) and multiple signals per episode, the function returns traces in a list of lists, like this:

```

data : [Segment_1, Segment_2, ..., Segment_n]
       with Segment_1 = [Trace_1, Trace_2, ..., Trace_n]

```

Since our file contains only a single recording episode, our list of traces is:

```
traces = data[0]
```

which we pass to eFEL as follows:

```
features = efel.get_feature_values(traces, ['AP_amplitude', 'voltage_base'])
```

2.5.1 Stimulus information within the file

Some file formats can store information about the current injection stimulus. In this second example, the file contains an Epoch object named “stimulation”, so we don’t need to explicitly specify *stim_start* and *stim_end*:

```
data2 = efel.io.load_neo_file("path/second_file.h5")
```

2.6 Loading NWB files using Neo

This notebook demonstrates how to load electrophysiology data using Neo for eFEL eFeatures extraction.

```
import efel

import numpy

%matplotlib notebook
%matplotlib inline
from matplotlib import pyplot as plt
plt.rcParams['figure.figsize'] = 10, 10
```

We will use the voltage trace recordings obtained from a mouse thalamic cell classified as a bursting accommodating (bAC) etype. The data is stored in an NWB (Neurodata Without Borders) file, and we will use the [Python Neo](#) library to load it.

```
test_data = "../../../tests/testdata/JY180308_A_1.nwb"
stim_start = 250
stim_end = 600
blocks = efel.io.load_neo_file(test_data, stim_start, stim_end)
traces = blocks[0][0]
```

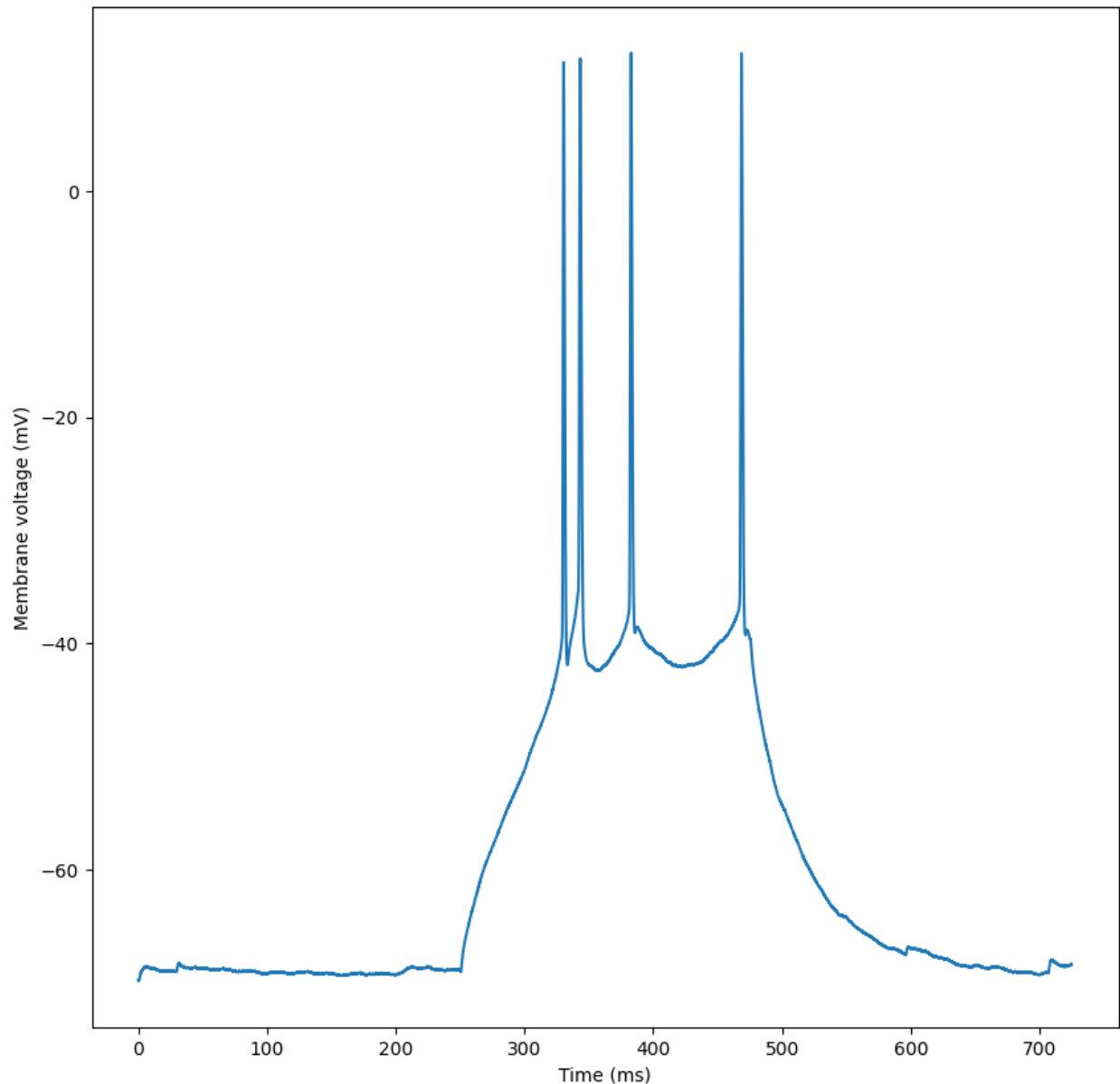
Let's get a trace that includes a burst

```
trace = traces[5]
```

```
time = trace['T']
voltage = trace['V']
```

```
plt.rcParams['figure.figsize'] = 10, 10

fig1, ax1 = plt.subplots(1)
ax1.plot(time, voltage)
ax1.set_xlabel('Time (ms)')
ax1.set_ylabel('Membrane voltage (mV)');
```



We can now use the eFEL to extract eFeature values from the trace shown above.

We will use the `get_feature_values()` function, which accepts a list of trace and the requested eFeature names as input.

Let's extract some burst-related features

```
efel.set_setting("ignore_first_ISI", False) # Don't ignore the first spike
efel.set_setting("strict_burst_factor", 4.0) # The burst detection can be fine-tuned by
↪ changing the setting strict_burst_factor. Default value is 2.0.
feature_values = efel.get_feature_values([trace], ['spikes_per_burst', 'strict_burst_
↪ number', 'strict_burst_mean_freq', 'peak_time', 'AP_height', 'peak_indices', 'burst_
↪ begin_indices', 'burst_end_indices'])[0]
feature_values
```



```
{'spikes_per_burst': array([4]),
 'strict_burst_number': array([1]),
 'strict_burst_mean_freq': array([28.96451846]),
 'peak_time': array([330.4, 343.2, 382.7, 468.5]),
 'AP_height': array([11.40000057, 11.71249962, 12.23750019, 12.19999981]),
 'peak_indices': array([3304, 3432, 3827, 4685]),
 'burst_begin_indices': array([0]),
 'burst_end_indices': array([3])}
```

```
burst_begin_indices = feature_values['burst_begin_indices'][0]
burst_end_indices = feature_values['burst_end_indices'][0]
peak_times = feature_values['peak_time']
ap_heights = feature_values['AP_height']
burst_mean_freq = feature_values['strict_burst_mean_freq']

time_spike_indices = numpy.where((time > stim_start) & (time < stim_end))
time_spike = time[time_spike_indices]
voltage_spike = voltage[time_spike_indices]

plt.figure(figsize=(10, 6))
plt.plot(time_spike, voltage_spike, label='Voltage Trace')

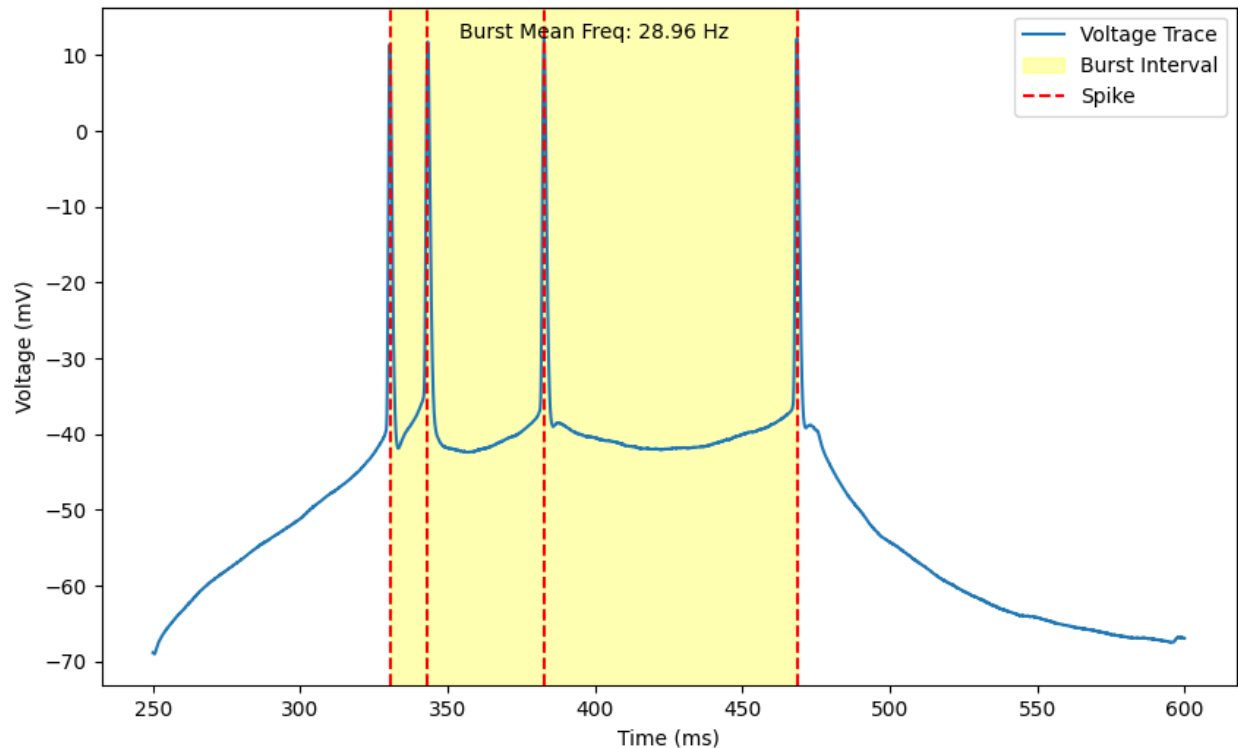
burst_start = peak_times[burst_begin_indices]
burst_end = peak_times[burst_end_indices]
mean_frequency = burst_mean_freq[0]

plt.axvspan(burst_start, burst_end, color='yellow', alpha=0.3, label='Burst Interval')

for spike_time in peak_times[burst_begin_indices:burst_end_indices+1]:
    plt.axvline(x=spike_time, color='red', linestyle='--', label='Spike' if 'Spike' not_
    ↪in plt.gca().get_legend_handles_labels()[1] else "")

plt.text((burst_start + burst_end) / 2, max(voltage_spike), f'Burst Mean Freq: {mean_
    ↪frequency:.2f} Hz', horizontalalignment='center', color='black')

plt.legend()
plt.xlabel('Time (ms)')
plt.ylabel('Voltage (mV)')
plt.show()
```



We can save the feature values obtained from `get_feature_values` for later use. Two functions, `save_feature_values_to_json` and `save_feature_values_to_csv`, are provided for this purpose. They save the feature values as a JSON file and a CSV file, respectively.

```
efel.io.save_feature_to_json(feature_values, 'output.json')
efel.io.save_feature_to_csv(feature_values, 'output.csv')
```

2.7 Use of eFEL on the models downloaded from the Neocortical Microcircuit Portal

Requirements: - Python 3.9+, including Pip (<https://pip.readthedocs.org>) - A version of Neuron (with Python support) installed on your computer (for instruction, see <https://bbp.epfl.ch/nmc-portal/tools>)

Make matplotlib plots show up in the notebook:

```
%matplotlib inline
```

Install the eFeature Extraction Library:

```
!pip install efel
import efel
```

Get a model package from the website

```
!curl -o L5_TTPC2.zip https://bbp.epfl.ch/nmc-portal/assets/documents/static/downloads-
zip/L5_TTPC2_cADpyr232_1.zip
```

Unzip the model package:

```
!unzip L5_TTPC2.zip
```

Change directory to the model package directory:

```
import os
os.chdir('L5_TTPC2_cADpyr232_1')
```

Compile the Neuron mechanisms (if this fails, you might not have installed Neuron correctly)

```
!nrnivmodl ./mechanisms
```

Import the model package in Python, and run it:

```
import run
run.main(plot_traces=True)
```

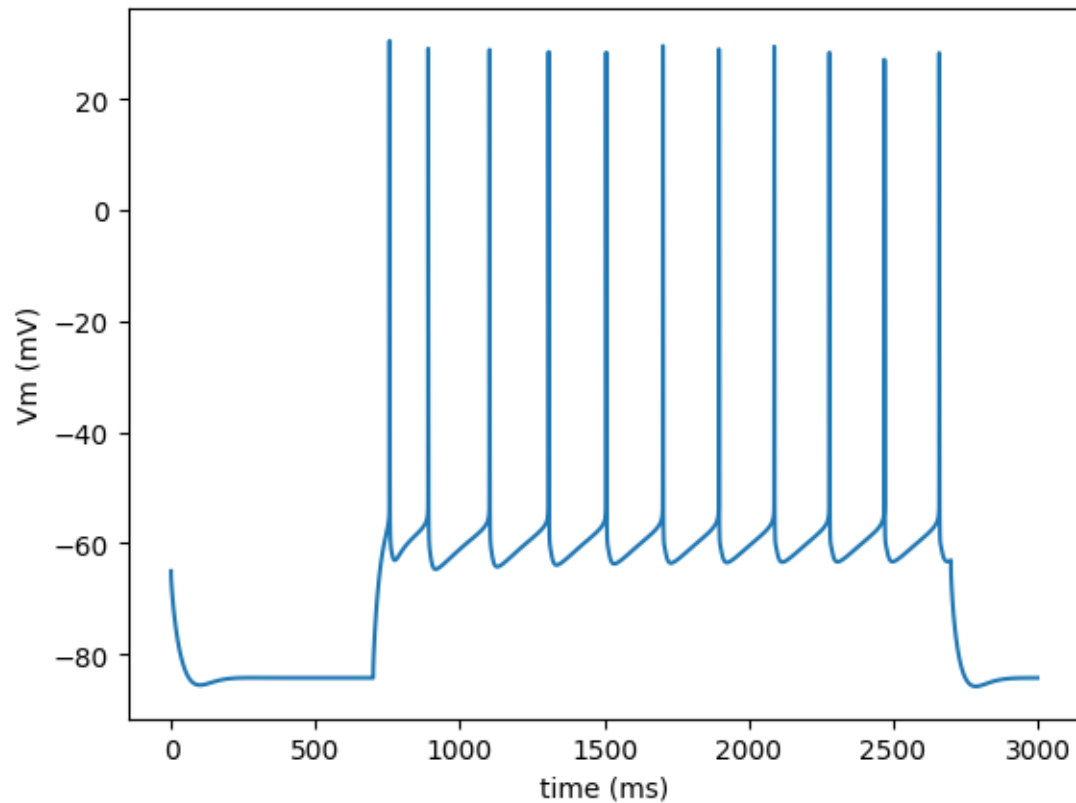
```
Warning: no DISPLAY environment variable.
--No graphics will be displayed.
```

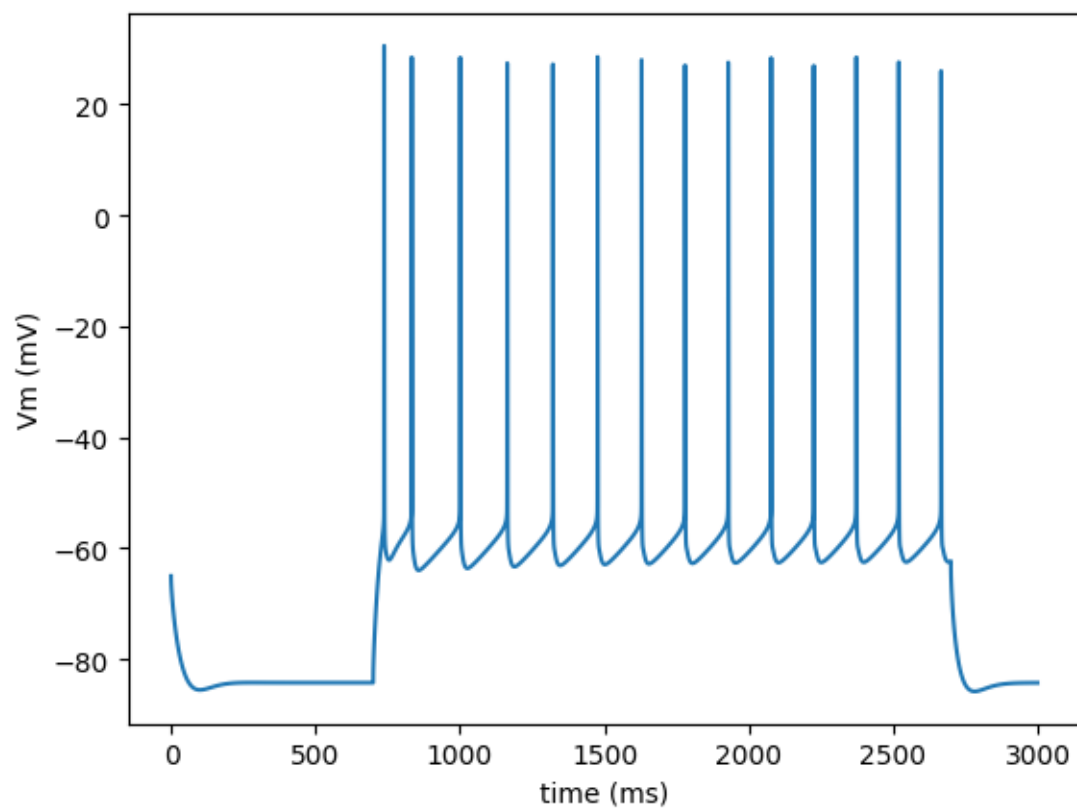
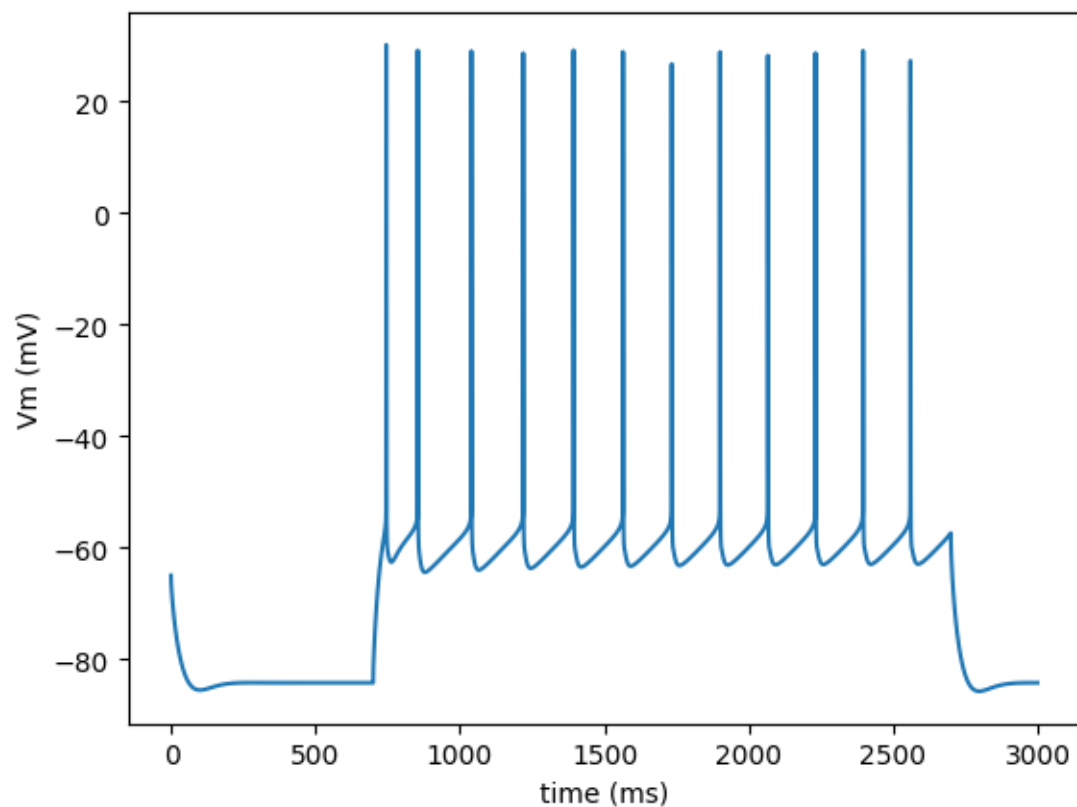
```
Loading constants
Setting temperature to 34.000000 C
Setting simulation time step to 0.025000 ms
1
1
1
Loading cell cADpyr232_L5_TTPC2_8052133265
Attaching stimulus electrodes
Setting up step current clamp: amp=0.593063 nA, delay=700.000000 ms, duration=2000.
↪000000 ms
Setting up hypamp current clamp: amp=-0.286011 nA, delay=0.000000 ms, duration=3000.
↪000000 ms
Attaching recording electrodes
Setting simulation time to 3s for the step currents
Disabling variable timestep integration
Running for 3000.000000 ms
Soma voltage for step 1 saved to: python_recordings/soma_voltage_step1.dat
Loading cell cADpyr232_L5_TTPC2_8052133265
Attaching stimulus electrodes
Setting up step current clamp: amp=0.642485 nA, delay=700.000000 ms, duration=2000.
↪000000 ms
Setting up hypamp current clamp: amp=-0.286011 nA, delay=0.000000 ms, duration=3000.
↪000000 ms
Attaching recording electrodes
Setting simulation time to 3s for the step currents
Disabling variable timestep integration
Running for 3000.000000 ms
Soma voltage for step 2 saved to: python_recordings/soma_voltage_step2.dat
Loading cell cADpyr232_L5_TTPC2_8052133265
Attaching stimulus electrodes
Setting up step current clamp: amp=0.691907 nA, delay=700.000000 ms, duration=2000.
↪000000 ms
Setting up hypamp current clamp: amp=-0.286011 nA, delay=0.000000 ms, duration=3000.
↪000000 ms
```

(continues on next page)

(continued from previous page)

```
Attaching recording electrodes
Setting simulation time to 3s for the step currents
Disabling variable timestep integration
Running for 3000.000000 ms
Soma voltage for step 3 saved to: python_recordings/soma_voltage_step3.dat
```





Load the output of the model package in numpy array

```
import numpy
times = []
voltages = []
for step_number in range(1,4):
    data = numpy.loadtxt('python_recordings/soma_voltage_step%d.dat' % step_number)
    times.append(data[:, 0])
    voltages.append(data[:, 1])
```

Prepare the traces for the eFEL

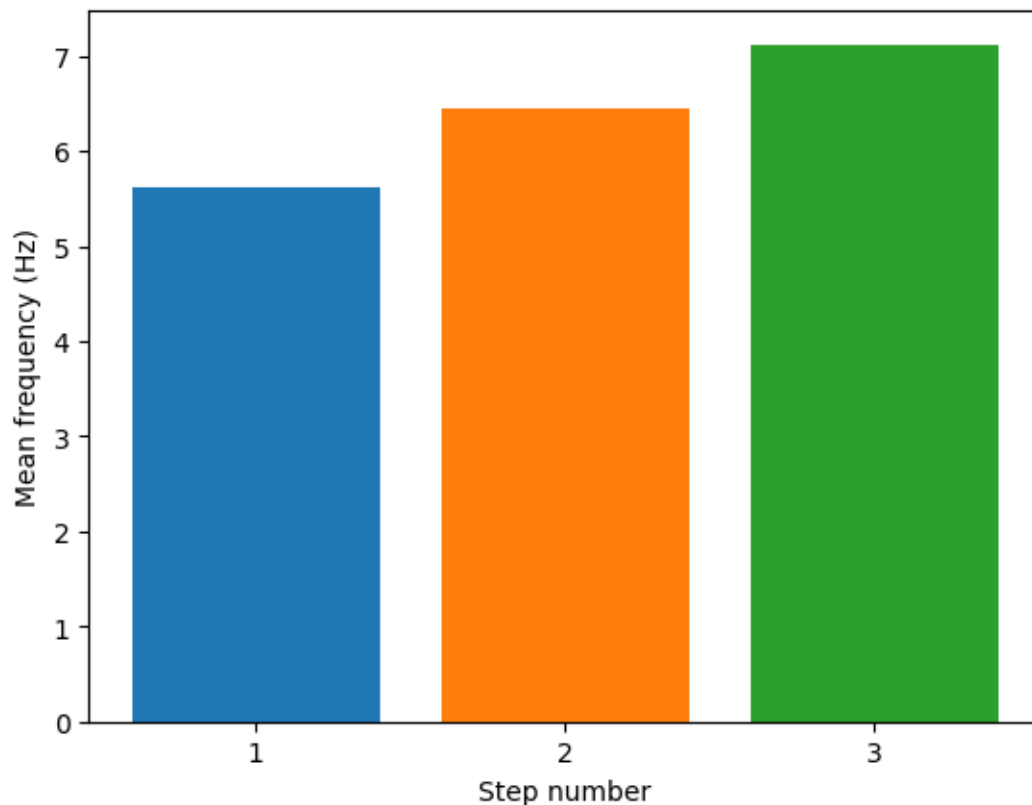
```
traces = []
for step_number in range(3):
    trace = {}
    trace['T'] = times[step_number]
    trace['V'] = voltages[step_number]
    trace['stim_start'] = [700]
    trace['stim_end'] = [2700]
    traces.append(trace)
```

Run the eFEL on the trace

```
feature_values = efel.get_feature_values(traces, ['mean_frequency', 'adaptation_index2',
↪ 'ISI_CV', 'doublet_ISI', 'time_to_first_spike', 'AP_height', 'AHP_depth_abs', 'AHP_
↪ depth_abs_slow', 'AHP_slow_time', 'AP_width', 'peak_time', 'AHP_time_from_peak'])
```

Plot frequencies over steps

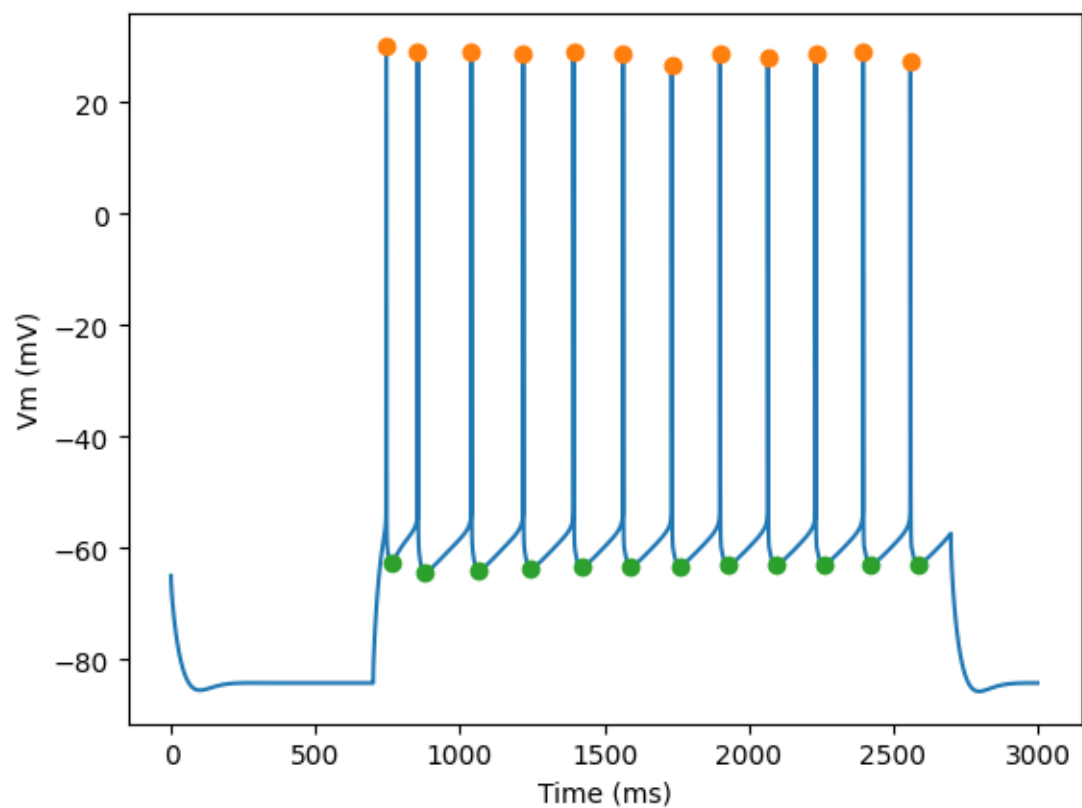
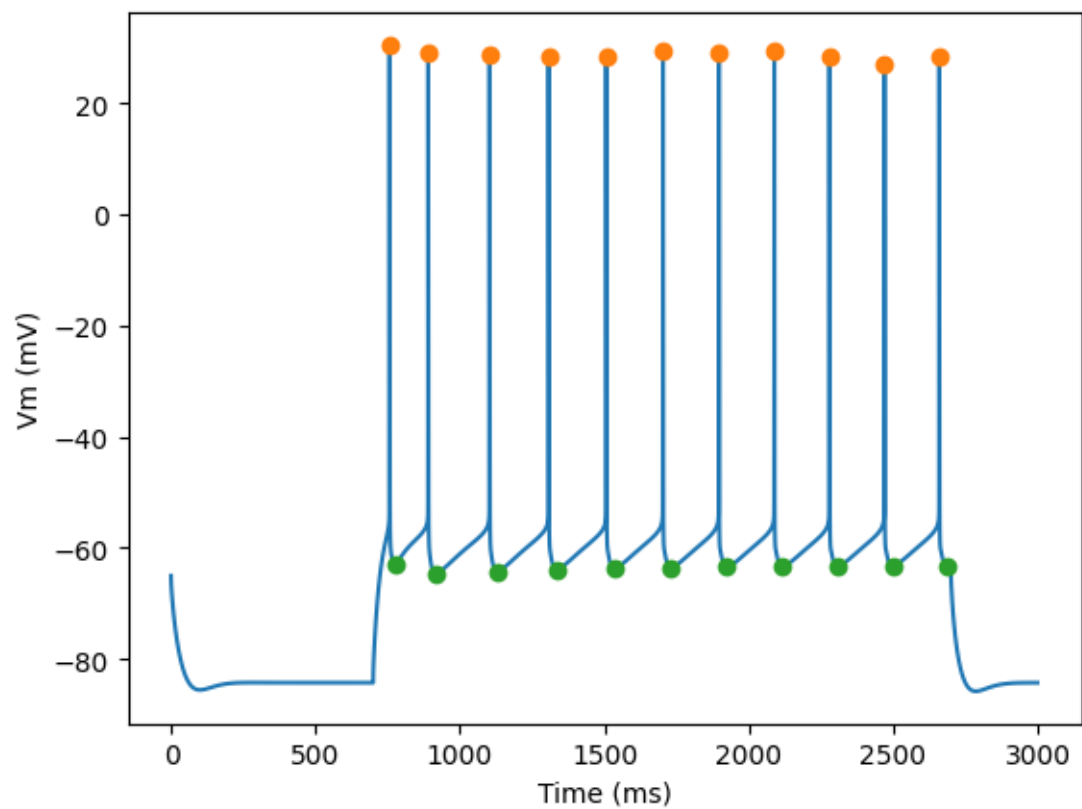
```
import pylab
for step_number in range(3):
    pylab.bar(step_number, feature_values[step_number]['mean_frequency'][0], align=
↪ 'center')
pylab.ylabel('Mean frequency (Hz)')
pylab.xlabel('Step number')
pylab.xticks(range(3), range(1,4))
pylab.show()
```

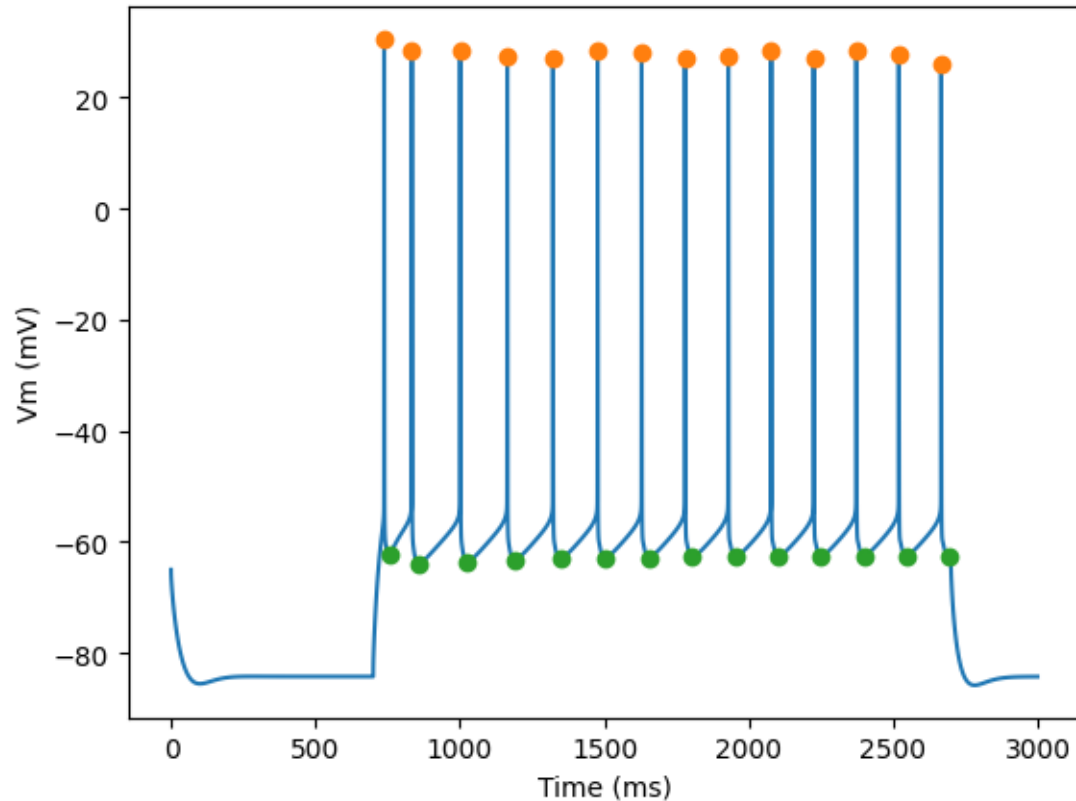


Plot AP height and AHP depth

```
for step_number in range(3):
    time = times[step_number]
    voltage = voltages[step_number]
    peak_times = feature_values[step_number]['peak_time']
    ahp_time = feature_values[step_number]['AHP_time_from_peak']
    ap_heights = feature_values[step_number]['AP_height']
    AHP_depth_abss = feature_values[step_number]['AHP_depth_abs']

    pylab.plot(time, voltage)
    pylab.plot(peak_times, ap_heights, 'o')
    pylab.plot(peak_times+ahp_time, AHP_depth_abss, 'o')
    pylab.xlabel('Time (ms)')
    pylab.ylabel('Vm (mV)')
    pylab.show()
```





2.8 Extracting features from SONATA Network simulations

This notebook shows how to extract features of a group of cells from a SONATA network, specifically focusing on a small portion of non-barrel primary somatosensory cortex circuit from juvenile rats, with the help of [BlueCellulab](#). For those interested in conducting a more in-depth analysis, the entire circuit dataset is accessible on [Zenodo](#). For more details about the simulation and in-depth insights on the circuit, please refer to the [BlueCellulab SONATA Network example](#) and the related [paper](#), respectively.

Note: The compiled mechanisms need to be provided before importing bluecellulab.

```
!nrnivmodl ./mechanisms
```

```
import json
from pathlib import Path

from matplotlib import pyplot as plt

from bluecellulab import CircuitSimulation
import efel
```

In this example, a small sub-circuit has been extracted from the [sscx circuit](#). This sub-circuit specifically consists of a random selection of cells exhibiting delayed stuttering (dSTUT) etype.

The `simulation_config` specifies the types of input stimuli to be applied to the cells. In this case, we have selected a 'relative_linear' stimulus of 70 ms and set the stimulus current at a level equivalent to 100 percent of the cell's threshold current.

```
simulation_config = Path("./") / "simulation_config.json"
with open(simulation_config) as f:
    simulation_config_dict = json.load(f)
print(json.dumps(simulation_config_dict, indent=4))
```

```
{
  "manifest": {
    "$OUTPUT_DIR": "."
  },
  "run": {
    "tstop": 100.0,
    "dt": 0.025,
    "random_seed": 1
  },
  "conditions": {
    "v_init": -65
  },
  "target_simulator": "NEURON",
  "network": "./01/circuit_config.json",
  "node_set": "dSTUT_mini",
  "output": {
    "output_dir": "$OUTPUT_DIR/output_sonata",
    "spikes_file": "out.h5",
    "spikes_sort_order": "by_time"
  },
  "inputs": {
    "continuous_linear": {
      "input_type": "current_clamp",
      "module": "relative_linear",
      "delay": 20.0,
      "duration": 70.0,
      "percent_start": 100,
      "node_set": "dSTUT_mini"
    }
  },
  "reports": {
    "soma": {
      "cells": "dSTUT_mini",
      "variable_name": "v",
      "type": "compartment",
      "dt": 1.0,
      "start_time": 0.0,
      "end_time": 20.0,
      "sections": "soma",
      "compartments": "center"
    }
  }
}
```

We use BlueCellulab for simulating smaller scale circuits, in contrast to the larger-scale simulations conducted with Neurodamus.

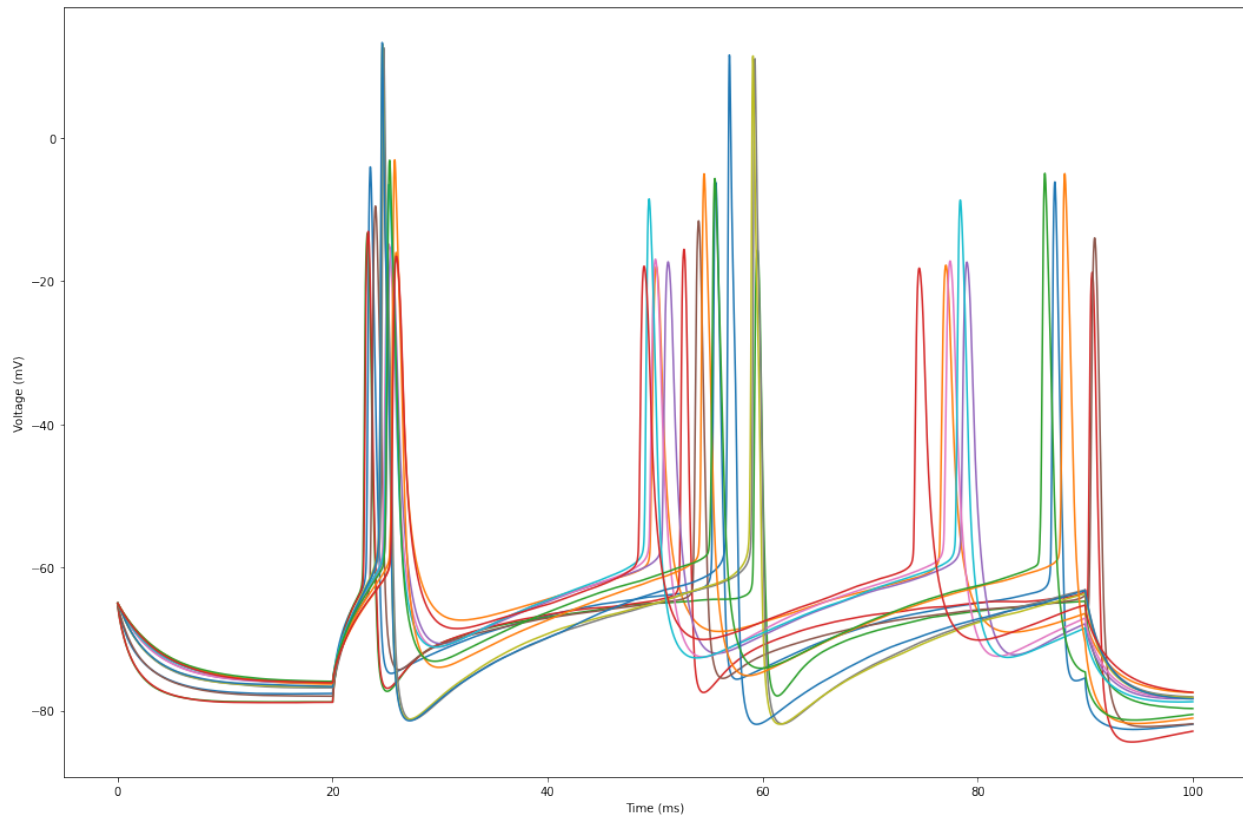
```
simulation_config = Path("./") / "simulation_config.json"
with open(simulation_config) as f:
    simulation_config_dict = json.load(f)
sim = CircuitSimulation(simulation_config)
```

```
from bluepysnap import Simulation as snap_sim
snap_access = snap_sim(simulation_config)
import pandas as pd
from bluepysnap import Simulation as snap_sim
all_nodes = pd.concat([x[1] for x in snap_access.circuit.nodes.get()])
dstut_cells = all_nodes[all_nodes["etype"] == "dSTUT"].index.to_list()
```

```
sim.instantiate_gids(dstut_cells, add_stimuli=True)
t_stop = 100.0
sim.run(t_stop)
```

The plot displays the voltage traces simulated for each cell in our circuit.

```
plt.figure(figsize=(18, 12))
for cell_id in sim.cells:
    time = sim.get_time_trace()
    voltage = sim.get_voltage_trace(cell_id)
    plt.plot(time, voltage, label=cell_id)
plt.xlabel("Time (ms)")
plt.ylabel("Voltage (mV)")
```

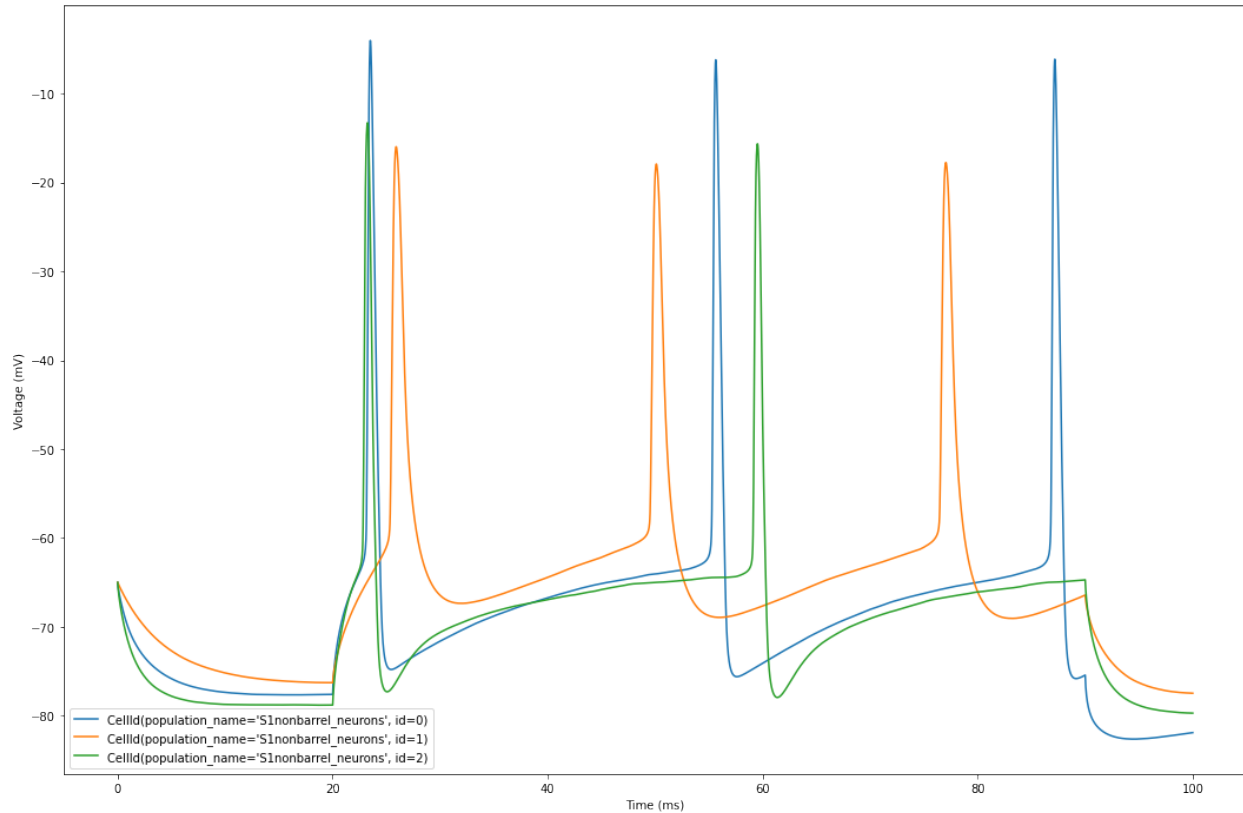


Let's focus on 3 cells for better visualization

```

sim.cells = dict(list(sim.cells.items())[:3])
plt.figure(figsize=(18, 12))
for cell_id in sim.cells:
    time = sim.get_time_trace()
    voltage = sim.get_voltage_trace(cell_id)
    plt.plot(time, voltage, label=cell_id)
    plt.xlabel("Time (ms)")
    plt.ylabel("Voltage (mV)")
    plt.legend()

```



We are now ready to extract features. First, we will build the data structure for eFEL

```

traces = []
for cell_id in sim.cells:
    voltage = sim.get_voltage_trace(cell_id)
    trace = {}
    trace['T'] = time
    trace['V'] = voltage
    trace['stim_start'] = [20]
    trace['stim_end'] = [90]
    traces.append(trace)

```

Next, we choose the specific features of interest

```

features = ['peak_time', 'AHP_time_from_peak', 'AP_height', 'AHP_depth_abs', 'all_ISI_
↪ values']

```

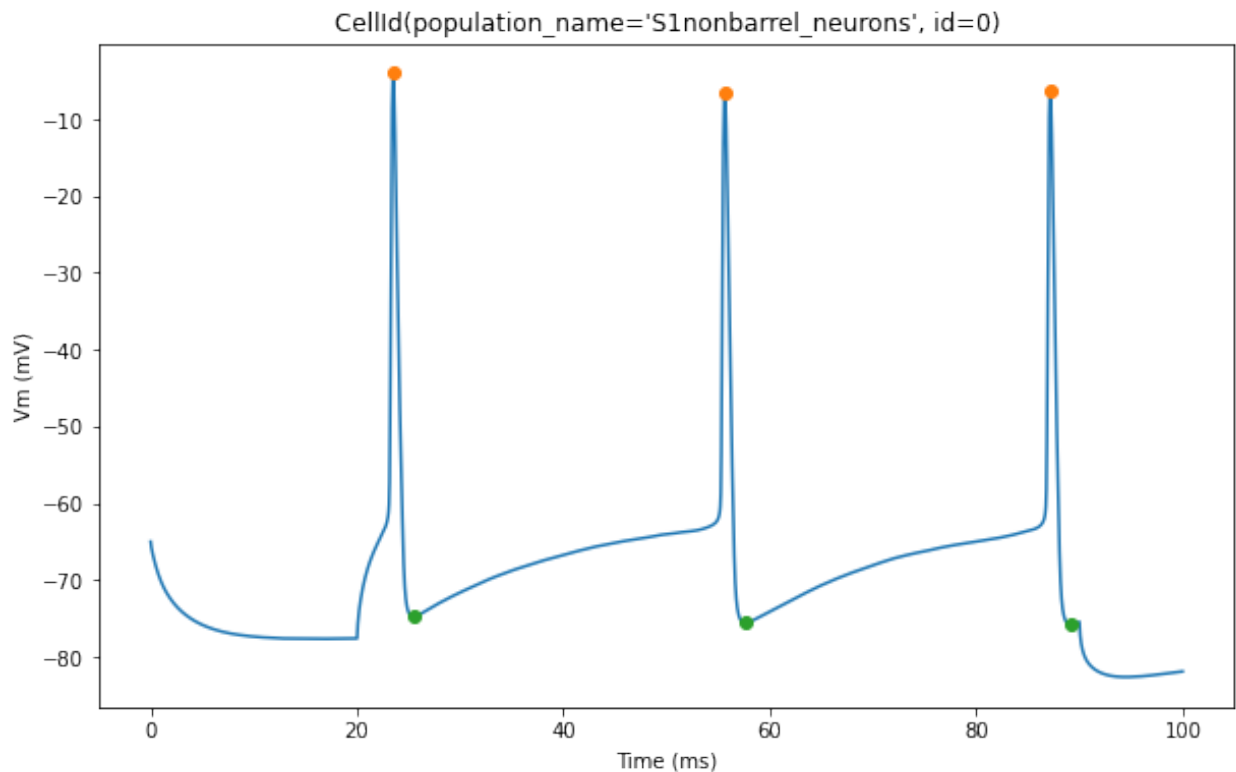
Finally, we perform the feature extraction

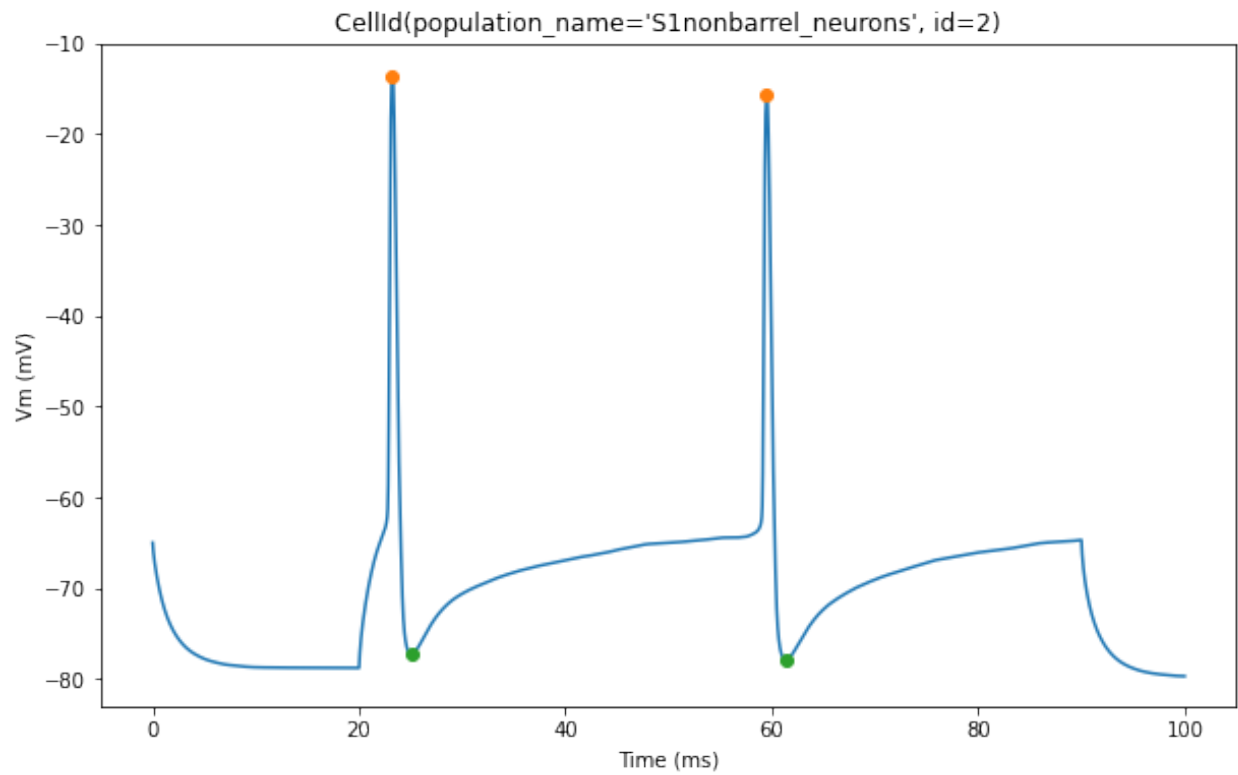
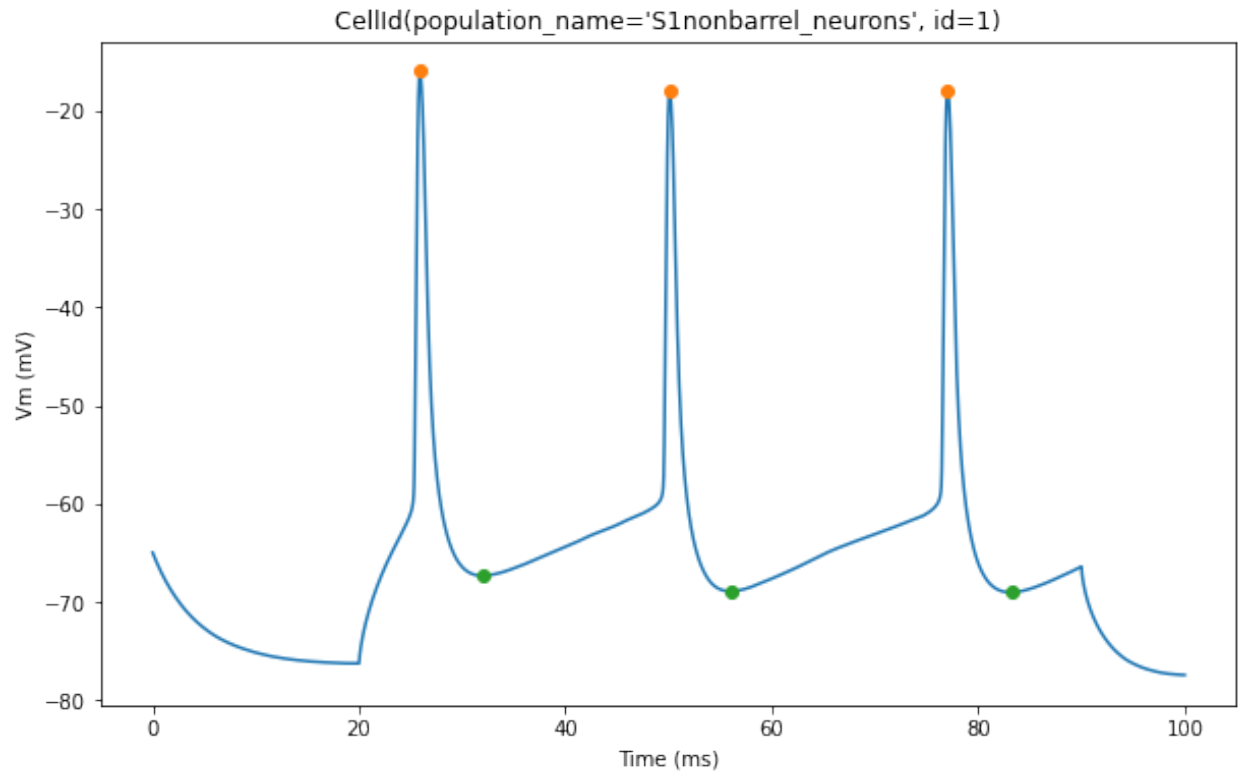
```
traces_results = efel.get_feature_values(traces, features)
```

The plot below shows action potential (AP) height and depth of those 3 cells

```
import pylab
for trace, trace_result, cell_id in zip(traces, traces_results, sim.cells):
    time = trace['T']
    voltage = trace['V']
    peak_times = trace_result['peak_time']
    ahp_time = trace_result['AHP_time_from_peak']
    ap_heights = trace_result['AP_height']
    AHP_depth_abss = trace_result['AHP_depth_abs']

    pylab.figure(figsize=(10, 6))
    pylab.title(cell_id)
    pylab.plot(time, voltage)
    pylab.plot(peak_times, ap_heights, 'o')
    pylab.plot(peak_times+ahp_time, AHP_depth_abss, 'o')
    pylab.xlabel('Time (ms)')
    pylab.ylabel('Vm (mV)')
    pylab.show()
```





Now, let's overlay the durations of the inter-spike intervals (ISIs) for a clearer visualization of the timing between spikes

```

for trace, trace_result, cell_id in zip(traces, traces_results, sim.cells):
    time = trace['T']
    voltage = trace['V']
    peak_times = trace_result['peak_time']
    ap_heights = trace_result['AP_height']

    all_isi_values = trace_result['all_ISI_values']

    pylab.figure(figsize=(10, 6))
    pylab.title(cell_id)
    pylab.plot(time, voltage, label='Voltage Trace')
    pylab.plot(peak_times, ap_heights, 'o', label='Spike Peaks')

    for i in range(len(peak_times) - 1):
        start_spike_time = peak_times[i]
        end_spike_time = peak_times[i + 1]
        duration = round(all_isi_values[i], 2)

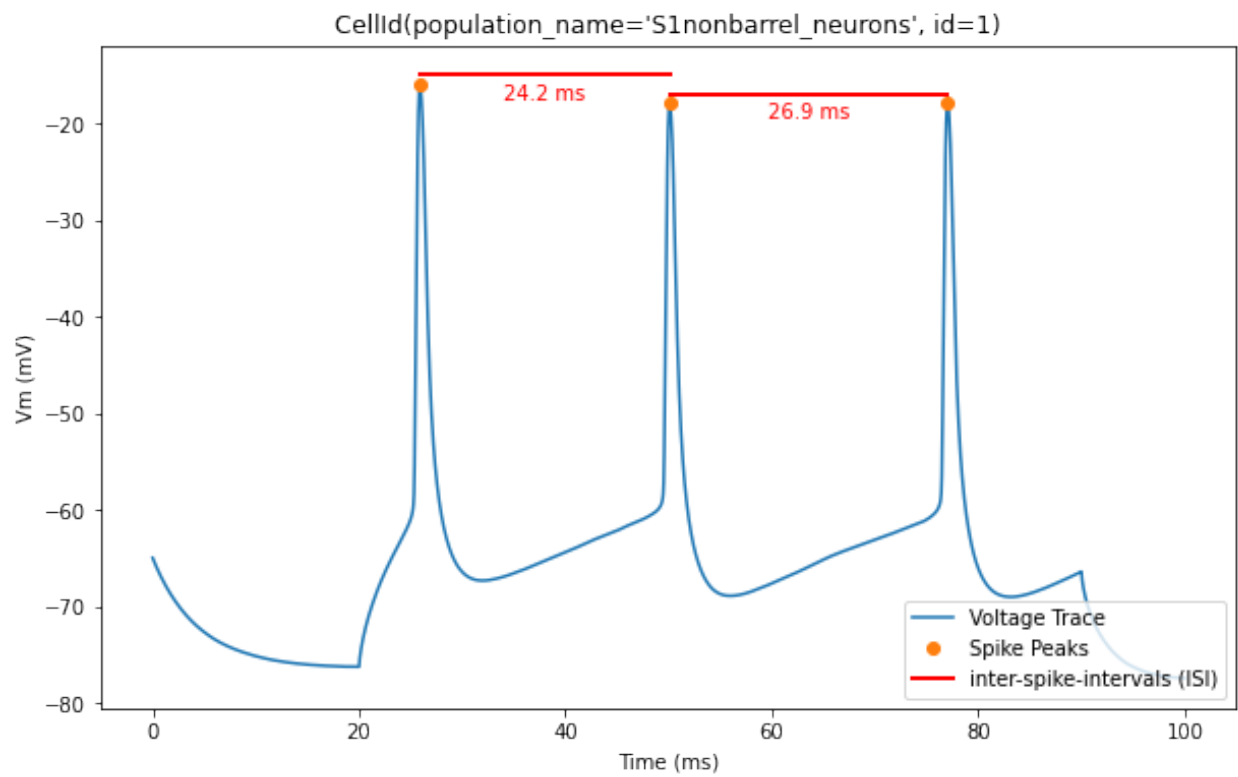
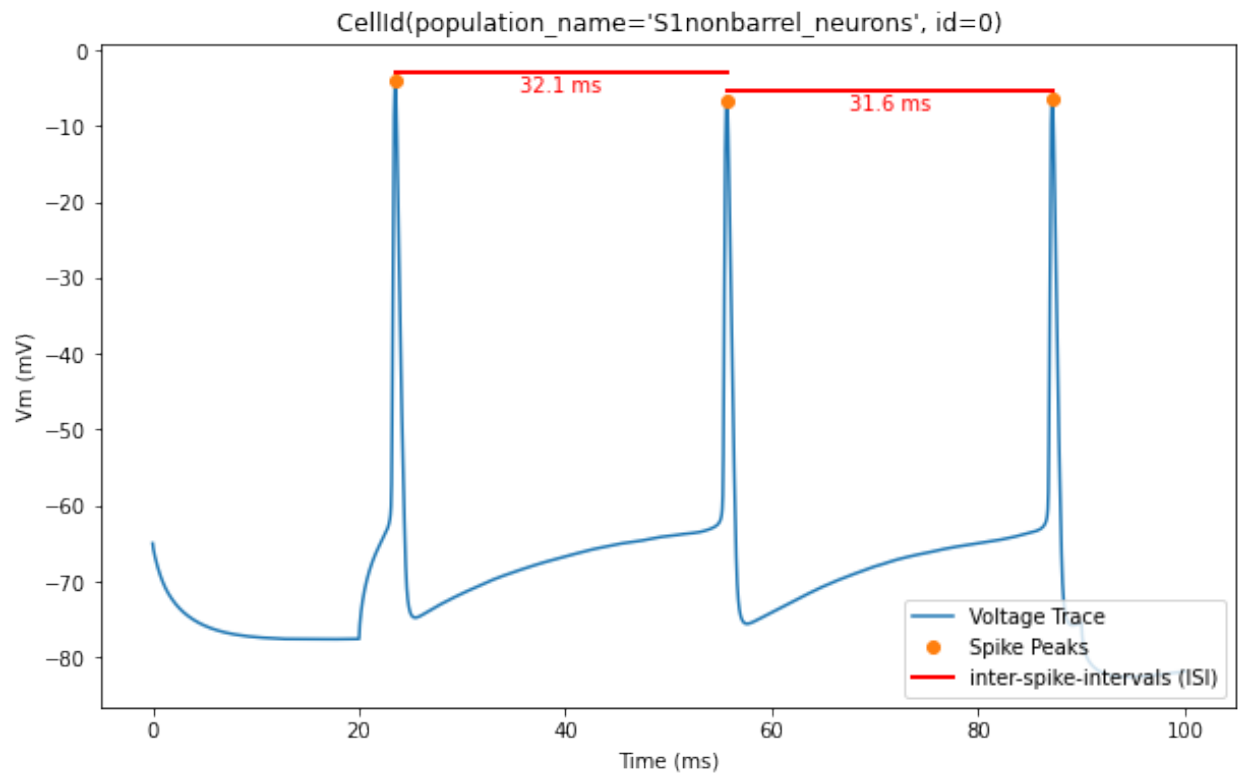
        y_position = max(ap_heights[i], ap_heights[i + 1]) + 1

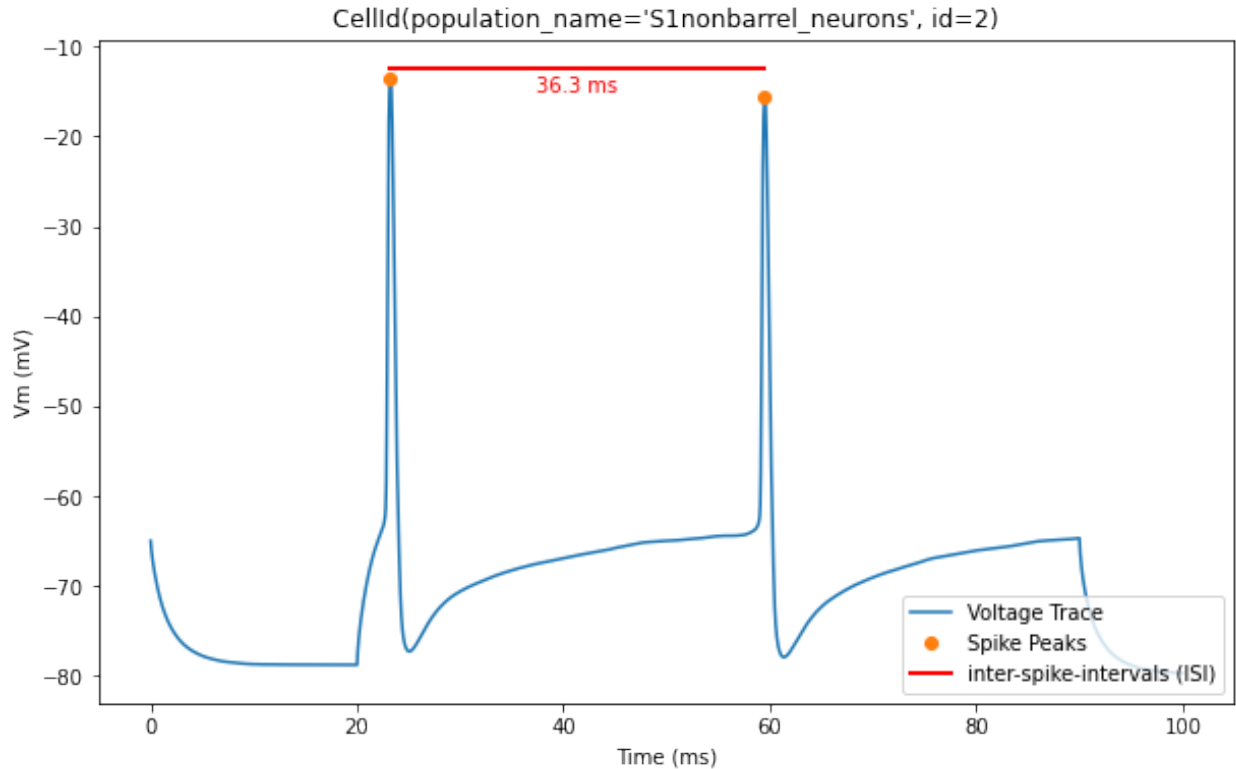
        # Check if it's the first ISI line to be drawn and add a label, otherwise draw
        ↪ without a label
        if i == 0:
            pylab.plot([start_spike_time, end_spike_time], [y_position, y_position], 'r-
            ↪', lw=2, label='inter-spike-intervals (ISI)')
        else:
            pylab.plot([start_spike_time, end_spike_time], [y_position, y_position], 'r-
            ↪', lw=2)

        # Adjust text position to be slightly lower
        midpoint = (start_spike_time + end_spike_time) / 2
        pylab.text(midpoint, y_position - 3, f'{duration} ms', verticalalignment='bottom
        ↪', horizontalalignment='center', color='red')

    pylab.xlabel('Time (ms)')
    pylab.ylabel('Vm (mV)')
    pylab.legend(loc='lower right')
    pylab.show()

```





2.9 Extracellular Features Extraction for MEA Data

This example shows how to use extracellular features implemented in eFEL for recordings from Microelectrode Arrays (MEA). Also see [Buccino et al., 2024](#) and another [example](#) using these efeatures for more details.

```
from efel.pyfeatures import extrafeats
import numpy as np
```

Currently, the following features are implemented in eFEL:

```
extrafeats.all_1D_features
```

```
['peak_to_valley',
 'halfwidth',
 'peak_trough_ratio',
 'repolarization_slope',
 'recovery_slope',
 'neg_peak_relative',
 'pos_peak_relative',
 'neg_peak_diff',
 'pos_peak_diff',
 'neg_image',
 'pos_image']
```

These features are described as functions in `extrafeats.py`. You can find the docstrings of these functions by using the `help` function in Python. You can find more documentation of the features here: <https://efel.readthedocs.io/en/latest/>

```
help(extrafeats.peak_to_valley)
```

Help on function peak_to_valley in module efel.pyfeatures.extrafeats:

```
peak_to_valley(waveforms, sampling_frequency)
    Time between trough and peak. If the peak precedes the trough,
    peak_to_valley is negative.

Parameters
-----
waveforms : numpy.ndarray (num_waveforms x num_samples)
    waveforms to compute feature for
sampling_frequency : float
    rate at which the waveforms are sampled (Hz)

Returns
-----
np.ndarray (num_waveforms)
    peak_to_valley in seconds
```

Let's get the test MEA data and calculate the extracellular features.

```
waveforms_fpath = "../tests/testdata/extrafeats/mean_waveforms.dat"

waveforms = np.loadtxt(waveforms_fpath)
waveform = np.array([waveforms[0]])
sampling_freq = 10000
```

```
extrafeats.peak_to_valley(waveforms=waveform, sampling_frequency=sampling_freq)
```

```
array([0.0013])
```

The waveform should have a structure similar to the one in eFEL/tests/testdata/extrafeats/mean_waveforms.dat. We will use the calculate_features function from extrafeats to calculate all the features.

```
feats = extrafeats.calculate_features(
    waveforms, sampling_freq
)
for feature_name in extrafeats.all_1D_features:
    print(feature_name, feats[feature_name])
```

```
peak_to_valley [ 0.0013 -0.0001 -0.0016 -0.0013 -0.0011 -0.0013 -0.0012  0.0009 -0.0011
-0.0002 -0.0004 -0.0008  0.0031 -0.0018  0.0016 -0.0007 -0.0009  0.0009
-0.0007 -0.0018 -0.0013 -0.0005 -0.0015 -0.0011  0.0013 -0.0034  0.0018
 0.001 -0.0019 -0.0015 -0.0014  0.0002 -0.0014 -0.0012 -0.0016  0.0008
 0.0014 -0.0005 -0.0025  0.0009  0.0014 -0.0016 -0.0013  0.0009 -0.0008
-0.0016 -0.0016 -0.0019 -0.0013 -0.0014 -0.0011  0.0003 -0.0014 -0.0014
-0.001 -0.0013 -0.0006 -0.0024  0.0018 -0.0035 -0.0012 -0.0012  0.0002
-0.0013 -0.0023 -0.0002 -0.0001  0.0032  0.0024 -0.0013  0.0029 -0.0006
 0.001 -0.001  0.0013  0.0022  0.0009 -0.0014 -0.0014  0.0018  0.0007
 0.0008 -0.0017  0.0014  0.0007 -0.0016 -0.0008  0.0027 -0.0021 -0.0012
-0.0015 -0.0001 -0.0001  0.0009  0.0016 -0.0001  0.0006  0.0015  0.0011
```

(continues on next page)

(continued from previous page)

```

-0.0009 0.0012 -0.0012 0.0013 -0.0012 0.0007 -0.0023 -0.0009 -0.0013
-0.0002 -0.001 -0.0011 -0.0012 -0.0015 -0.0011 -0.0012 -0.0014 0.0003
0.0009 0.0009 0.0006 0.0002 0.0008 -0.0006 0.0034 0.0014 -0.0013
-0.0011 -0.0008 0.0007 -0.0011 -0.0013 0.0009 -0.0013 0.0009 0.0009
-0.0018 -0.0012 0.0002 -0.0013 -0.0019 -0.0002 -0.0041 0.0009 -0.0009
-0.0006 -0.001 -0.0015 0.0009 -0.0012 -0.002 -0.0008 -0.0032 0.0015
0.0021 -0.0002 -0.0005 -0.0009 -0.0008 -0.001 -0.0013 0.0014 -0.001
0.0009 -0.0013 -0.0017 0.0021 -0.0016 -0.0011 -0.0014 -0.0019 0.0011
-0.0016 -0.0002 -0.001 -0.0018 -0.0015 -0.0001 0.002 0.0009 -0.0015
-0.0014 -0.0017 0.0009 -0.0007 0.0013 -0.0024 -0.0013 -0.0037 -0.0002
-0.0007 -0.0006 -0.0011 -0.0016 -0.0013 -0.0001 -0.001 -0.0014 -0.0015
-0.0005 -0.0004 -0.0015 -0.0001 -0.0015 -0.0011 0.0002 -0.0013 -0.0012
-0.0011 -0.0013]
halfwidth [ 0.0015 nan -0.0028 -0.0093 -0.0025 -0.0017 -0.0018 0.0003 -0.0024
nan nan -0.0055 0.0017 -0.003 0.0006 -0.0017 -0.0014 nan
-0.0011 nan -0.0025 -0.0068 -0.003 -0.0024 0.0007 -0.0054 0.0007
nan -0.0035 nan -0.0023 nan -0.0026 -0.0019 -0.0029 nan
nan nan -0.0039 nan nan -0.0029 -0.0025 nan -0.0017
-0.0026 nan -0.0035 -0.0023 -0.0018 -0.0017 nan -0.0028 nan
-0.0016 -0.0023 nan -0.0042 0.0008 -0.0045 -0.0022 -0.0019 nan
-0.0027 -0.0026 -0.0008 nan 0.0009 0.0008 nan 0.0014 nan
0.0005 -0.0017 0.0007 0.0015 0.0012 -0.002 -0.0019 0.0012 0.0004
0.0009 -0.0033 0.0006 0.0005 -0.0031 -0.0018 0.0006 -0.0038 -0.0024
-0.0025 nan nan 0.002 0.0006 nan nan 0.0005 0.0006
-0.0022 nan -0.0026 nan -0.0025 nan -0.0039 -0.0019 -0.0018
nan -0.0016 -0.0015 -0.0024 -0.003 -0.0023 nan -0.002 nan
nan nan nan nan nan nan 0.0019 nan -0.0018
-0.0017 -0.002 0.0005 -0.0017 -0.0023 0.002 -0.0067 0.0005 nan
nan -0.0019 nan -0.0022 -0.0036 nan -0.0063 0.0005 -0.0015
-0.0018 -0.0023 -0.0019 nan -0.0026 -0.0037 -0.0019 -0.0051 0.0006
0.0008 nan nan -0.0019 -0.002 -0.0016 -0.0027 0.0007 -0.0024
nan -0.002 -0.003 0.0007 -0.0031 -0.0024 nan -0.0036 0.0014
nan nan -0.0016 -0.0035 -0.0081 nan 0.0005 0.0003 -0.0024
-0.0028 nan 0.0007 -0.0018 nan -0.0046 -0.0024 -0.0059 nan
-0.0019 -0.0009 -0.0018 -0.003 -0.0024 nan -0.0016 -0.0028 -0.0029
-0.0073 -0.0044 -0.0025 nan -0.003 nan nan -0.0019 -0.0018
-0.0048 -0.0023]
peak_trough_ratio [ 0.53804035 1.21802672 2.36845269 0.13818893 2.01288236 0.
67247375
0.71054337 0.51431686 1.2270848 6.12281799 0.16201052 0.52271024
0.29541054 2.10863154 0.16400563 0.71021801 0.82259676 0.80606722
0.59820375 1.67405533 1.22329924 0.19102443 2.6186373 1.35055906
0.42408852 6.55820515 0.16284988 0.93212375 3.95194239 0.61651983
4.12676067 0.7973423 0.82446904 0.63280487 0.94402227 0.83385778
0.31889983 0.40030984 13.01325066 0.41722616 1.08734046 1.28142004
1.31052961 0.65915915 0.59007358 8.24318384 1.05782205 3.37112047
0.54651881 0.73548706 1.13247942 2.20866198 2.01767528 0.97482789
0.47452749 0.6706796 6.99816194 5.11126778 0.18733708 6.95707896
0.48524825 0.60336188 0.9870102 2.79221104 9.70193971 0.82660372
1.19167412 0.21163596 0.20311777 0.72466183 0.14251774 1.44321506
0.53334603 0.90387643 0.41733713 0.24190003 0.53594822 0.73317705
0.69903345 0.31630318 0.64635683 0.97632033 2.84277294 0.57842661

```

(continues on next page)

(continued from previous page)

```

0.67426552 3.12048394 0.65460616 0.18484049 4.40800015 3.50836316
2.37435616 2.72846297 0.81115791 0.57493701 0.39853579 1.71386828
5.21433688 0.16672223 0.49929396 1.67457084 1.5233474 2.58614124
0.66188939 2.5763163 1.11925818 4.77781519 0.53758802 0.36358919
3.00864117 0.96411726 0.45938812 2.93000602 2.33622968 1.09317424
0.49026222 1.13618701 1.50261237 1.095218 0.38294177 0.8881845
3.26153958 0.8232666 0.5709424 0.22638494 2.34391786 0.59851957
0.40645699 1.3843255 0.75217496 0.58369778 0.85124889 0.5570616
0.33895087 0.48671009 0.6832888 2.75731074 0.58309425 7.37870261
0.62792104 4.58416164 6.30390847 4.72034757 0.59310884 0.67547227
0.22271289 1.72432913 0.28557396 0.69169406 1.80447415 4.37127311
0.96403529 9.13984054 0.36332231 0.18233296 0.81281747 0.39035498
1.56471095 1.34022733 0.79437089 4.12955594 0.39527652 2.11227852
0.73075634 1.26226321 1.79751798 0.1394111 2.89838121 1.93914669
0.69080368 3.72053666 0.32107693 1.08166173 1.62173755 0.83715129
3.76741793 0.19781369 1.34602015 0.22818912 0.85238754 5.67776085
7.99932939 2.45151786 0.3190137 1.05551016 1.1821116 3.85214816
1.04723524 4.71443887 0.30886562 0.39164463 0.41260801 0.51914942
3.12387873 1.49623728 1.28426178 0.87490264 2.41248035 2.18445792
0.24783548 0.2482083 3.15501273 1.11067998 2.65673549 0.37181242
6.12408649 0.64401332 1.50091732 0.48633446 0.72905253]
repolarization_slope [7.31257213e+01 nan 6.50182478e-01 4.99764537e+01
1.01266031e+00 1.22505405e+01 3.66794226e+00 3.96887490e+01
2.08781290e+00 nan 2.66397081e+01 3.32739630e+01
3.99171396e+01 1.18709472e+00 1.89150163e+02 5.33856699e+00
4.44666871e+00 nan 6.59520145e+00 1.35164366e+00
2.82988737e+00 2.03543510e+01 4.85872501e-01 1.38560311e+00
8.93961074e+01 1.15821548e-01 9.74608965e+01 4.35300936e+01
2.48036396e-01 5.35377718e+00 4.19533385e-01 nan
3.87647107e+00 2.49081159e+01 2.53626726e+00 3.08366795e+02
1.12289446e+02 7.72393222e+00 1.02580692e-01 nan
5.83851918e+00 1.57677921e+00 1.92203478e+00 6.57320993e+01
1.54010808e+01 1.55766806e-01 2.41955804e+00 3.11922659e-01
1.52125356e+01 3.20115632e+00 4.41163627e+00 nan
7.79157923e-01 2.62026144e+00 6.73181141e+01 7.50472345e+00
nan 1.36751428e-01 5.34852086e+01 2.24802060e-01
5.15398199e+01 2.30980102e+01 nan 5.65438604e-01
3.84641022e-01 nan nan 1.90051711e+01
5.68442402e+01 6.28630703e+00 1.20718467e+01 nan
8.79612883e+01 3.09045447e+00 1.36219748e+02 1.07188996e+01
1.19465006e+02 3.74992327e+00 5.11446110e+00 3.21324376e+01
2.96762385e+02 2.87781921e+02 4.56945190e-01 1.36938154e+02
2.44544112e+02 3.82173278e-01 4.28763535e+00 1.25128169e+01
1.69948984e-01 4.05290769e-01 7.46124701e-01 nan
nan 6.40825688e+01 7.90614702e+01 nan
nan 5.41251231e+02 5.78885301e+01 1.34395745e+00
nan 7.11621115e-01 4.54818048e+01 6.68880108e-01
nan 1.63878756e-01 9.48684456e+00 1.27452569e+01
nan 7.83978570e+00 2.70935240e+01 5.88313421e-01
6.93437974e-01 2.65871059e+00 2.13100092e+01 2.53094985e+00
nan nan 1.06479171e+02 nan
nan 1.97160273e+02 6.90330690e+00 4.15423798e+00

```

(continues on next page)

(continued from previous page)

```

nan 7.32781106e+00 9.63477588e+01 1.89820897e+00
2.54461786e+02 6.19672821e+01 4.78810459e+00 1.05531879e+02
1.04548966e+01 1.24175790e+02 6.02423181e+01 9.37029728e-01
3.43643506e+01 nan 1.24964841e+01 1.56175850e-01
nan 1.77657282e-01 1.04539302e+02 2.10076218e+01
1.22608385e+01 1.41715843e+00 6.93772653e+00 1.63626246e+01
1.10691890e+00 1.69822463e-01 2.90615132e+00 9.16422355e-02
8.81610344e+01 3.49923843e+01 nan 1.45976613e+01
9.41618161e-01 1.76125600e+00 1.09905341e+01 3.63785992e-01
3.26088462e+01 9.29907719e-01 7.64039846e+01 2.86223544e+00
8.74881544e-01 4.62083858e+01 4.79718173e-01 1.10037652e+00
7.23525875e+00 2.33396674e-01 6.76706409e+01 2.92417470e+00
nan 1.39168207e+01 2.47310145e-01 2.75639683e+01
nan 1.20943344e+02 1.26141937e+01 3.12423625e-01
1.69656396e-01 1.80390321e+00 2.74981969e+01 2.43878552e+00
nan 2.14533667e-01 2.97934250e+00 1.55177673e-01
nan 6.74687462e+00 1.16212026e+01 6.70505162e+01
4.12664511e-01 2.44764617e+00 nan 9.28700444e+00
8.35177563e-01 7.86531878e-01 7.90299729e+00 2.12171085e+01
5.12327842e-01 nan 5.99487790e-01 2.87914657e+01
nan 5.07647764e+00 3.12094982e+00 5.37494106e+01
8.01730656e+00]
recovery_slope [-3.63355521e+00 4.54342435e+00 -2.10494514e+00 -3.21612844e+01
-1.59355549e+01 -2.91941227e+01 -4.48096741e+00 -7.79253365e+00
-1.93142445e+01 -5.43590968e+00 -7.84073969e+01 -4.67504152e+01
-1.24537129e+00 9.04851649e-01 -9.23155491e+00 -5.20729298e+01
-2.88969544e+01 -1.92130265e+01 -4.31626734e+01 2.16413225e+00
-1.04642376e+01 -8.08408718e+01 -5.87632497e+00 -1.09565866e+01
-5.85332555e+00 -1.16608216e+00 -3.21579887e+00 -2.17739809e+01
-5.07126055e+00 -1.02362027e+01 -2.03299056e+00 -7.71686527e-01
-2.51163221e+00 -2.95649418e+01 -8.15932912e-01 -5.13599982e+01
-9.32113618e+00 -4.50290554e+01 -3.80578627e+00 -1.24285027e+01
-9.27802447e-01 -2.00340018e+00 -6.20759074e+00 -2.66042436e+01
-5.09533788e+01 -3.97681438e+00 -1.18503344e+00 -4.60929399e+00
-6.92211691e+00 -6.16165160e+00 -1.12842409e+01 -1.75193125e+00
-6.96007348e+00 -4.96107984e+00 -3.46614424e+01 -5.24159703e+00
-8.08372841e+00 -2.71314048e+00 -2.65011956e+00 5.62134860e-01
-1.78506949e+01 -1.81238510e+01 2.00340882e+00 -1.06992198e+01
-3.91580166e+00 -4.59364500e-01 2.75542585e+00 -3.77812716e-01
-1.39327173e+00 -1.32048883e+01 -1.86680577e-01 -3.93456185e+00
-7.64656083e+00 -1.68954306e+01 -4.34881443e+00 -8.69229998e-01
-1.19791230e+01 -6.67007742e+00 -1.23338715e+01 -3.92168564e+00
-3.26267482e+01 -4.89466829e+01 -5.50323478e+00 -4.06361556e+01
-2.04860872e+01 -6.46205930e+00 -2.42733005e+01 -3.29076079e-01
-2.18624572e+00 -8.64471967e+00 -2.57717988e+00 -4.83779739e-01
5.09670643e+00 -2.81096643e+01 -1.24310724e+01 -2.41915951e+00
-1.51126413e+00 -2.88979477e+01 -3.37945486e+00 -2.08947684e+01
-1.87378608e+01 -1.32479872e+01 -8.10451785e+00 -7.21654548e+00
-7.39351372e+00 -2.88267735e+00 -2.66157324e+01 -2.01121237e+01
-3.67737909e+00 -2.19363285e+01 -1.54238008e+01 -6.18020049e+00
-8.82350024e+00 -1.78646598e+01 1.80605389e+00 -4.14716330e+00
-5.43485978e+00 -1.06769351e+01 -5.30541741e+01 -2.97564361e+00

```

(continues on next page)

(continued from previous page)

```

-3.98414646e-01 -2.97626033e+01 -4.44445283e+01 -7.16824627e-02
-7.93094190e+00 -1.32033245e+01 -2.81984189e+01 -3.04757744e+01
-3.08283335e+01 -4.18498964e+01 -6.57149163e+00 -4.39076727e+01
-1.37147948e+01 -1.12276678e+01 -2.03882091e+01 1.19113164e+00
-1.87193042e+01 -4.29831701e+00 -1.10030127e+01 -2.22290578e+00
-5.84355534e+00 1.97905425e+00 -1.04236166e+01 -3.58815562e+01
-4.53173319e+01 -2.11364258e+01 -1.20790837e+01 -1.46042168e+01
-1.16542322e+01 -2.16928796e+00 -3.84443397e+01 -1.10930702e+00
-1.35325880e+01 -1.02588094e+00 4.08781011e+00 -6.78991393e+01
-9.65564853e+00 -3.02388143e+01 -2.85310794e+01 -5.89109388e+00
-1.32740623e+00 -1.60552191e+01 -3.65615502e+01 -1.23223306e+01
-1.19126869e+00 -1.38971257e+00 -7.08911891e+00 -1.72027697e+01
-1.10750593e+01 -2.40240059e+00 -3.18275518e+00 -2.23994382e+00
-2.46730697e+00 -2.53630321e+01 -4.32395820e+00 -3.46648203e+01
8.31871590e+00 -2.99608434e+00 -1.09693318e+01 -1.76399365e+00
-5.12560876e+00 -3.74529408e+00 -2.74762612e+01 -3.48078286e+01
-1.38484473e+01 -3.84538670e+00 -1.56589216e+00 -1.16947634e+00
1.22042755e+01 -3.35964001e+01 -6.04604562e+01 -3.74568162e+01
-7.22145075e+00 -7.70012328e+00 6.40803600e+00 -1.85702048e+01
-1.07774241e+01 -1.07239397e+01 -3.73939408e+01 -7.05625361e+01
-2.21224102e+00 8.72531968e-01 -8.22518343e+00 -5.93215146e+00
3.04381793e+00 -9.72311749e+00 -1.89302412e+01 -3.75853073e+01
-8.14213856e+00]
neg_peak_relative [9.51339213e-02 1.00388035e-02 3.28724224e-03 8.63910846e-02
1.23287678e-02 2.81335375e-02 1.35270547e-02 3.26901910e-02
1.75885984e-02 5.17095565e-03 1.10390041e-01 5.58803425e-02
8.93325636e-02 5.76913099e-03 4.77727744e-01 4.79924335e-02
2.80982170e-02 2.47240713e-02 4.01139776e-02 6.51939114e-03
2.71661034e-02 8.94666451e-02 4.32888964e-03 1.12655112e-02
1.44575946e-01 8.34988705e-04 2.75001421e-01 2.65226106e-02
2.80205172e-03 1.80731068e-02 1.04085591e-03 1.01575434e-02
2.66661308e-02 5.82650282e-02 1.86321664e-02 2.87389870e-01
5.76186146e-02 4.13010277e-02 6.84346712e-04 1.97046304e-02
3.54932940e-03 9.85326097e-03 1.91838933e-02 4.03909653e-02
5.65890648e-02 1.05026994e-03 1.00670272e-02 3.24793977e-03
7.57699886e-02 1.20001955e-02 1.74041886e-02 4.45264419e-03
6.35282086e-03 1.33618267e-02 1.03643170e-01 4.93752872e-02
3.23877510e-03 1.29991624e-03 1.79916050e-01 2.00675228e-03
9.85910621e-02 5.49954957e-02 1.24116089e-02 6.46973356e-03
1.52057421e-03 1.29378784e-02 1.64824978e-02 6.58365941e-02
1.47911176e-01 2.02344756e-02 5.88078649e-02 8.70456343e-03
1.11993825e-01 1.82114418e-02 1.15731751e-01 6.45806158e-02
1.49667367e-01 1.48037917e-02 1.83947157e-02 1.26785426e-01
2.93172551e-01 2.44271425e-01 4.66676594e-03 3.49910014e-01
2.49263853e-01 4.24394443e-03 2.39841963e-02 6.20606547e-02
1.40386954e-03 3.65911205e-03 2.80730871e-03 8.65798745e-03
1.67705222e-02 4.02185452e-02 2.58374175e-01 1.24129259e-02
1.19555561e-03 1.00000000e+00 6.84316052e-02 1.45336614e-02
1.38767710e-02 8.06665123e-03 2.41699581e-02 4.09648680e-03
1.80323783e-02 1.67985703e-03 3.35210216e-02 2.76382021e-02
7.01153240e-03 3.53932427e-02 7.93280312e-02 3.32079102e-03
6.57178672e-03 1.74822659e-02 7.32635742e-02 1.02818073e-02

```

(continues on next page)

(continued from previous page)

```

7.33690637e-03 1.53876936e-02 9.23369396e-02 1.60976328e-02
4.62550161e-03 1.96154390e-01 3.60257600e-02 2.74494341e-02
5.33309832e-03 2.13874320e-02 1.33876755e-01 2.17033620e-02
2.26948459e-01 8.75767315e-02 3.61205975e-02 6.40393055e-02
2.34164085e-02 1.65463342e-01 3.68799608e-02 4.15603502e-03
6.46803276e-02 3.75104615e-03 6.13998454e-02 1.21713189e-03
2.64842149e-03 1.76493118e-03 1.21477080e-01 4.50434043e-02
5.25971651e-02 1.65012657e-02 2.51654796e-02 2.87957457e-02
9.98412841e-03 1.41134268e-03 3.34588127e-02 4.90472664e-04
2.65779037e-01 1.21801997e-01 1.59984875e-02 6.99526537e-02
6.97657454e-03 2.18151914e-02 3.91343011e-02 2.43429319e-03
4.56749521e-02 1.15116091e-02 4.38027355e-02 2.05676271e-02
6.21842810e-03 1.65450507e-01 5.48683405e-03 1.23075447e-02
1.82412304e-02 1.99493478e-03 1.02296150e-01 1.20691796e-02
1.45060461e-02 3.25582148e-02 2.48795939e-03 5.73896552e-02
1.75166794e-02 2.37964517e-01 1.95533483e-02 6.73091533e-04
1.09128246e-03 7.45454447e-03 7.32739006e-02 2.68284733e-02
1.49532548e-02 2.31259318e-03 2.79921566e-02 1.46586883e-03
3.83552881e-02 3.49975742e-02 6.03611556e-02 9.86314676e-02
4.82697766e-03 1.07961656e-02 2.16553606e-02 2.87630514e-02
8.80652699e-03 9.10429364e-03 4.56176900e-02 9.03832156e-02
1.79535329e-03 1.43656137e-02 6.85879100e-03 7.56633815e-02
2.27921586e-03 1.62732679e-02 2.73694031e-02 7.65484421e-02
4.10695154e-02]
pos_peak_relative [0.2135929 0.0510241 0.03248875 0.04981713 0.10355585 0.07894711
0.04010794 0.07015922 0.09006216 0.13211701 0.07462935 0.12188679
0.11012156 0.05076304 0.32694581 0.14223321 0.09645002 0.08316255
0.10013397 0.04554214 0.1386745 0.07131594 0.04730297 0.06348937
0.25585218 0.02285084 0.18687832 0.10316358 0.04620865 0.04649609
0.01792406 0.03379637 0.09174257 0.15385596 0.07339765 1.
0.07667498 0.0689912 0.03716195 0.0343065 0.01610454 0.05268756
0.10491079 0.11109924 0.13933982 0.03612705 0.04443758 0.04568975
0.17279807 0.03682985 0.08224711 0.04103777 0.05348776 0.05435385
0.20522895 0.13818512 0.09458044 0.02772557 0.14064691 0.05825823
0.19963564 0.13846549 0.05111946 0.07538261 0.06156059 0.04462693
0.08196286 0.05814246 0.12536764 0.06118767 0.03497364 0.05242212
0.2492526 0.06868944 0.20154689 0.06518905 0.33472374 0.04529168
0.05365715 0.16734373 0.79073727 0.99517981 0.05535983 0.84458077
0.7013371 0.0552622 0.06551516 0.04786852 0.02582289 0.0535694
0.02781459 0.09857609 0.05676603 0.09649019 0.42968777 0.08877449
0.0260139 0.69571292 0.14257704 0.10155823 0.08821124 0.08705267
0.06675716 0.04404 0.08422089 0.03349178 0.0751975 0.04193313
0.08802781 0.14239239 0.15206981 0.04060192 0.06406717 0.07974871
0.14988324 0.04874789 0.0460041 0.07032515 0.14755189 0.05966254
0.06295323 0.67386839 0.08583057 0.02593089 0.05216252 0.05341627
0.22706821 0.12537236 0.71233235 0.21331104 0.12830632 0.14886287
0.03312025 0.33605371 0.10515533 0.04781911 0.15737927 0.11549654
0.16088252 0.02328274 0.06966803 0.03476469 0.30065283 0.12696245
0.04888147 0.11873369 0.02998889 0.08311491 0.07517914 0.02574406
0.13459844 0.01870639 0.40294832 0.09267362 0.05426359 0.11394637
0.04555257 0.122004 0.12972314 0.04194815 0.07533828 0.10146676
0.13357045 0.10833547 0.04664342 0.09625028 0.06636115 0.09959067

```

(continues on next page)

(continued from previous page)

```

0.052583 0.03097211 0.13705818 0.05447607 0.09816715 0.11373682
0.03911323 0.04737252 0.09838749 0.22659155 0.06954962 0.01594732
0.03642733 0.07625929 0.0975428 0.11816665 0.07376168 0.03717396
0.12232555 0.02883777 0.04943464 0.05719613 0.10392781 0.21367043
0.06292251 0.06740724 0.11605278 0.10501014 0.08865536 0.08299014
0.04717733 0.09361396 0.02363674 0.0665809 0.07603831 0.11739408
0.05824563 0.04373269 0.17141888 0.15534882 0.12494387]
neg_peak_diff [ 0.0009 0. 0.0018 0.0011 0.0019 0.0013 0.0014 0. 0.0019
0. 0.0003 0.0011 -0.0001 0.0018 0.0001 0.0014 0.0017 -0.0001
0.0015 0.0017 0.0018 0.0003 0.0024 0.0019 0.0008 0.0036 0.0001
-0.0001 0.0029 0.0014 0.0016 0. 0.0016 0.0013 0.0018 0.0007
0. 0.0004 0.0033 -0.0001 0.0002 0.0018 0.0018 -0.0001 0.0016
0.0023 0.0015 0.0029 0.0014 0.0014 0.0015 -0.0001 0.0022 0.0015
0.001 0.0014 0.0007 0.0035 0.0002 0.0033 0.0013 0.0013 -0.0001
0.0022 0.0025 0.0007 0. -0.0001 -0.0001 0.0014 0.0004 0.0006
0.0008 0.0018 0.0011 0.0003 0.0008 0.0014 0.0014 0.0001 0.0008
0.0007 0.0026 0.0001 0.0008 0.0026 0.0016 0.0004 0.0031 0.002
0.0017 -0.0001 0. -0.0001 0. 0.0002 0.0002 0. 0.0009
0.0017 -0.0002 0.0021 0. 0.0019 -0.0003 0.0033 0.0017 0.0014
0. 0.0014 0.0014 0.0019 0.0024 0.002 0.0015 0.0014 -0.0001
-0.0001 -0.0002 -0.0002 -0.0001 0.0007 0.0005 0.0005 -0.0002 0.0014
0.0012 0.0016 0.0008 0.0011 0.0015 -0.0001 0.0014 0.0008 -0.0001
0.0018 0.0013 -0.0001 0.0014 0.0029 0.0002 0.0039 0.0008 0.0013
0.0014 0.0018 0.0014 -0.0001 0.002 0.003 0.0015 0.0034 0.0001
0.0003 -0.0001 0.0004 0.0016 0.0016 0.0018 0.002 0.0009 0.0018
-0.0001 0.0017 0.0019 0.0002 0.0025 0.002 0.0015 0.0028 0.0009
0.0015 0.0004 0.0013 0.0028 0.0013 -0.0002 -0.0001 0. 0.0017
0.0022 0.0017 -0.0001 0.0015 -0.0002 0.0036 0.0015 0.0039 0.0004
0.0015 0.0014 0.0012 0.0025 0.0017 -0.0002 0.0014 0.0022 0.0024
0.0004 0.0003 0.0017 0.0002 0.0024 0.0014 -0.0001 0.0014 0.0017
0.0011 0.0014]
pos_peak_diff [ 0.0007 -0.0016 -0.0013 -0.0017 -0.0007 -0.0015 -0.0013 -0.0006 -0.0007
-0.0017 -0.0016 -0.0012 0.0015 -0.0015 0.0002 -0.0008 -0.0007 -0.0007
-0.0007 -0.0016 -0.001 -0.0017 -0.0006 -0.0007 0.0006 -0.0013 0.0004
-0.0006 -0.0005 -0.0016 -0.0013 -0.0013 -0.0013 -0.0014 -0.0013 0.
-0.0001 -0.0016 -0.0007 -0.0007 0.0001 -0.0013 -0.001 -0.0007 -0.0007
-0.0008 -0.0016 -0.0005 -0.0014 -0.0015 -0.0011 -0.0013 -0.0007 -0.0014
-0.0015 -0.0014 -0.0014 -0.0004 0.0005 -0.0017 -0.0014 -0.0014 -0.0014
-0.0006 -0.0013 -0.001 -0.0016 0.0016 0.0008 -0.0014 0.0018 -0.0015
0.0003 -0.0007 0.0009 0.001 0.0002 -0.0015 -0.0015 0.0004 0.
0. -0.0006 0. 0. -0.0005 -0.0007 0.0016 -0.0005 -0.0007
-0.0013 -0.0017 -0.0016 -0.0007 0.0001 -0.0014 -0.0007 0. 0.0005
-0.0007 -0.0005 -0.0006 -0.0002 -0.0008 -0.0011 -0.0005 -0.0007 -0.0014
-0.0017 -0.0011 -0.0012 -0.0008 -0.0006 -0.0006 -0.0012 -0.0015 -0.0013
-0.0007 -0.0008 -0.0011 -0.0014 0. -0.0016 0.0024 -0.0003 -0.0014
-0.0014 -0.0007 0. -0.0015 -0.0013 -0.0007 -0.0014 0.0002 -0.0007
-0.0015 -0.0014 -0.0014 -0.0014 -0.0005 -0.0015 -0.0017 0.0002 -0.0011
-0.0007 -0.0007 -0.0016 -0.0007 -0.0007 -0.0005 -0.0008 -0.0013 0.0001
0.0009 -0.0018 -0.0016 -0.0008 -0.0007 -0.0007 -0.0008 0.0008 -0.0007
-0.0007 -0.0011 -0.0013 0.0008 -0.0006 -0.0006 -0.0014 -0.0006 0.0005
-0.0016 -0.0013 -0.0012 -0.0005 -0.0017 -0.0018 0.0004 -0.0006 -0.0013
-0.0007 -0.0015 -0.0007 -0.0007 -0.0004 -0.0003 -0.0013 -0.0013 -0.0013

```

(continues on next page)

(continued from previous page)

```

-0.0007 -0.0007 -0.0014 -0.0006 -0.0011 -0.0018 -0.0011 -0.0007 -0.0006
-0.0016 -0.0016 -0.0013 -0.0014 -0.0006 -0.0012 -0.0014 -0.0014 -0.001
-0.0015 -0.0014]
neg_image [ 6.08446809e-02 1.00388035e-02 -6.05265898e-03 1.78245655e-02
-1.24821376e-02 -1.89190656e-02 3.08945388e-03 3.26901910e-02
-1.43230784e-02 5.17095565e-03 1.29688093e-02 -2.57703947e-02
6.39995784e-02 -1.21649715e-02 4.62402535e-01 -2.66267054e-02
-5.11740429e-03 1.74402736e-02 -5.07638723e-03 -9.52536749e-04
-1.86204841e-02 2.88399853e-02 -6.97229695e-03 -1.06917556e-02
-5.48151898e-02 -2.11465317e-03 2.35417100e-01 2.28362432e-02
-5.24303852e-03 -9.99384449e-03 -3.55590704e-03 1.01575434e-02
-1.70972605e-02 -3.36730026e-02 -1.28623455e-02 -7.87008393e-02
5.76186146e-02 -1.48221564e-02 -6.83807751e-03 1.14414297e-02
-1.18791361e-03 -9.58097136e-03 -1.66246015e-02 2.66268916e-02
1.25091639e-02 -4.03529947e-03 -7.10377009e-03 -5.44089702e-03
-3.53217079e-02 -8.82598847e-03 -1.31622694e-02 -2.53898707e-03
-8.64855042e-03 2.80748180e-03 -4.91815329e-02 -2.77268095e-02
-6.11335231e-03 -3.34263389e-03 1.15050169e-01 2.83327962e-04
-4.34206887e-02 -3.09019508e-02 7.93579896e-03 -8.61473952e-03
-2.30328131e-03 -4.93573286e-03 1.64824978e-02 4.16151399e-02
1.13755709e-01 3.73595986e-03 -3.47312106e-03 -1.25625570e-02
-3.13228925e-02 -5.06339728e-03 -3.39662070e-02 3.01342044e-02
7.55061262e-02 -1.08538003e-02 -1.28585217e-02 1.00740279e-01
-1.01669584e-01 1.04805660e-01 -6.86061335e-03 3.19225849e-01
-6.08905680e-02 -6.03417942e-03 -6.53320778e-03 1.88661022e-02
-3.92897568e-03 -1.15966324e-03 -5.42422620e-03 6.58619445e-03
1.67705222e-02 2.54909612e-02 2.58374175e-01 -4.02927754e-03
-5.09458215e-03 1.00000000e+00 -1.13930148e-02 -1.47001049e-02
-1.11415553e-02 -9.83958346e-03 2.41699581e-02 -5.37060702e-03
-1.33439461e-02 -3.99577596e-03 9.69961083e-03 -4.04012984e-03
7.01153240e-03 -2.28320677e-02 -1.07702337e-02 -6.36215860e-03
-8.74680377e-03 -4.18488252e-03 3.77310393e-03 -1.16820558e-02
-1.93667216e-03 1.28458224e-02 4.37043940e-02 -8.55250833e-03
1.33614096e-03 -2.74869251e-02 -1.85565685e-02 5.79815554e-03
-4.91421357e-03 -1.06350172e-02 -5.28184043e-02 -1.84791937e-02
5.41836128e-02 -5.11183439e-02 -2.41262529e-02 3.29196935e-02
-4.87750652e-03 -5.57221094e-02 2.77128268e-02 -1.14594800e-02
-3.51176076e-02 -1.63332712e-03 -3.27090474e-02 -3.41422013e-03
-1.66954067e-02 -1.17571147e-03 7.65362183e-03 -2.54347316e-02
-7.01149193e-03 -1.48849940e-02 -4.70009869e-03 2.29834419e-02
-1.12293652e-02 -3.77008421e-03 -2.28329429e-02 -2.20408006e-03
2.48999662e-01 7.39586173e-02 4.43774582e-04 -1.12027798e-02
1.50636568e-03 -1.77538970e-02 2.02450197e-02 -1.96999141e-03
-1.37745967e-02 -1.22174924e-02 2.60673799e-02 -1.66182621e-02
-8.21762118e-03 6.08330391e-02 -7.19549430e-03 -1.17398307e-02
2.55255787e-03 -4.71851271e-03 4.82709768e-02 1.93112806e-04
-1.16513332e-03 -1.94897140e-02 -4.81753971e-03 6.05581856e-04
-8.41006117e-03 1.80700429e-01 1.95533483e-02 -3.12721720e-03
-8.65113939e-03 -1.82749489e-02 4.32173811e-02 -1.97172171e-02
-7.71759605e-03 -3.01673378e-03 -2.17012063e-02 -2.14954356e-03
-1.35687955e-03 -7.51241270e-03 -4.17973323e-03 -4.94607047e-02
-7.16576989e-03 -1.02791855e-02 -5.18789896e-03 -1.76493478e-02

```

(continues on next page)

(continued from previous page)

```

-1.07474676e-02 -9.33063939e-03  4.88762801e-03  9.14357372e-03
-4.57420933e-03 -3.35417773e-03 -7.99455792e-03  1.85335304e-02
-1.92801512e-03  1.02063490e-02 -1.86219774e-02 -3.72281451e-02
-2.57639530e-02]
pos_image [ 1.08501167e-01 -1.38519656e-02 -8.62022439e-03 -2.06040891e-01
-1.79766671e-02 -9.78701610e-02 -5.58041488e-02  1.84963418e-02
-4.93451784e-02  1.37370617e-02 -7.37622137e-02 -9.94313678e-02
-1.18824789e-01 -1.95720633e-02  2.81616741e-01 -1.94448349e-01
-1.06678543e-01 -5.87641944e-02 -1.67391070e-01 -2.47799004e-02
-9.71399793e-02 -6.68305654e-02  1.10565445e-02 -2.21346697e-02
 1.48155547e-01  1.30670198e-02  1.03530938e-01 -3.37769755e-02
 2.33584482e-02 -7.10758824e-02 -3.87808177e-03 -3.89553391e-02
-1.08534293e-01 -1.93999750e-01 -6.14097940e-02  1.00000000e+00
 6.82101090e-02 -9.59625210e-02  1.17900262e-02 -5.24792533e-02
 1.48255488e-02 -3.00570823e-02 -5.99721529e-02 -8.23330821e-02
-2.21169173e-01  4.99799369e-03 -4.20085603e-02  2.53395523e-02
-2.70302090e-01 -4.83977594e-02 -7.26257011e-02  2.77995504e-02
-1.21964756e-03 -5.57573843e-02 -1.24945457e-01 -1.94711921e-01
 1.01392275e-03  1.92395461e-02  3.55537231e-02  2.15808434e-02
-2.77599157e-01 -1.79099660e-01 -2.65259729e-02  9.48269964e-03
-1.13702423e-03  2.73471808e-03 -2.74834528e-02 -3.98016538e-02
 5.93752186e-02 -8.29345619e-02 -9.66860512e-02  3.57025466e-03
 2.08817380e-01 -5.76370788e-02  7.54080715e-05 -1.02562228e-01
 2.78335925e-01 -6.04362390e-02 -7.27901478e-02  6.75434944e-02
 7.90737268e-01  9.95179813e-01  2.12303735e-02  8.44580774e-01
 7.01337105e-01  2.62646044e-02 -9.73851748e-02 -7.26134305e-02
 1.62556610e-02 -6.24921544e-03 -9.93458290e-03  5.11836886e-03
-1.49627525e-02 -8.97409188e-02  4.15328294e-01 -2.76223181e-03
 1.60881718e-02  6.95712920e-01  7.71219698e-02 -4.72679225e-02
 6.96080174e-04  5.44668299e-03  5.80394053e-02 -9.98433280e-03
-2.95011461e-02  2.09944738e-02 -1.21975646e-01 -1.02535812e-01
 7.61346209e-03 -1.46825996e-01 -3.05124429e-01 -6.01563021e-03
 9.78195653e-03 -2.91949742e-02 -3.05720570e-01 -4.27620740e-02
 4.07094422e-03 -1.90400710e-02 -1.67519563e-01  7.73910420e-04
-1.91953275e-02  6.73868393e-01 -1.08789703e-01 -8.49929088e-02
 2.97077167e-02 -8.15660190e-02 -2.73192579e-01 -8.31348899e-02
 7.12332350e-01 -1.29242840e-01 -1.50727149e-01 -1.46132154e-01
-8.65385479e-02  2.91433351e-01 -6.28626016e-02 -1.31243287e-02
-1.92992638e-01  1.57161322e-02 -2.28444937e-01  1.34367481e-02
 1.16189420e-02  2.34086650e-02  2.64145675e-01 -1.38216002e-01
-2.04069815e-01 -3.76140374e-02 -1.00001743e-01 -3.24994651e-02
-1.33672327e-02  1.63751749e-02 -1.39619823e-01  8.87501353e-03
 3.79814657e-01 -1.30388820e-02  1.38912513e-02 -9.41480201e-02
-2.86422267e-02 -8.54048672e-02 -1.08057469e-01 -2.75544036e-03
-2.71497552e-03 -2.06840393e-02 -1.36119696e-01 -7.42773676e-02
-1.03460912e-02 -3.48134643e-02  2.23164246e-02 -5.47293966e-03
-7.61185814e-02  1.59368359e-02 -8.36007928e-03 -5.03633149e-02
 1.37868462e-02 -1.25896227e-01  1.96586742e-02 -1.84120761e-01
 1.03126190e-02  1.72570998e-01  1.82149623e-03 -2.23764069e-03
 3.39241504e-03 -3.04666832e-02 -7.51461458e-02 -1.11952170e-01
 2.42018668e-02  2.77979044e-02 -1.16808089e-01  2.04911006e-02
 1.76164628e-02 -1.46040900e-01 -2.37644831e-01 -2.07194603e-01

```

(continues on next page)

(continued from previous page)

```

1.82503388e-02 -4.13795071e-02 1.30805556e-02 -1.14840546e-01
2.49402162e-03 1.65120438e-02 -1.12337528e-01 -8.06768107e-02
-5.84977379e-03 -4.73614935e-03 2.48863050e-02 -2.92409448e-01
3.56957519e-03 -6.70057126e-02 -1.06023022e-01 -9.88549097e-02
-1.63652624e-01]

```

Another example of extraction of these features from the experimental data can be found in the [multimodalfitting](#) repository.

2.10 Voltage clamp trace and eFEL

This notebook explains how to use traces from voltage clamp in eFEL. First, you need to load your modules and your trace data. Here, we will use experimental data from Channelpedia: <https://channelpedia.epfl.ch/expdata/details/9430>

Ranjan R, Logette E, Marani M, Herzog M, Tache V, Scantamburlo E, Buchillier V and [↪Markram H \(2019\) A Kinetic Map of the Homomeric Voltage-Gated Potassium Channel \(Kv\) ↪Family. Front. Cell. Neurosci. 13:358. doi: 10.3389/fncel.2019.00358](#)

Blue Brain Project Portal (<https://portal.bluebrain.epfl.ch>) and Channelpedia (<https://channelpedia.epfl.ch>)

```

import h5py
import numpy as np

from matplotlib import pyplot as plt
from scipy.optimize import curve_fit

import efel

```

2.10.1 Activation traces

We will start to load the data from the nwb file. In this notebook, we will use the traces from three experiments: Activation, Deactivation and Inactivation.

```

data_path = "../../tests/testdata/basic/rCell9430.nwb"

data = {}
exp_types = ["Activation", "Deactivation", "Inactivation"]
with h5py.File(data_path, "r") as content:
    for exp_type in exp_types:
        data[exp_type] = {}
        reps = content["acquisition"]["timeseries"][exp_type]["repetitions"]
        for rep_name, rep in reps.items():
            data[exp_type][rep_name] = {
                "dt": np.asarray(rep["x_interval"], dtype=np.float32),
                "current": np.asarray(rep["data"], dtype=np.float32),
            }

```

Now we will turn this data into the traces list expected by eFEL. Here, we will start with the traces from the activation experiment. For this example, we will only use the 1st repetition. You can change the repetition by renaming the rep_name variable into 'repetition2' or 'repetition3'. Notice that we have to give the time in ms.

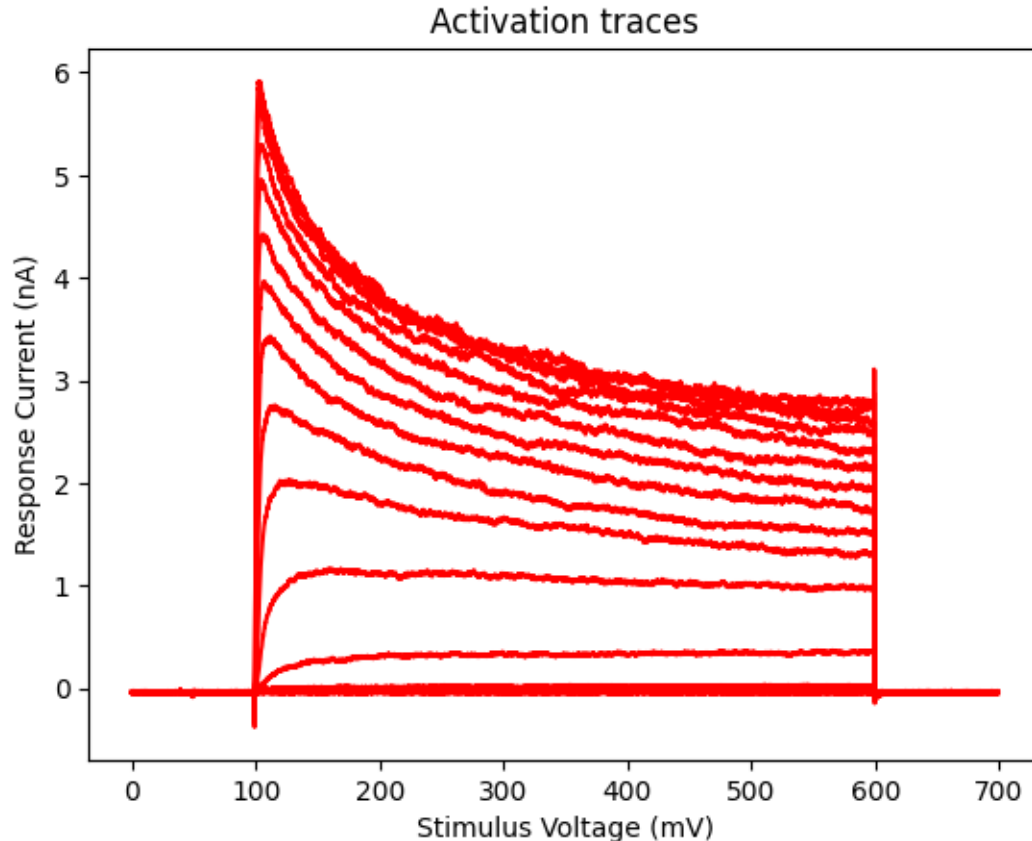
Attention! eFEL was designed mainly to extract features from voltage timeseries. The voltage clamp data are currents. To use the eFEL features with voltage clamp data, we have to give the current recording to the “V” key in the trace dict. eFEL will treat the current trace as if it were a voltage trace.

```
traces = []
rep_name = "repetition1"
exp_type = "Activation"
for idx in range(len(data[exp_type][rep_name]["dt"])):
    trace = {}
    i = data[exp_type][rep_name]["current"][:,idx]
    t = np.arange(i.size) * data[exp_type][rep_name]["dt"][idx]
    # efel expects ms: s -> ms
    t = t * 1000.0
    trace["T"] = t
    trace["V"] = i # trick: input current as if it was voltage
    trace["stim_start"] = [99]
    trace["stim_end"] = [600]
    traces.append(trace)
```

We can plot the traces to see how they look:

```
for t in traces:
    plt.plot(t["T"], t["V"], c="red")
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Response Current (nA)")
plt.title("Activation traces")
```

```
Text(0.5, 1.0, 'Activation traces')
```



We can now extract the desired features from eFEL. Even if there is ‘voltage’ in the feature names, since we gave the current trace instead of the voltage trace, it will still compute these features for the current trace. So here, ‘maximum_voltage’ will actually compute the maximum of the current, ‘voltage_base’ will actually compute the current base and ‘steady_state_voltage_stimend’ will actually compute the current steady state at the end of the stimulus. While those three features are usually used on voltage traces, the last one, ‘activation_time_constant’ has been written specifically for this kind of voltage clamp activation trace.

```
feature_names = ["maximum_voltage", "voltage_base", "steady_state_voltage_stimend",
↪ "activation_time_constant"]
feats = efel.get_feature_values(traces, feature_names)
```

```
/Users/ajaquier/Documents/eFEL/efel/pyfeatures/pyfeatures.py:398: OptimizeWarning:↪
↪ Covariance of the parameters could not be estimated
popt, _ = curve_fit(
```

From these features, we can get several information. We can, for example, extract the ‘activation voltage’, defined as the voltage at which the maximum current of the trace is at least 10% of the maximum current among all of the traces.

```
# Stimulus Voltage used in the experiment - used later for plotting
stim_v = list(range(-90, 90, 10))

max_i = np.array([feat_dict["maximum_voltage"][0] for feat_dict in feats])
max_i = max_i / np.max(max_i)

act_v_idx = np.argmax(max_i >= 0.1)[0][0]
```

(continues on next page)

(continued from previous page)

```
act_v = stim_v[act_v_idx]
act_i = max_i[act_v_idx]
ylim = [np.min(max_i) - 0.05, 1.05]
print(f"Activation voltage is at {act_v} mV")
```

```
Activation voltage is at -20 mV
```

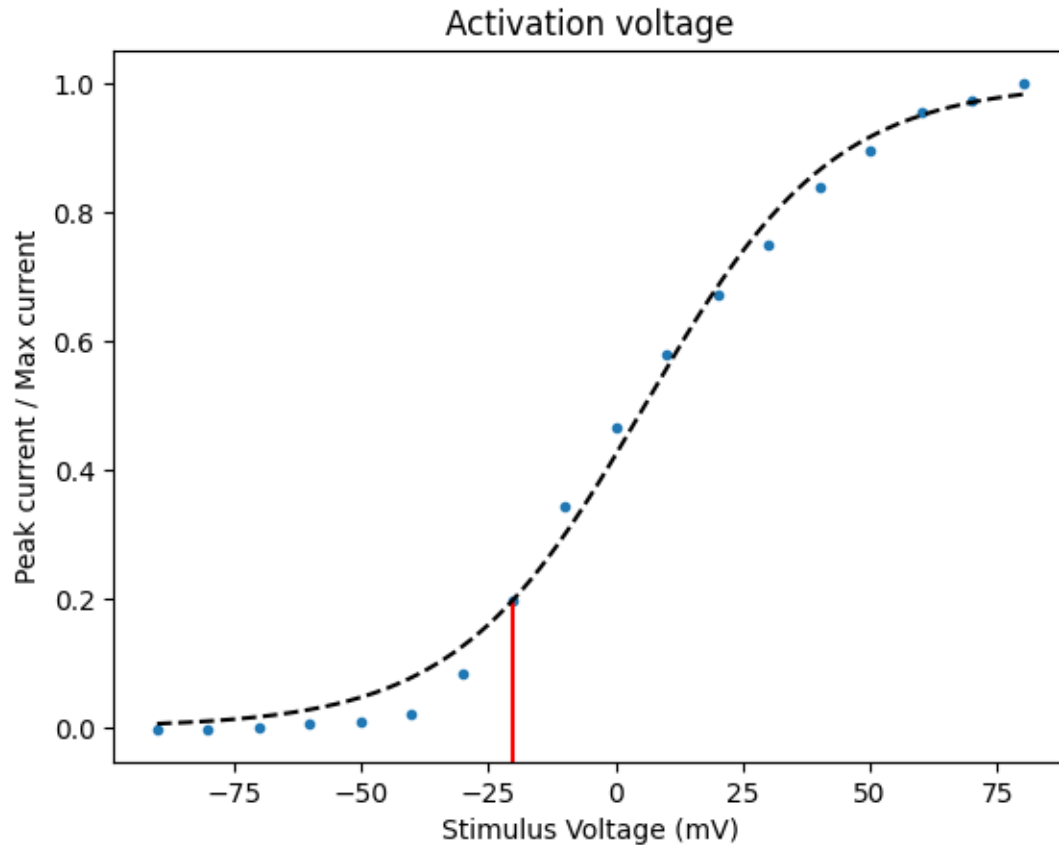
Here, we plot the I-V curve for the 1st repetition, and we mark the activation voltage with a red line

```
plt.plot(stim_v, max_i, '.')
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Peak current / Max current")
plt.plot([act_v, act_v], [ylim[0], act_i], color="red")
plt.ylim(ylim)
plt.title("Activation voltage")

# sigmoid fit
def sigmoid(v, delta, tau):
    return 1 / (1 + np.exp(-(v - delta) / tau))

popt, _ = curve_fit(sigmoid, stim_v, max_i)
v = np.linspace(stim_v[0], stim_v[-1], 200)
i = sigmoid(v, *popt)
plt.plot(v, i, "--", c="black")
```

```
[<matplotlib.lines.Line2D at 0x1338178e0>]
```



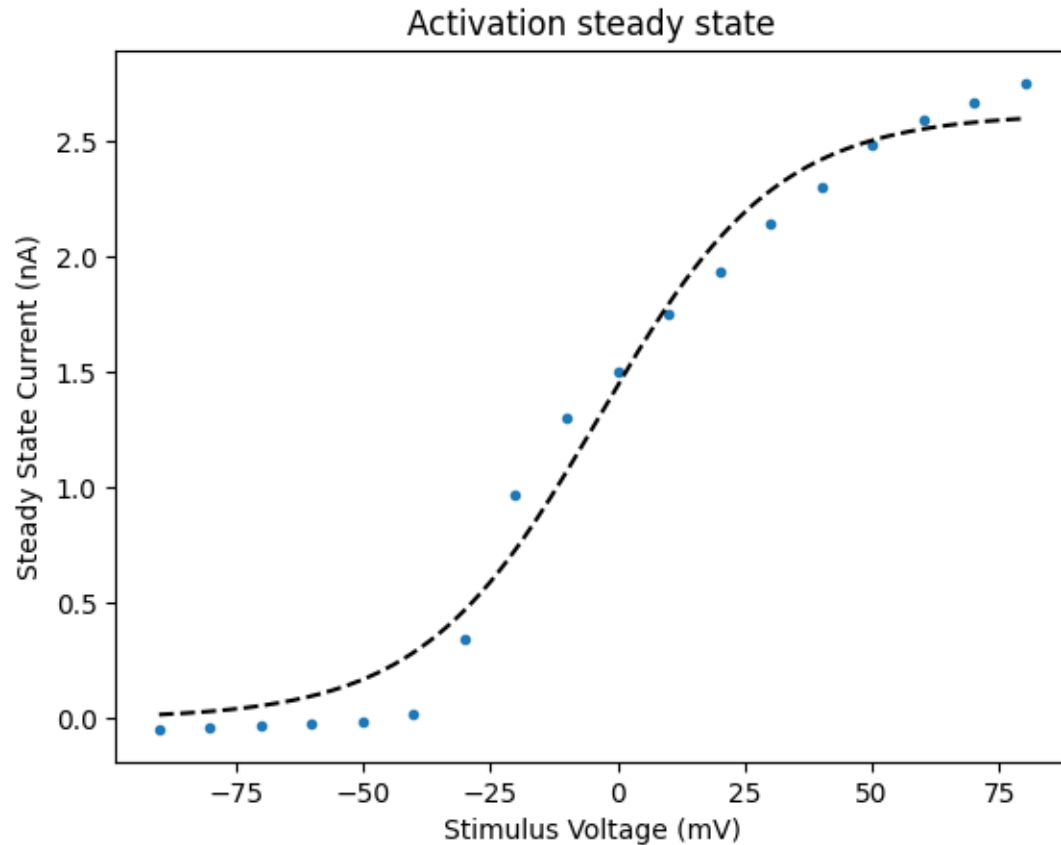
We can also show the evolution of the steady state current at the end of the stimulus across input voltage:

```
ssis = np.array([feat_dict["steady_state_voltage_stimend"][0] for feat_dict in feats])
plt.plot(stim_v, ssis, '.')
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Steady State Current (nA)")
plt.title("Activation steady state")

# sigmoid fit
def sigmoid_scaled(v, delta, tau, A):
    return A / (1 + np.exp(-(v - delta) / tau))

popt, _ = curve_fit(sigmoid_scaled, stim_v, ssis)
ss = sigmoid_scaled(v, *popt)
plt.plot(v, ss, "--", c="black")
```

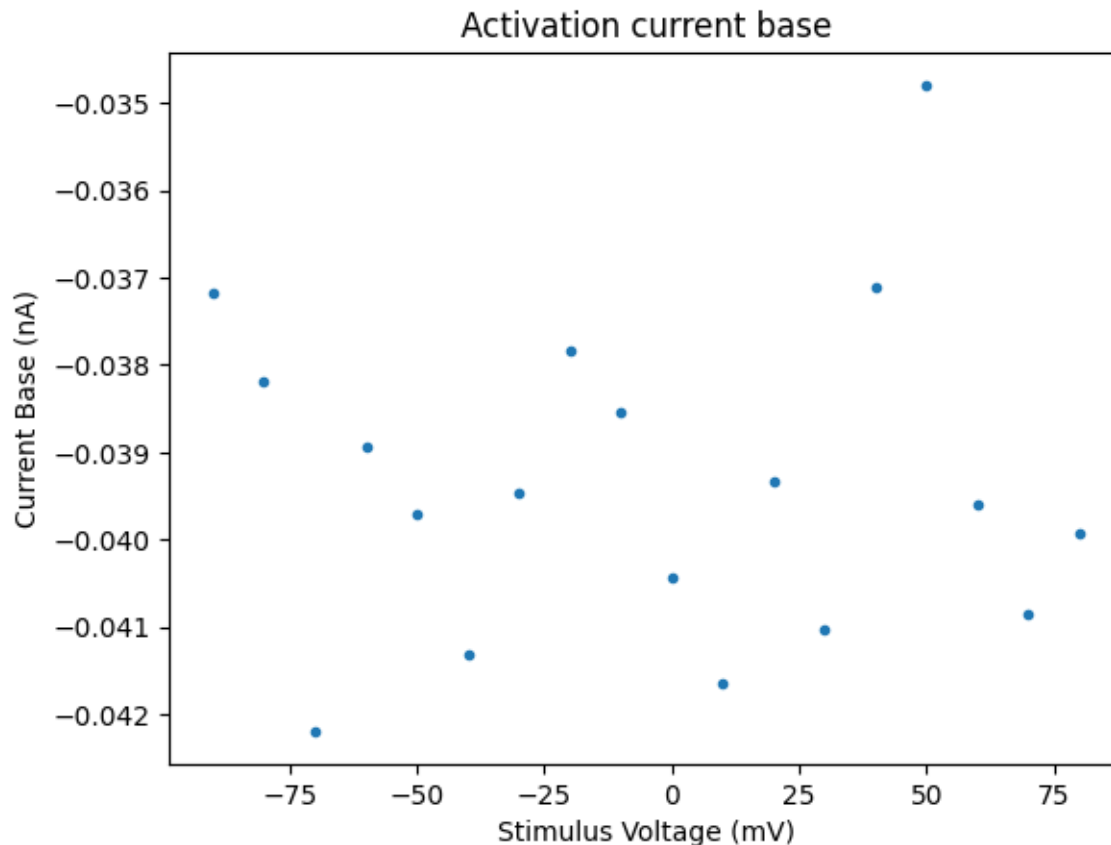
```
[<matplotlib.lines.Line2D at 0x1358cdb40>]
```



We can also plot the current base. Since this feature is computed on the trace before the stimulus is applied, it should be constant across the traces.

```
i_base = np.array([feat_dict["voltage_base"][0] for feat_dict in feats])
plt.plot(stim_v, i_base, '.')
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Current Base (nA)")
plt.title("Activation current base")
```

```
Text(0.5, 1.0, 'Activation current base')
```

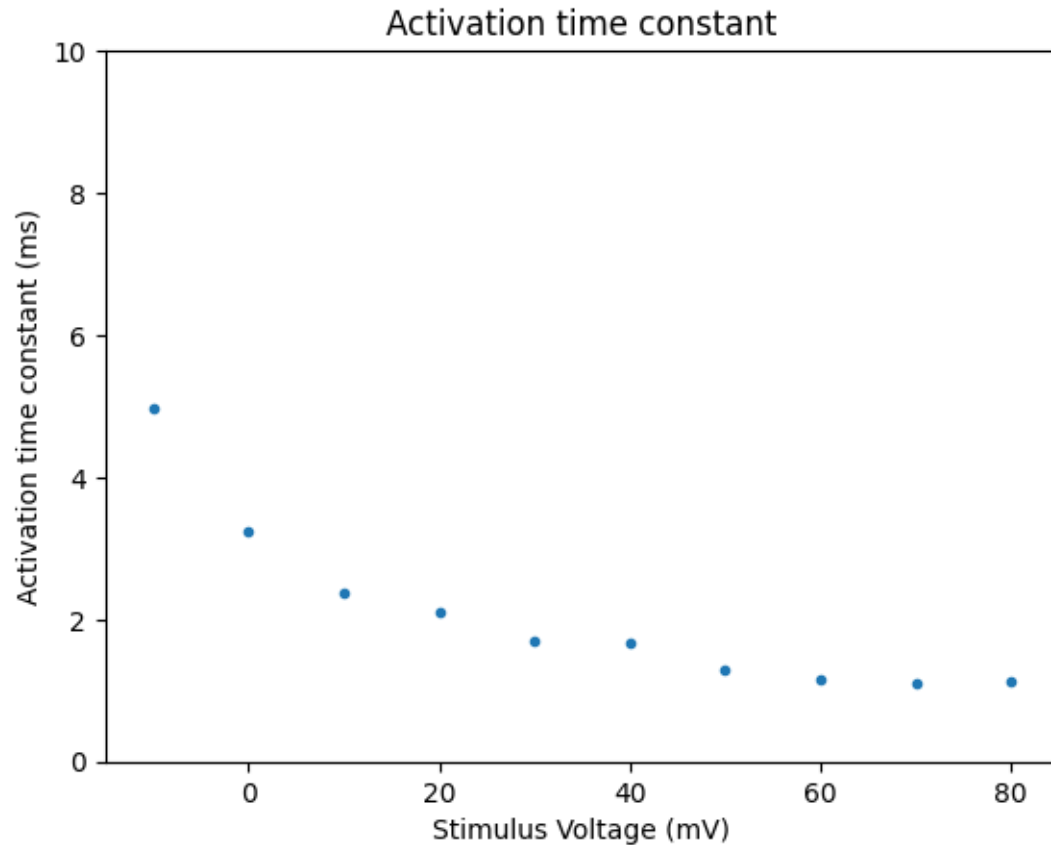



We can see that the current base is the same for all stimulus voltages, which is expected since it is computed before the stimulus is applied.

Finally, we can get the time constant of the activation curve (from `stim_start` to the maximum of the trace):

```
act_tau_efel = np.array([feat_dict["activation_time_constant"][0] for feat_dict in
    ↪ feats])
plt.plot(stim_v, act_tau_efel, '.')
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Activation time constant (ms)")
plt.xlim((-15, 85))
plt.ylim((0, 10))
plt.title("Activation time constant")
```

```
Text(0.5, 1.0, 'Activation time constant')
```



As a visual verification, we can do an exponential fit to the traces, use our time constant as a parameter of the exponential and plot the exponential on top of the trace.

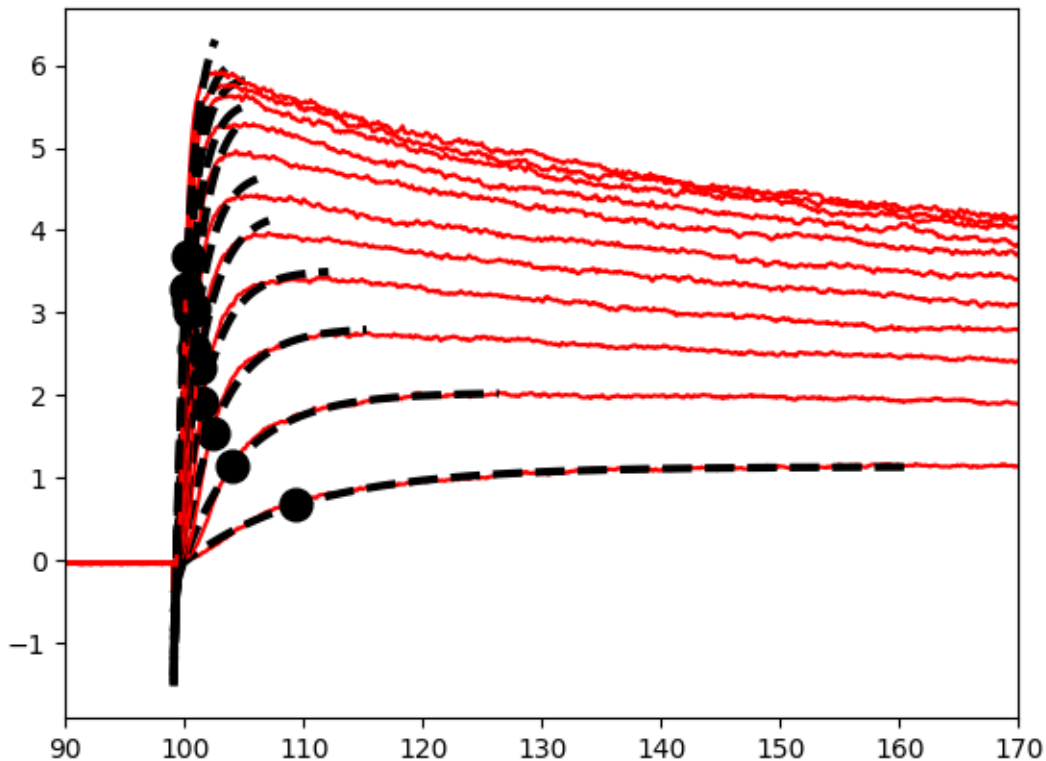
```
def exp_fit(t, tau_1, A1, A0):
    return A0 - A1 * np.exp(-t / tau_1)

for i, trace in enumerate(traces):
    stim_start_idx = np.argwhere(trace["T"] >= trace["stim_start"])[0][0]
    stim_end_idx = np.argwhere(trace["V"] >= np.max(trace["V"]))[0][0]
    # trace["stim_end"] - 1 to account for artefact
    if stim_end_idx < stim_start_idx or trace["T"][stim_end_idx] > trace["stim_end"]
    ↪ "[0] - 1:
        continue
    t_interval = trace["T"][stim_start_idx:stim_end_idx + 1]
    v_interval = trace["V"][stim_start_idx:stim_end_idx + 1]
    t0 = t_interval[0]
    t_interval_corrected = t_interval - t0

    popt, _ = curve_fit(exp_fit, t_interval_corrected, v_interval)

    v_fit = exp_fit(t_interval_corrected, act_tau_efel[i], popt[1], popt[2])
    plt.plot(trace["T"], trace["V"], c="red")
    plt.plot(t_interval, v_fit, "--", c="black", lw=3)
    t_tau = popt[0] + t0
    plt.plot(t_tau, exp_fit(popt[0], *popt), 'o', color='black', markersize=12)
plt.xlim(90, 170)
```

(90.0, 170.0)



2.10.2 Deactivation traces

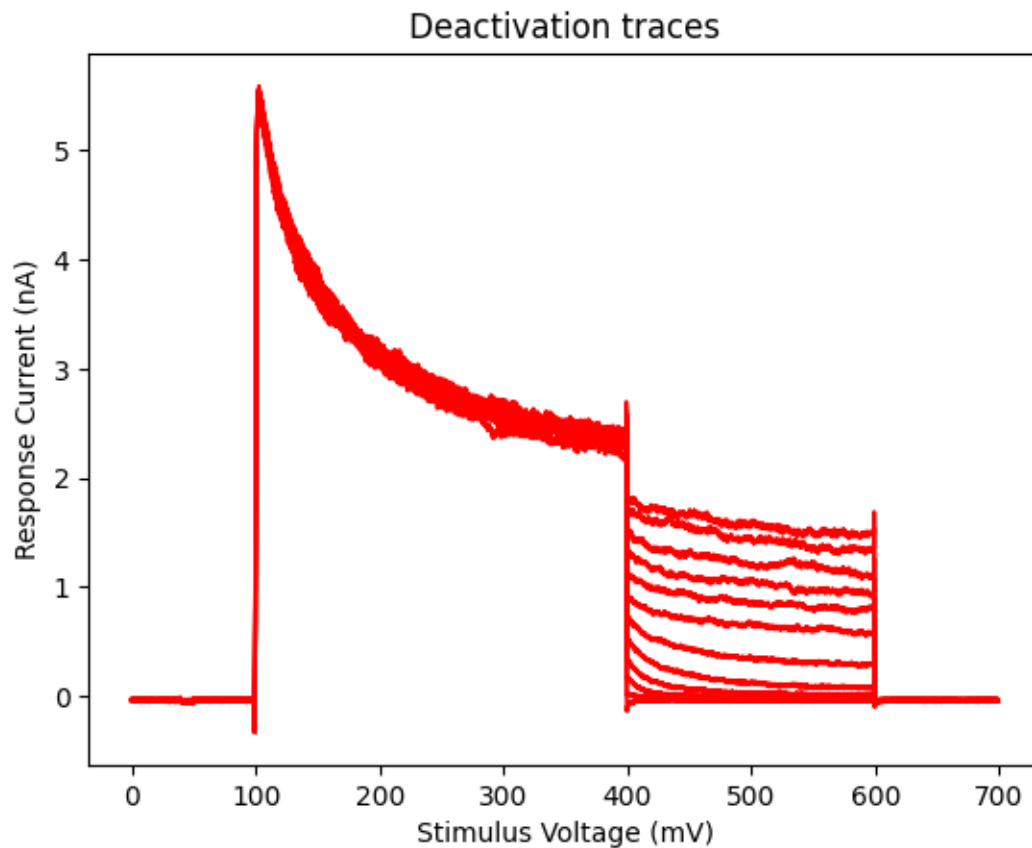
Now we will also perform efeature extraction on traces from deactivation experiment. First, we load the data from the file. Once again, we will put the current data under the 'V' key representing the voltage, so that eFEL treats our trace like any voltage trace. Notice that here, we are setting the stimulus start at 401 and stimulus end at 599 because from 400 to 600 is the interval of interest for our time constant fit, and we start slightly after and end slightly before 'real' stimulus in order to prevent artifacts that show when there is a stimulus change that affect our computation

```
traces = []
rep_name = "repetition1"
exp_type = "Deactivation"
for idx in range(len(data[exp_type][rep_name]["dt"])):
    trace = {}
    i = data[exp_type][rep_name]["current"][idx]
    t = np.arange(i.size) * data[exp_type][rep_name]["dt"][idx]
    # efel expects ms: s -> ms
    t = t * 1000.0
    trace["T"] = t
    trace["V"] = i # trick: input current as if it was voltage
    trace["stim_start"] = [401]
    trace["stim_end"] = [599]
    traces.append(trace)
```

As usual, we can start by plotting the traces:

```
for t in traces:
    plt.plot(t["T"], t["V"], c="red")
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Response Current (nA)")
plt.title("Deactivation traces")
```

```
Text(0.5, 1.0, 'Deactivation traces')
```

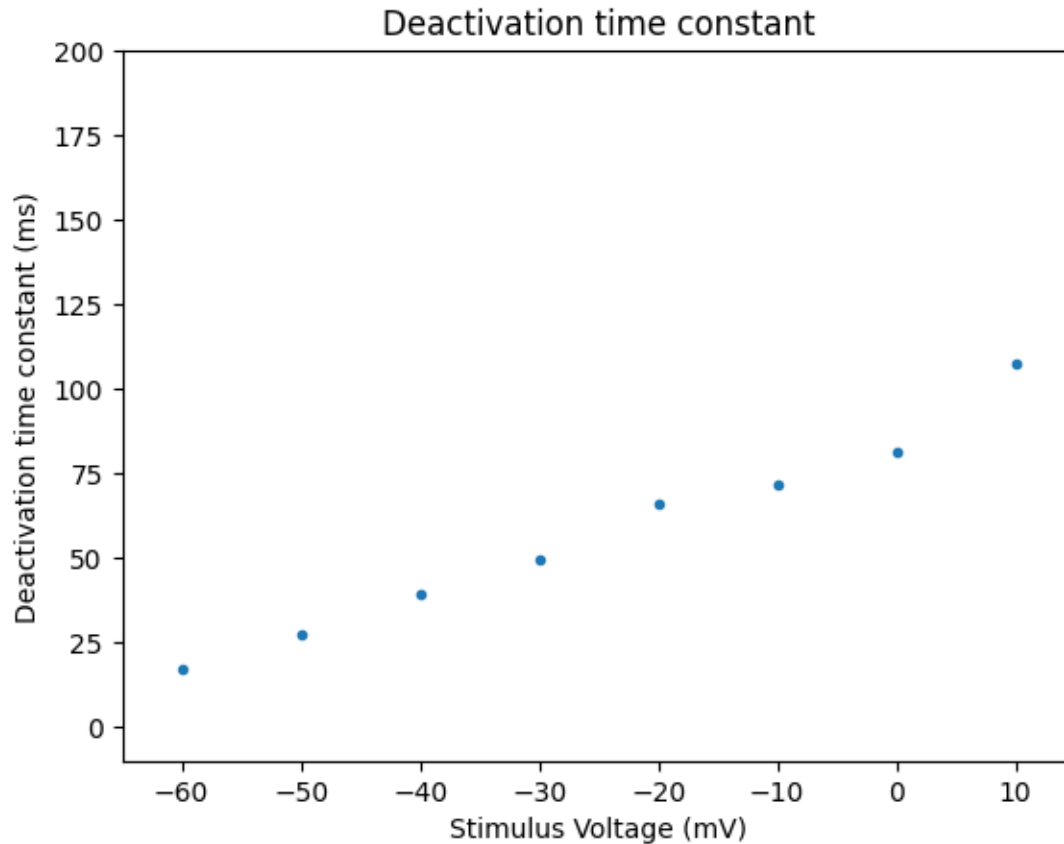


We can now extract the deactivation time constant using eFEL:

```
efel.reset()
stim_v = list(range(-80, 40, 10))
feature_names = ["deactivation_time_constant"]
feats = efel.get_feature_values(traces, feature_names)

deact_tau_efel = np.array([feat_dict["deactivation_time_constant"][0] for feat_dict in
    ↳ feats])
plt.plot(stim_v, deact_tau_efel, '.')
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Deactivation time constant (ms)")
plt.title("Deactivation time constant")
plt.xlim((-65, 15))
plt.ylim((-10, 200))
```

```
(-10.0, 200.0)
```



As a visual verification, we can do an exponential fit to the traces, use our time constant as a parameter of the exponential and plot the exponential on top of the trace.

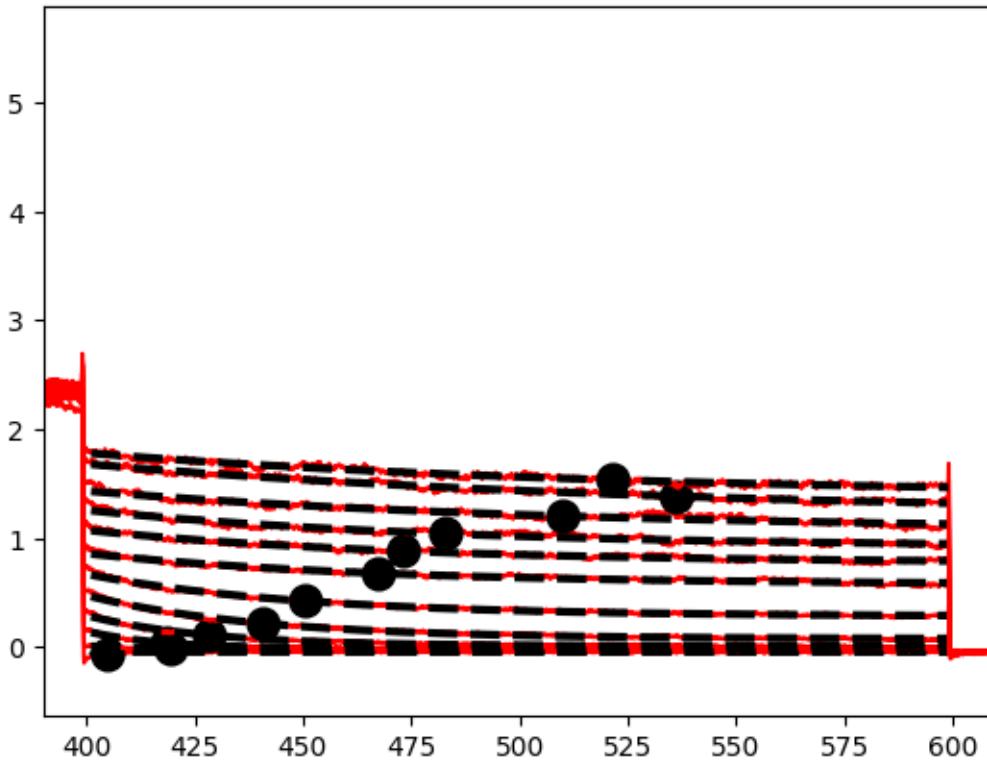
```
for i, trace in enumerate(traces):
    interval_indices = np.where((trace["T"] >= trace["stim_start"][0]) & (trace["T"]
    <- trace["stim_end"][0]))
    t_interval = trace["T"][interval_indices]
    v_interval = trace["V"][interval_indices]
    t0 = t_interval[0]
    t_interval_corrected = t_interval - t0

    popt, _ = curve_fit(exp_fit, t_interval_corrected, v_interval)

    v_fit = exp_fit(t_interval_corrected, deact_tau_efel[i], popt[1], popt[2])
    plt.plot(trace["T"], trace["V"], c="red")
    plt.plot(t_interval, v_fit, "--", c="black", lw=3)
    t_tau = popt[0] + t0
    plt.plot(t_tau, exp_fit(popt[0], *popt), 'o', color='black', markersize=12)
plt.xlim(390, 610)
```

```
/var/folders/9j/1kf4nxwd62v5vnvhcnf87h616469rk/T/ipykernel_12656/1772092254.py:2:
RuntimeWarning: overflow encountered in exp
return A0 - A1 * np.exp(-t / tau_1)
```

(390.0, 610.0)



2.10.3 Inactivation traces

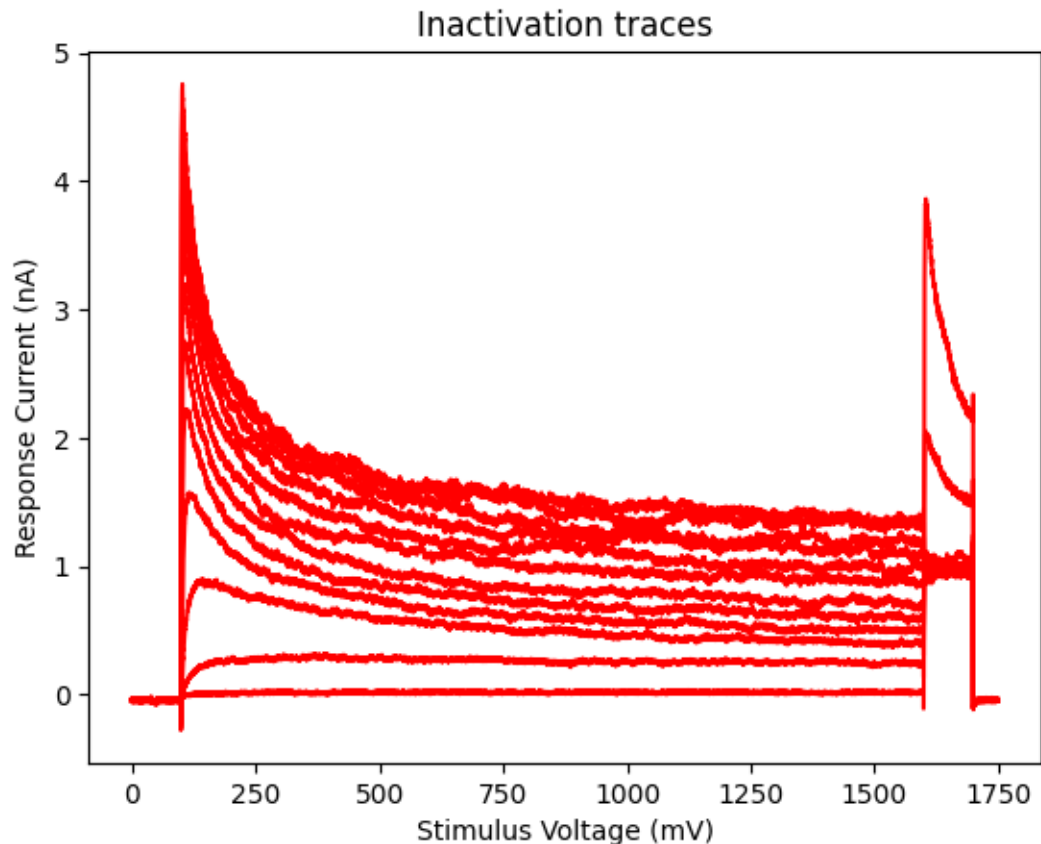
Now we will also perform efeature extraction on traces from inactivation experiment. First, we load the data from the file. Once again, we will put the current data under the 'V' key representing the voltage, so that eFEL treats our trace like any voltage trace. For this one, we will have to extract the features in two steps, because they depend on which stimulus interval we are interested in. We will first look into the features we can extract from the stimulus interval between 1600 and 1700 ms.

```
traces = []
rep_name = "repetition1"
exp_type = "Inactivation"
for idx in range(len(data[exp_type][rep_name]["dt"])):
    trace = {}
    i = data[exp_type][rep_name]["current"][idx]
    t = np.arange(i.size) * data[exp_type][rep_name]["dt"][idx]
    # efel expects ms: s -> ms
    t = t * 1000.0
    trace["T"] = t
    trace["V"] = i # trick: input current as if it was voltage
    trace["stim_start"] = [1600]
    trace["stim_end"] = [1700]
    traces.append(trace)
```

As usual, we will start by plotting the traces:

```
for t in traces:
    plt.plot(t["T"], t["V"], c="red")
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Response Current (nA)")
plt.title("Inactivation traces")
```

```
Text(0.5, 1.0, 'Inactivation traces')
```



We will now plot the I-V curve showing the maximum current in function of the input voltage. Remember that even though we are using the ‘maximum_voltage’ feature, since the trace we have given as input under ‘V’ is actually current, we will actually get the maximum current by using this feature:

```
efel.reset()
stim_v = list(range(-40, 80, 10))
feature_names = ["maximum_voltage"]
feats = efel.get_feature_values(traces, feature_names)

max_i = np.array([feat_dict["maximum_voltage"][0] for feat_dict in feats])
max_i = max_i / np.max(max_i)

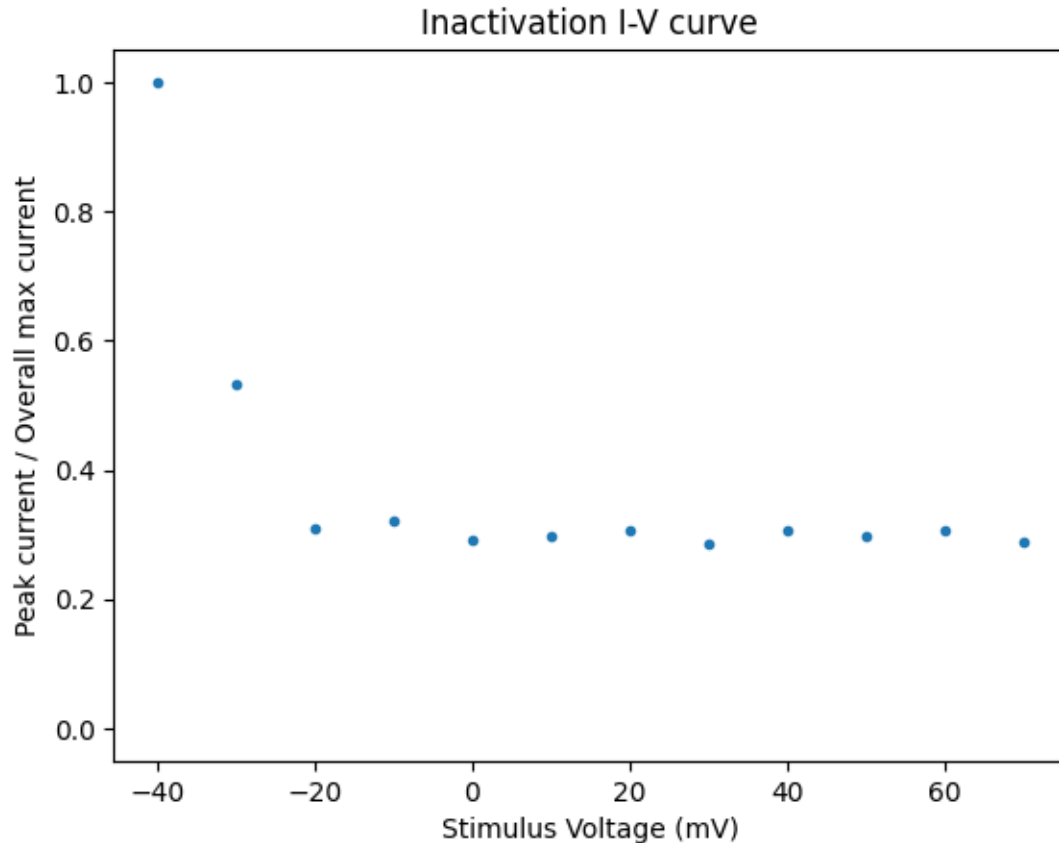
plt.clf()
plt.plot(stim_v, max_i, '.')
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Peak current / Overall max current")
plt.title("Inactivation I-V curve")
```

(continues on next page)

(continued from previous page)

```
plt.ylim((-0.05, 1.05))
```

```
(-0.05, 1.05)
```



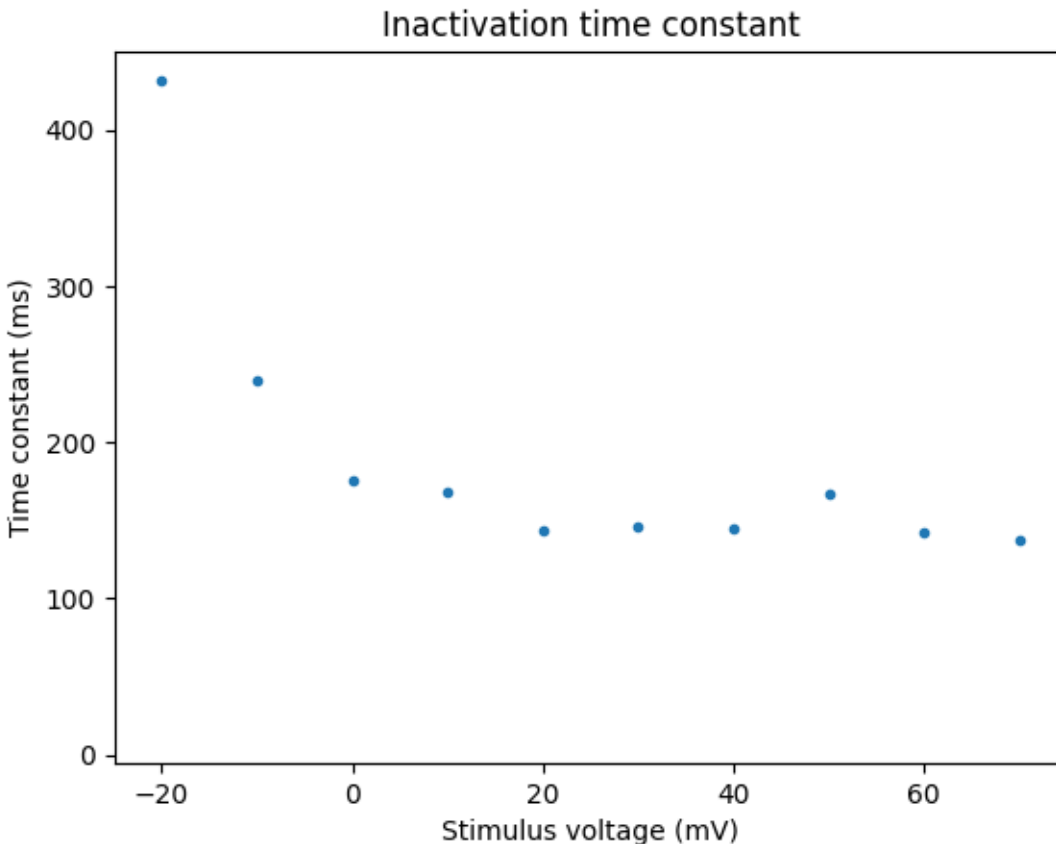
We will now compute the inactivation time constant. But in order to do that, we need to change the stimulus start and stimulus end to 100 and 1600 respectively:

```
traces = []
rep_name = "repetition1"
exp_type = "Inactivation"
for idx in range(len(data[exp_type][rep_name]["dt"])):
    trace = {}
    i = data[exp_type][rep_name]["current"][idx]
    t = np.arange(i.size) * data[exp_type][rep_name]["dt"][idx]
    # efel expects ms: s -> ms
    t = t * 1000.0
    trace["T"] = t
    trace["V"] = i # trick: input current as if it was voltage
    trace["stim_start"] = [100]
    trace["stim_end"] = [1600]
    traces.append(trace)
```

And now we can compute the inactivation time constant. It will select the trace from the trace maximum up to the end of the stimulus (minus a few data points to avoid artifacts).


```
feature_names = ["inactivation_time_constant"]
feats = efel.get_feature_values(traces, feature_names)
inact_tau_efel = np.array([feat_dict["inactivation_time_constant"][0] for feat_dict in
    ↪feats])
plt.plot(stim_v, inact_tau_efel, '.')
plt.xlim((-25, 75))
plt.ylim((-5, 450))
plt.title("Inactivation time constant")
plt.xlabel("Stimulus voltage (mV)")
plt.ylabel("Time constant (ms)")
```

```
Text(0, 0.5, 'Time constant (ms)')
```



Note that if you want to change the number of data points to remove from the end of the trace, you can change it by changing the setting of `inactivation_tc_end_skip`. Setting it to 0, for example, will keep the artifacts in and can bias the computation, depending on the trace.

```
efel.set_setting("inactivation_tc_end_skip", 0)
feature_names = ["inactivation_time_constant", "steady_state_voltage_stimend"]
feats = efel.get_feature_values(traces, feature_names)
inact_tau_efel = np.array([feat_dict["inactivation_time_constant"][0] for feat_dict in
    ↪feats])
plt.plot(stim_v, inact_tau_efel, '.')
plt.xlim((-25, 75))
plt.title("Inactivation time constant")
```

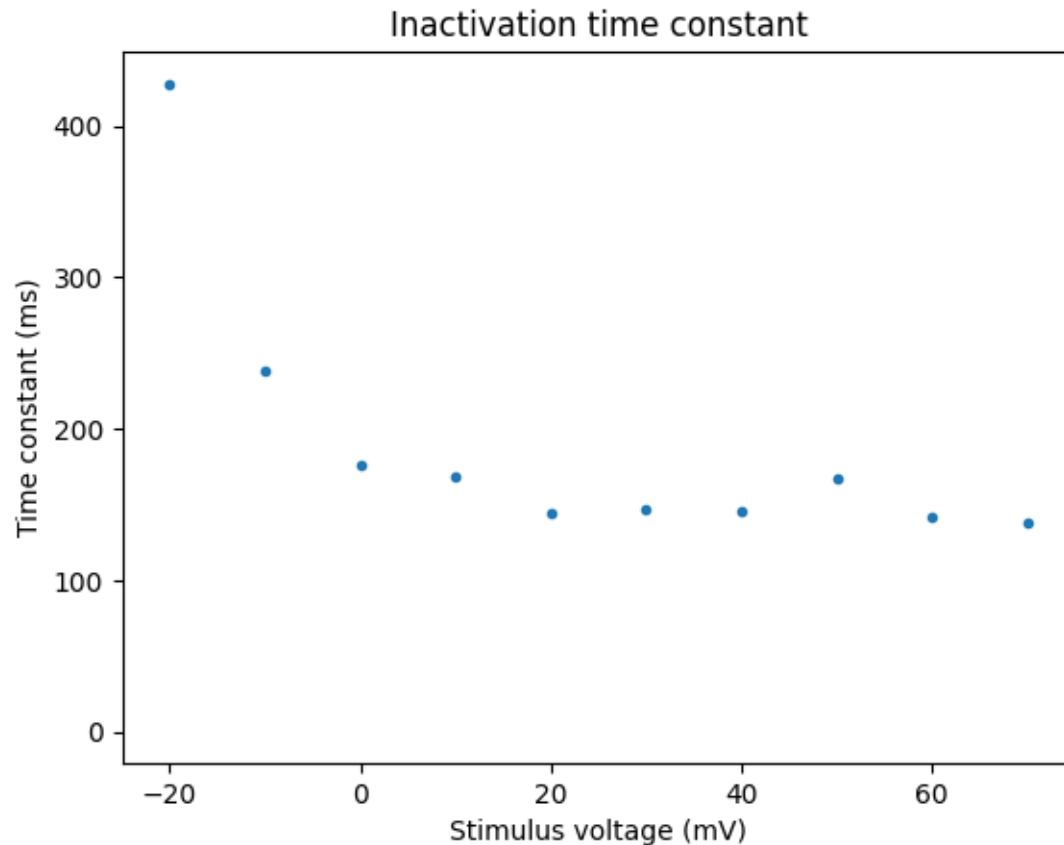
(continues on next page)

(continued from previous page)

```
plt.xlabel("Stimulus voltage (mV)")
plt.ylabel("Time constant (ms)")
```

```
/Users/ajaquier/Documents/eFEL/efel/pyfeatures/pyfeatures.py:477: OptimizeWarning:
Covariance of the parameters could not be estimated
popt, _ = curve_fit(
```

```
Text(0, 0.5, 'Time constant (ms)')
```



As a visual verification, we can do an exponential fit to the traces, use our time constant as a parameter of the exponential and plot the exponential on top of the trace.

```
for i, trace in enumerate(traces):
    # - 1 to account for artifacts
    interval_indices = np.where((trace["T"] >= trace["stim_start"][0]) & (trace["T"]
    < trace["stim_end"][0] - 1))
    t_interval = trace["T"][interval_indices]
    v_interval = trace["V"][interval_indices]
    stim_start_idx = np.argmax(v_interval >= np.max(v_interval))[0][0]
    t_interval = t_interval[stim_start_idx:]
    v_interval = v_interval[stim_start_idx:]
    t0 = t_interval[0] # for testing
    t_interval_corrected = t_interval - t_interval[0]
```

(continues on next page)

(continued from previous page)

```

popt, _ = curve_fit(exp_fit, t_interval_corrected, v_interval)

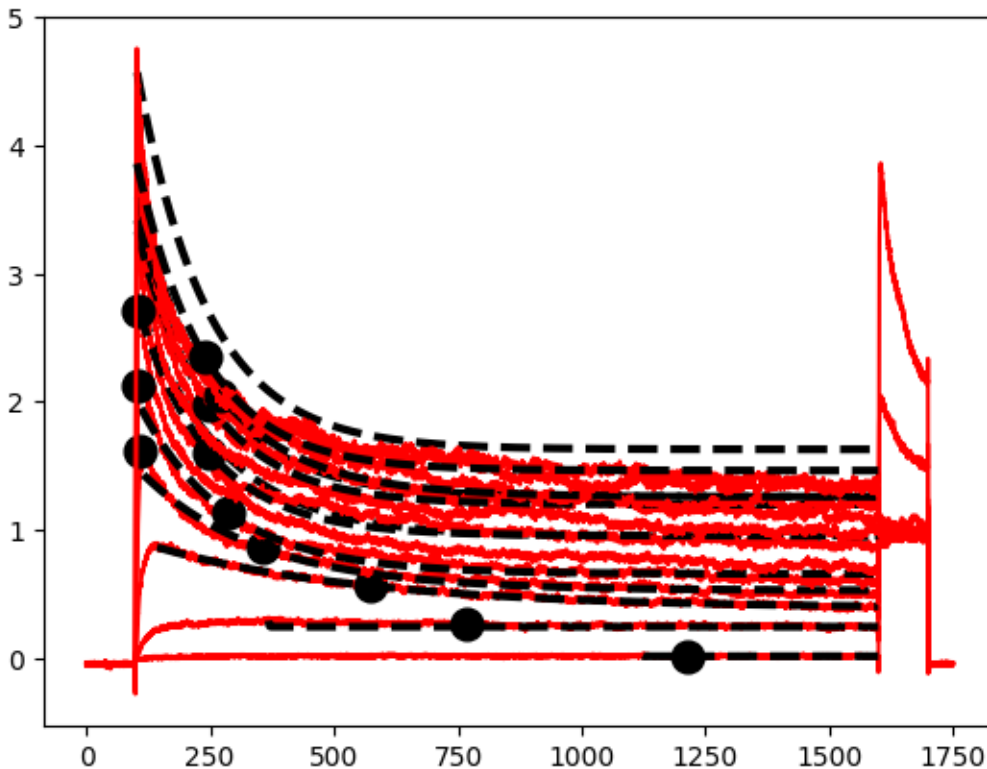
v_fit = exp_fit(t_interval_corrected, inact_tau_efel[i], popt[1], popt[2])
plt.plot(trace["T"], trace["V"], c="red")
plt.plot(t_interval, v_fit, "--", c="black", lw=3)
t_tau = popt[0] + t0
plt.plot(t_tau, exp_fit(popt[0], *popt), 'o', color='black', markersize=12)
# plt.xlim(390, 610)

```

```

/var/folders/9j/1kf4nxwd62v5vnhcnf87h616469rk/T/ipykernel_12656/1772092254.py:2:
RuntimeWarning: overflow encountered in exp
    return A0 - A1 * np.exp(-t / tau_1)
/var/folders/9j/1kf4nxwd62v5vnhcnf87h616469rk/T/ipykernel_12656/870918093.py:12:
OptimizeWarning: Covariance of the parameters could not be estimated
popt, _ = curve_fit(exp_fit, t_interval_corrected, v_interval)

```



Finally, the steady state of the Inactivation curve can be plotted as follow:

```

ssis = np.array([feat_dict["steady_state_voltage_stimend"][0] for feat_dict in feats])
plt.plot(stim_v, ssis, '.')
plt.xlabel("Stimulus Voltage (mV)")
plt.ylabel("Steady State Current (nA)")
plt.title("Inactivation steady state")

popt, _ = curve_fit(sigmoid_scaled, stim_v, ssis)
v = np.linspace(stim_v[0], stim_v[-1], 200)

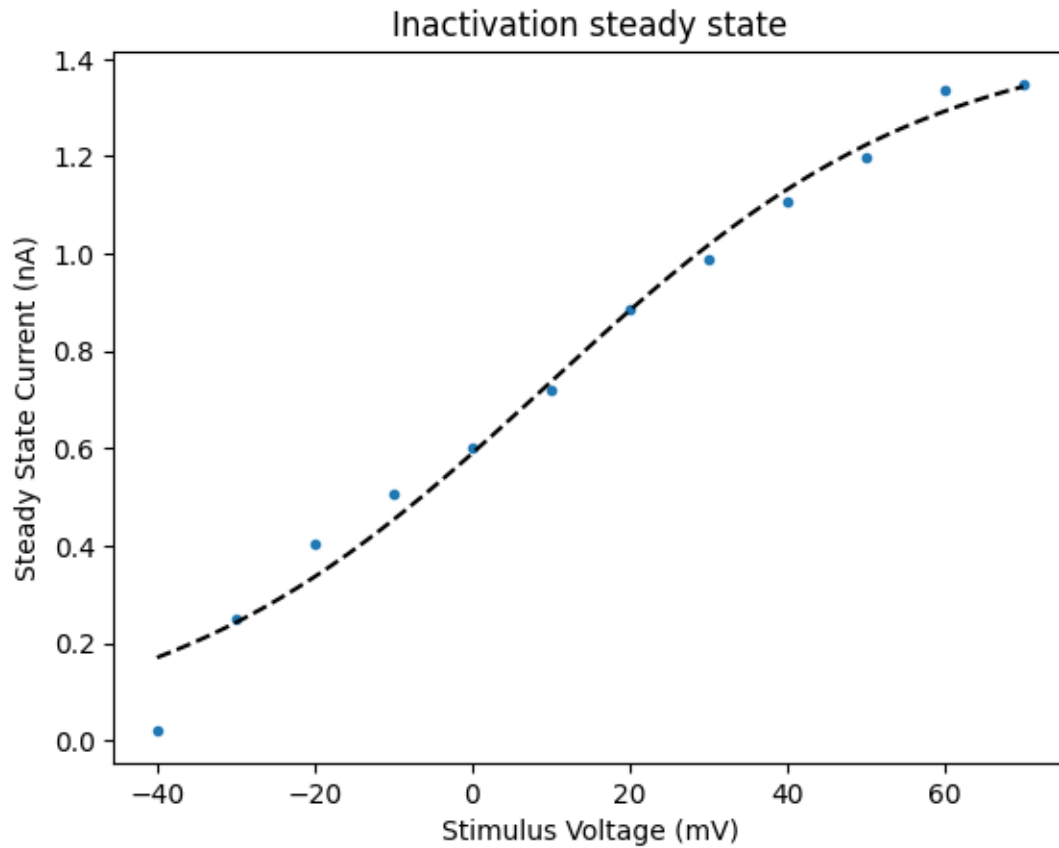
```

(continues on next page)

(continued from previous page)

```
ss = sigmoid_scaled(v, *popt)
plt.plot(v, ss, "--", c="black")
```

```
[<matplotlib.lines.Line2D at 0x135f9bcd0>]
```

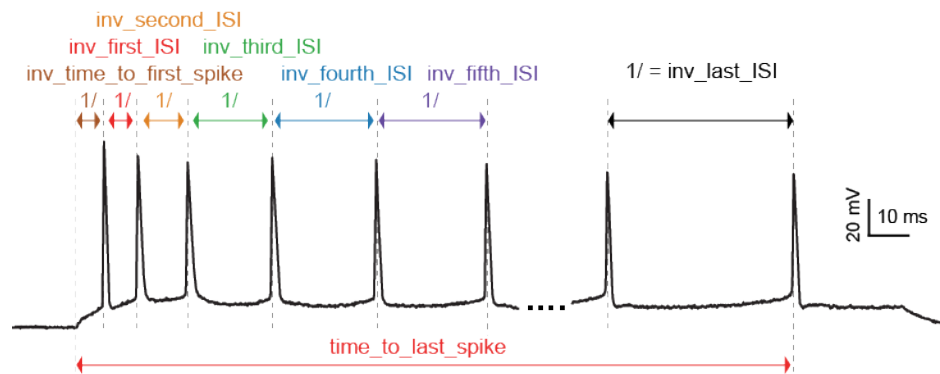


EFEATURE DESCRIPTIONS

Time, voltage and current (if given) are interpolated using *interp_step* setting (default *interp_step* = 0.1 ms) before efeatures are extracted from them. Since, they are technically features in eFEL, you can retrieve the interpolated time, voltage and current (if given), like any other feature.

3.1 Implemented eFeatures

3.1.1 Spike event features



peak_indices

SpikeEvent : The indices of the maxima of the peaks.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:**
- **Units:** constant
- **Pseudocode:**

```
peak_indices = [i for i in range(1, len(v)) if v[i] > threshold and v[i - 1] <
↳ threshold]
```

peak_time

SpikeEvent : The times of the maxima of the peaks

- **Required features:** peak_indices
- **Units:** ms
- **Pseudocode:**

```
peak_time = time[peak_indices]
```

time_to_first_spike

SpikeEvent : Time from the start of the stimulus to the maximum of the first peak

- **Required features:** peak_time
- **Units:** ms
- **Pseudocode:**

```
time_to_first_spike = peakttime[0] - stimstart
```

time_to_last_spike

SpikeEvent : Time from stimulus start to last spike

- **Required features:** peak_time (ms), stimstart (ms)
- **Units:** ms
- **Pseudocode:**

```
if len(peak_time) > 0:  
    time_to_last_spike = peak_time[-1] - stimstart  
else:  
    time_to_last_spike = 0
```

time_to_second_spike

SpikeEvent : Time from the start of the stimulus to the maximum of the second peak

- **Required features:** peak_time
- **Units:** ms
- **Pseudocode:**

```
time_to_second_spike = peakttime[1] - stimstart
```

inv_time_to_first_spike

SpikeEvent : 1.0 over time to first spike (times 1000 to convert it to Hz); returns 0 when no spike

- **Required features:** time_to_first_spike
- **Units:** Hz
- **Pseudocode:**

```

if len(time_to_first_spike) > 0:
    inv_time_to_first_spike = 1000.0 / time_to_first_spike[0]
else:
    inv_time_to_first_spike = 0

```

ISI_values

ISI Python efeature : The interspike intervals (i.e. time intervals) between adjacent peaks.

- **Required features:** peak_time (ms)
- **Units:** ms
- **Pseudocode:**

```
isi_values = numpy.diff(peak_time)[1:]
```

all_ISI_values

SpikeEvent : The interspike intervals, i.e., the time intervals between adjacent peaks.

- **Required features:** peak_time (ms)
- **Units:** ms
- **Pseudocode:**

```
all_isi_values_vec = numpy.diff(peak_time)
```

inv_ISI_values

ISI Python efeature : Computes all inverse spike interval values.

- **Required features:** peak_time (ms)
- **Units:** Hz
- **Pseudocode:**

```

all_isi_values_vec = numpy.diff(peak_time)
inv_isi_values = 1000.0 / all_isi_values_vec

```

inv_first_ISI, inv_second_ISI, inv_third_ISI, inv_fourth_ISI, inv_fifth_ISI, inv_last_ISI

ISI Python efeature : 1.0 over first/second/third/fourth/fifth/last ISI; returns 0 when no ISI

- **Required features:** peak_time (ms)
- **Units:** Hz
- **Pseudocode:**

```

all_isi_values_vec = numpy.diff(peak_time)

if len(all_isi_values_vec) > 0:
    inv_first_ISI = 1000.0 / all_isi_values_vec[0]
else:
    inv_first_ISI = 0

```

(continues on next page)

(continued from previous page)

```

if len(all_isi_values_vec) > 1:
    inv_second_ISI = 1000.0 / all_isi_values_vec[1]
else:
    inv_second_ISI = 0

if len(all_isi_values_vec) > 2:
    inv_third_ISI = 1000.0 / all_isi_values_vec[2]
else:
    inv_third_ISI = 0

if len(all_isi_values_vec) > 3:
    inv_fourth_ISI = 1000.0 / all_isi_values_vec[3]
else:
    inv_fourth_ISI = 0

if len(all_isi_values_vec) > 4:
    inv_fifth_ISI = 1000.0 / all_isi_values_vec[4]
else:
    inv_fifth_ISI = 0

if len(all_isi_values_vec) > 0:
    inv_last_ISI = 1000.0 / all_isi_values_vec[-1]
else:
    inv_last_ISI = 0

```

doublet_ISI

SpikeEvent : The time interval between the first two peaks

- **Required features:** peak_time (ms)
- **Units:** ms
- **Pseudocode:**

```
doublet_ISI = peak_time[1] - peak_time[0]
```

ISI_semilog_slope

ISI Python efeature : The slope of a linear fit to a semilog plot of the ISI values.

Attention: the 1st ISI is not taken into account unless ignore_first_ISI is set to 0. See Python efeature: ISIs feature for more details.

- **Required features:** t, V, stim_start, stim_end, ISI_values
- **Units:** ms
- **Pseudocode:**

```

x = range(1, len(ISI_values)+1)
log_ISI_values = numpy.log(ISI_values)
slope, _ = numpy.polyfit(x, log_ISI_values, 1)

ISI_semilog_slope = slope

```


ISI_log_slope

ISI Python efeature : The slope of a linear fit to a loglog plot of the ISI values.

Attention: the 1st ISI is not taken into account unless ignore_first_ISI is set to 0. See Python efeature: ISIs feature for more details.

- **Required features:** t, V, stim_start, stim_end, ISI_values
- **Units:** ms
- **Pseudocode:**

```
log_x = numpy.log(range(1, len(ISI_values)+1))
log_ISI_values = numpy.log(ISI_values)
slope, _ = numpy.polyfit(log_x, log_ISI_values, 1)

ISI_log_slope = slope
```

ISI_log_slope_skip

ISI Python efeature : The slope of a linear fit to a loglog plot of the ISI values, but not taking into account the first ISI values.

The proportion of ISI values to be skipped is given by spike_skipf (between 0 and 1). However, if this number of ISI values to skip is higher than max_spike_skip, then max_spike_skip is taken instead.

- **Required features:** t, V, stim_start, stim_end, ISI_values
- **Parameters:** spike_skipf (default=0.1), max_spike_skip (default=2)
- **Units:** ms
- **Pseudocode:**

```
start_idx = min([max_spike_skip, round((len(ISI_values) + 1) * spike_skipf)])
ISI_values = ISI_values[start_idx:]
log_x = numpy.log(range(1, len(ISI_values)+1))
log_ISI_values = numpy.log(ISI_values)
slope, _ = numpy.polyfit(log_x, log_ISI_values, 1)

ISI_log_slope = slope
```

ISI_CV

ISI Python efeature : The coefficient of variation of the ISIs.

Attention: the 1st ISI is not taken into account unless ignore_first_ISI is set to 0. See Python efeature: ISIs feature for more details.

- **Required features:** ISIs
- **Units:** constant
- **Pseudocode:**

```
ISI_mean = numpy.mean(ISI_values)
ISI_CV = np.std(isi_values, ddof=1) / ISI_mean
```

irregularity_index

ISI Python efeature : Mean of the absolute difference of all ISIs, except the first one (see Python efeature: ISIs feature for more details.)

The first ISI can be taken into account if `ignore_first_ISI` is set to 0.

- **Required features:** ISI_values
- **Units:** ms
- **Pseudocode:**

```
irregularity_index = numpy.mean(numpy.absolute(ISI_values[1:] - ISI_values[:-1]))
```

adaptation_index

SpikeEvent : Normalized average difference of two consecutive ISIs, skipping the first ISIs

The proportion of ISI values to be skipped is given by `spike_skipf` (between 0 and 1). However, if this number of ISI values to skip is higher than `max_spike_skip`, then `max_spike_skip` is taken instead.

The adaptation index is zero for a constant firing rate and bigger than zero for a decreasing firing rate

- **Required features:** stim_start, stim_end, peak_time
- **Parameters:** offset (default=0), spike_skipf (default=0.1), max_spike_skip (default=2)
- **Units:** constant
- **Pseudocode:**

```
# skip the first ISIs
peak_selection = [peak_time >= stim_start - offset, peak_time <= stim_end - offset]
spike_time = peak_time[numpy.all(peak_selection, axis=0)]

start_idx = min([max_spike_skip, round(len(spike_time) * spike_skipf)])
spike_time = spike_time[start_idx:]

# compute the adaptation index
ISI_values = spike_time[1:] - spike_time[:-1]
ISI_sum = ISI_values[1:] + ISI_values[:-1]
ISI_sub = ISI_values[1:] - ISI_values[:-1]
adaptation_index = numpy.mean(ISI_sum / ISI_sub)
```

adaptation_index_2

SpikeEvent : Normalized average difference of two consecutive ISIs, starting at the second ISI

The adaptation index is zero for a constant firing rate and bigger than zero for a decreasing firing rate

- **Required features:** stim_start, stim_end, peak_time
- **Parameters:** offset (default=0)
- **Units:** constant
- **Pseudocode:**

```
# skip the first ISI
peak_selection = [peak_time >= stim_start - offset, peak_time <= stim_end - offset]
spike_time = peak_time[numpy.all(peak_selection, axis=0)]

spike_time = spike_time[1:]

# compute the adaptation index
ISI_values = spike_time[1:] - spike_time[:-1]
ISI_sum = ISI_values[1:] + ISI_values[:-1]
ISI_sub = ISI_values[1:] - ISI_values[:-1]
adaptation_index = numpy.mean(ISI_sum / ISI_sub)
```

spike_count

Python efeature : Number of spikes in the trace, including outside of stimulus interval

- **Required features:** peak_indices
- **Units:** constant
- **Pseudocode:**

```
spike_count = len(peak_indices)
```

Note: “spike_count” is the new name for the feature “Spikecount”. “Spikecount”, while still available, will be removed in the future.

spike_count_stimint

Python efeature : Number of spikes inside the stimulus interval

- **Required features:** peak_time
- **Units:** constant
- **Pseudocode:**

```
peaktimes_stimint = numpy.where((peak_time >= stim_start) & (peak_time <= stim_end))
spike_count_stimint = len(peaktimes_stimint)
```

Note: “spike_count_stimint” is the new name for the feature “Spikecount_stimint”. “Spikecount_stimint”, while still available, will be removed in the future.

number_initial_spikes

SpikeEvent : Number of spikes at the beginning of the stimulus

- **Required features:** peak_time
- **Required parameters:** initial_perc (default=0.1)
- **Units:** constant
- **Pseudocode:**

```
initial_length = (stimend - stimstart) * initial_perc
number_initial_spikes = len(numpy.where( \
    (peak_time >= stimstart) & \
    (peak_time <= stimstart + initial_length)))
```

mean_frequency

SpikeEvent : The mean frequency of the firing rate

- **Required features:** stim_start, stim_end, peak_time
- **Units:** Hz
- **Pseudocode:**

```
condition = np.all((stim_start < peak_time, peak_time < stim_end), axis=0)
spikecount = len(peak_time[condition])
last_spike_time = peak_time[peak_time < stim_end][-1]
mean_frequency = 1000 * spikecount / (last_spike_time - stim_start)
```

burst_begin_indices

SpikeEvent : The indices of the first peak of each burst.

The indices are to be applied to the peak_time feature, and not time or voltage.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** all_ISI_values
- **Units:** constant
- **Pseudocode:**

```
burst_begin_indices = [1]
burst_end_indices = []
count = 1

for i in range(2, len(isi_values)):
    d_median = numpy.median(isi_values[count:i])
    in_burst = (
        len(burst_end_indices) == 0 or
        burst_begin_indices[-1] > burst_end_indices[-1]
    )

    // look for end burst
    if in_burst and isi_values[i] > (burst_factor * d_median):
        burst_end_indices.append(i)
        count = i

    if isi_values[i] < isi_values[i - 1] / burst_factor:
        if in_burst:
            burst_begin_indices[-1] = i
        else:
            burst_begin_indices.append(i)
            count = i
```

burst_end_indices

SpikeEvent : The indices of the last peak of each burst.

The indices are to be applied to the peak_time feature, and not time or voltage.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** all_ISI_values
- **Units:** constant
- **Pseudocode:**

```
burst_begin_indices = [1]
burst_end_indices = []
count = 1

for i in range(2, len(isi_values)):
    d_median = numpy.median(isi_values[count:i])
    in_burst = (
        len(burst_end_indices) == 0 or
        burst_begin_indices[-1] > burst_end_indices[-1]
    )

    // look for end burst
    if in_burst and isi_values[i] > (burst_factor * d_median):
        burst_end_indices.append(i)
        count = i

    if isi_values[i] < isi_values[i - 1] / burst_factor:
        if in_burst:
            burst_begin_indices[-1] = i
        else:
            burst_begin_indices.append(i)
            count = i
```

strict_burst_mean_freq

SpikeEvent : The mean frequency during a burst for each burst

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** burst_begin_indices, burst_end_indices, peak_time
- **Units:** Hz
- **Pseudocode:**

```
if burst_begin_indices is None or burst_end_indices is None:
    strict_burst_mean_freq = None
else:
```

(continues on next page)

(continued from previous page)

```

strict_burstmean_freq = (
    (burst_end_indices - burst_begin_indices + 1) * 1000 / (
        peak_time[burst_end_indices] - peak_time[burst_begin_indices]
    )
)

```

burst_ISI_indices

ISI Python efeature : The indices for each burst beginning. The indices are to be applied to the peak_time feature, and not time or voltage.

- **Required features:** all_ISI_values
- **Units:** constant
- **Pseudocode:**

```

burst_indices = []
count = -1

for i in range(1, len(isi_values) - 1):
    isi_p_copy = isi_values[count + 1: i]
    n = len(isi_p_copy)

    if n == 0:
        continue

    d_median = np.median(isi_p_copy)

    # Check burst condition
    if isi_values[i] > (burst_factor * d_median) and isi_values[i + 1] < (
        isi_values[i] / burst_factor
    ):
        burst_indices.append(i + 1)
        count = i - 1

```

burst_mean_freq

ISI Python efeature : The mean frequency during a burst for each burst

If burst_ISI_indices did not detect any burst beginning, then the spikes are not considered to be part of any burst

- **Required features:** burst_ISI_indices, peak_time
- **Units:** Hz
- **Pseudocode:**

```

if burst_ISI_indices is None:
    return None
elif len(burst_ISI_indices) == 0:
    return []

burst_mean_freq = []
burst_index = numpy.insert(

```

(continues on next page)

(continued from previous page)

```

    burst_index_tmp, burst_index_tmp.size, len(peak_time) - 1
)

# 1st burst
span = peak_time[burst_index[0]] - peak_time[0]
N_peaks = burst_index[0] + 1
burst_mean_freq.append(N_peaks * 1000 / span)

for i, burst_idx in enumerate(burst_index[:-1]):
    if burst_index[i + 1] - burst_idx != 1:
        span = peak_time[burst_index[i + 1]] - peak_time[burst_idx + 1]
        N_peaks = burst_index[i + 1] - burst_idx
        burst_mean_freq.append(N_peaks * 1000 / span)

return burst_mean_freq

```

strict_burst_number

ISI Python efeature : The number of bursts

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** strict_burst_mean_freq
- **Units:** constant
- **Pseudocode:**

```
burst_number = len(strict_burst_mean_freq)
```

burst_number

Python efeature : The number of bursts

- **Required features:** burst_mean_freq
- **Units:** constant
- **Pseudocode:**

```
burst_number = len(burst_mean_freq)
```

single_burst_ratio

ISI Python efeature : Length of the second isi over the median of the rest of the isis. The first isi is not taken into account, because it could bias the feature. See ISI_values feature for more details.

If ignore_first_ISI is set to 0, then single burst ratio becomes the length of the first isi over the median of the rest of the isis.

- **Required features:** ISI_values
- **Units:** constant
- **Pseudocode:**

```
single_burst_ratio = ISI_values[0] / numpy.mean(ISI_values)
```

spikes_per_burst

Python efeature : Number of spikes in each burst.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** burst_begin_indices, burst_end_indices
- **Units:** constant
- **Pseudocode:**

```
spike_per_bursts = []  
for idx_begin, idx_end in zip(burst_begin_indices, burst_end_indices):  
    spike_per_bursts.append(idx_end - idx_begin + 1)
```

spikes_per_burst_diff

Python efeature : Difference of number of spikes between each burst and the next one.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** spikes_per_burst
- **Units:** constant
- **Pseudocode:**

```
spikes_per_burst[:-1] - spikes_per_burst[1:]
```

spikes_in_burst1_burst2_diff

Python efeature : Difference of number of spikes between the first burst and the second one.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** spikes_per_burst_diff
- **Units:** constant
- **Pseudocode:**

```
numpy.array([spikes_per_burst_diff[0]])
```

spikes_in_burst1_burstlast_diff

Python efeature : Difference of number of spikes between the first burst and the last one.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** spikes_per_burst
- **Units:** constant

- **Pseudocode:**

```
numpy.array([spikes_per_burst[0] - spikes_per_burst[-1]])
```

strict_interburst_voltage

SpikeEvent : The voltage average in between two bursts

Iterating over the burst indices determine the first peak of each burst. Starting 5 ms after the previous peak, take the voltage average until 5 ms before the peak.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `burst_begin_indices`, `peak_indices`
- **Units:** mV
- **Pseudocode:**

```
interburst_voltage = []
for idx in burst_begin_idxs[1:]:
    ts_idx = peak_idxs[idx - 1]
    t_start = t[ts_idx] + 5
    start_idx = numpy.argwhere(t < t_start)[-1][0]

    te_idx = peak_idxs[idx]
    t_end = t[te_idx] - 5
    end_idx = numpy.argwhere(t > t_end)[0][0]

    interburst_voltage.append(numpy.mean(v[start_idx:end_idx + 1]))
```

interburst_voltage

ISI Python efeature : The voltage average in between two bursts

Iterating over the burst ISI indices determine the last peak before the burst. Starting 5 ms after that peak take the voltage average until 5 ms before the first peak of the subsequent burst.

- **Required features:** `burst_ISI_indices`, `peak_indices`
- **Units:** mV
- **Pseudocode:**

```
interburst_voltage = []
for idx in burst_ISI_idxs:
    ts_idx = peak_idxs[idx]
    t_start = time[ts_idx] + 5
    start_idx = numpy.argwhere(time < t_start)[-1][0]

    te_idx = peak_idxs[idx + 1]
    t_end = time[te_idx] - 5
    end_idx = numpy.argwhere(time > t_end)[0][0]

    interburst_voltage.append(numpy.mean(voltage[start_idx:end_idx + 1]))
```

interburst_min_indices

SpikeEvent : Indices at minimum voltage between the end of a burst and the next spike.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `peak_indices`, `burst_end_indices`
- **Units:** constant
- **Pseudocode:**

```
interburst_min = [  
    numpy.argmin(  
        v[peak_indices[i]:peak_indices[i + 1]]  
    ) for i in burst_end_indices if i + 1 < len(peak_indices)  
]
```

interburst_min_values

SpikeEvent : The minimum voltage between the end of a burst and the next spike.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `peak_indices`, `burst_end_indices`
- **Units:** mV
- **Pseudocode:**

```
interburst_min = [  
    numpy.min(  
        v[peak_indices[i]:peak_indices[i + 1]]  
    ) for i in burst_end_indices if i + 1 < len(peak_indices)  
]
```

interburst_duration

SpikeEvent : Duration between the last spike of each burst and the next spike.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `burst_end_indices`, `peak_time`
- **Units:** ms
- **Pseudocode:**

```

interburst_duration = [
    peak_time[idx + 1] - peak_time[idx]
    for idx in burst_end_indices
    if idx + 1 < len(peak_time)
]

```

interburst_15percent_indices, interburst_20percent_indices, interburst_25percent_indices, interburst_30percent_indices, interburst_40percent_indices, interburst_60percent_indices

SpikeEvent : Indices after a given percentage (15%, 20%, 25%, 30%, 40% or 60%) of the interburst duration after the fast AHP.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `postburst_fast_ahp_indices`, `burst_end_indices`, `peak_indices`
- **Units:** constant
- **Pseudocode:**

```

interburst_XXpercent_indices = []
for i, postburst_fahp_i in enumerate(postburst_fahpi):
    if i < len(burst_endi) and burst_endi[i] + 1 < len(peaki):
        time_interval = t[peaki[burst_endi[i] + 1]] - t[postburst_fahp_i]
        time_at_XXpercent = t[postburst_fahp_i] + time_interval * percentage / 100.
        index_at_XXpercent = numpy.argwhere(t >= time_at_XXpercent)[0][0]
        interburst_XXpercent_indices.append(index_at_XXpercent)

```

interburst_15percent_values, interburst_20percent_values, interburst_25percent_values, interburst_30percent_values, interburst_40percent_values, interburst_60percent_values

SpikeEvent : Voltage value after a given percentage (15%, 20%, 25%, 30%, 40% or 60%) of the interburst duration after the fast AHP.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `postburst_fast_ahp_indices`, `burst_end_indices`, `peak_indices`
- **Units:** mV
- **Pseudocode:**

```

interburst_XXpercent_values = []
for i, postburst_fahp_i in enumerate(postburst_fahpi):
    if i < len(burst_endi) and burst_endi[i] + 1 < len(peaki):
        time_interval = t[peaki[burst_endi[i] + 1]] - t[postburst_fahp_i]
        time_at_XXpercent = t[postburst_fahp_i] + time_interval * percentage / 100.

```

(continues on next page)

(continued from previous page)

```
index_at_XXpercent = numpy.argwhere(t >= time_at_XXpercent)[0][0]
interburst_XXpercent_values.append(v[index_at_XXpercent])
```

time_to_interburst_min

SpikeEvent : The time between the last spike of a burst and the minimum between that spike and the next.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `peak_indices`, `burst_end_indices`, `peak_time`
- **Units:** ms
- **Pseudocode:**

```
time_to_interburst_min = [
    t[peak_indices[i] + numpy.argmin(
        v[peak_indices[i]:peak_indices[i + 1]]
    )] - peak_time[i]
    for i in burst_end_indices if i + 1 < len(peak_indices)
]
```

time_to_postburst_slow_ahp

SpikeEvent : The time between the last spike of a burst and the slow ahp afterwards.

The number of ms to skip after the spike to skip fast AHP and look for slow AHP can be set with `sahp_start`. Default is 5.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `postburst_slow_ahp_indices`, `burst_end_indices`, `peak_time`
- **Units:** ms
- **Pseudocode:**

```
time_to_postburst_slow_ahp_py = t[postburst_slow_ahp_indices] - peak_time[burst_end_
↪indices]
```

postburst_min_indices

SpikeEvent : The indices of the minimum voltage after the end of a burst.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** peak_indices, burst_end_indices
- **Units:** constant
- **Pseudocode:**

```

postburst_min = [
    numpy.argmin(
        v[peak_indices[i]:peak_indices[i + 1]]
    ) + peak_indices[i]
    for i in burst_end_indices
    if i + 1 < len(peak_indices)
]

if len(postburst_min) < len(burst_end_indices):
    if t[burst_end_indices[-1]] < stim_end:
        end_idx = numpy.where(t >= stim_end)[0][0]
        postburst_min.append(
            numpy.argmin(
                v[peak_indices[burst_end_indices[-1]]:end_idx]
            ) + peak_indices[burst_end_indices[-1]]
        )
    else:
        postburst_min.append(
            numpy.argmin(
                v[peak_indices[burst_end_indices[-1]]:]
            ) + peak_indices[burst_end_indices[-1]]
        )

```

postburst_min_values

SpikeEvent : The minimum voltage after the end of a burst.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting ignore_first_ISI to 0.

The burst detection can be fine-tuned by changing the setting strict_burst_factor. Default value is 2.0.

- **Required features:** peak_indices, burst_end_indices
- **Units:** mV
- **Pseudocode:**

```

postburst_min = [
    numpy.min(
        v[peak_indices[i]:peak_indices[i + 1]]
    ) for i in burst_end_indices if i + 1 < len(peak_indices)
]

if len(postburst_min) < len(burst_end_indices):
    if t[burst_end_indices[-1]] < stim_end:
        end_idx = numpy.where(t >= stim_end)[0][0]
        postburst_min.append(numpy.min(
            v[peak_indices[burst_end_indices[-1]]:end_idx]
        ))

```

(continues on next page)

(continued from previous page)

```

else:
    postburst_min.append(numpy.min(
        v[peak_indices[burst_end_indices[-1]]:]
    ))

```

postburst_slow_ahp_indices

SpikeEvent : The indices of the slow AHP voltage after the end of a burst.

The number of ms to skip after the spike (in order to skip fast AHP and look for slow AHP) can be set with `sahp_start`. Default is 5.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** `peak_indices`, `burst_end_indices`
- **Units:** constant
- **Pseudocode:**

```

postburst_slow_ahp = []
for i in burst_end_indices:
    i_start = numpy.where(t >= t[peak_indices[i]] + sahp_start)[0][0]
    if i + 1 < len(peak_indices):
        postburst_slow_ahp.append(numpy.argmin(v[i_start:peak_indices[i + 1]]) + i_
→start)
    else:
        if t[burst_end_indices[-1]] < stim_end:
            end_idx = numpy.where(t >= stim_end)[0][0]
            postburst_slow_ahp.append(numpy.argmin(v[i_start:end_idx]) + i_start)
        else:
            postburst_slow_ahp.append(numpy.argmin(v[i_start:]) + i_start)

```

postburst_slow_ahp_values

SpikeEvent : The slow AHP voltage after the end of a burst.

The number of ms to skip after the spike to skip fast AHP and look for slow AHP can be set with `sahp_start`. Default is 5.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `peak_indices`, `burst_end_indices`
- **Units:** mV
- **Pseudocode:**

```

postburst_slow_ahp = []
for i in burst_end_indices:
    i_start = numpy.where(t >= t[peak_indices[i]] + sahp_start)[0][0]
    if i + 1 < len(peak_indices):
        postburst_slow_ahp.append(numpy.min(v[i_start:peak_indices[i + 1]]))
    else:
        if t[burst_end_indices[-1]] < stim_end:
            end_idx = numpy.where(t >= stim_end)[0][0]
            postburst_slow_ahp.append(numpy.min(v[i_start:end_idx]))
        else:
            postburst_slow_ahp.append(numpy.min(v[i_start:]))

```

postburst_fast_ahp_indices

SpikeEvent : The indices of the fast AHP voltage after the end of a burst.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** `peak_indices`, `burst_end_indices`
- **Units:** constant
- **Pseudocode:**

```

postburst_fahp = []
for i in burst_end_indices:
    if i + 1 < len(peak_indices):
        stop_i = peak_indices[i + 1]
    elif i + 1 < stim_end_index:
        stop_i = stim_end_index
    else:
        stop_i = len(v) - 1

    v_crop = v[peak_indices[i]:stop_i]
    # get where the voltage is going up
    crop_args = numpy.argwhere(numpy.diff(v_crop) >= 0)[: , 0]
    # the voltage should go up for at least two consecutive points
    crop_arg_arg = numpy.argwhere(numpy.diff(crop_args) == 1)[0][0]
    crop_arg = crop_args[crop_arg_arg]
    end_i = peak_indices[i] + crop_arg + 1
    # the fast ahp is between last peak of burst and the point where voltage is_
    ↪going back up
    postburst_fahp.append(numpy.argmax(v[peak_indices[i]:end_i]) + peak_indices[i])

return postburst_fahp

```

postburst_fast_ahp_values

SpikeEvent : The fast AHP voltage after the end of a burst.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `peak_indices`, `burst_end_indices`
- **Units:** mV
- **Pseudocode:**

```
postburst_fahp = []
for i in burst_end_indices:
    if i + 1 < len(peak_indices):
        stop_i = peak_indices[i + 1]
    elif i + 1 < stim_end_index:
        stop_i = stim_end_index
    else:
        stop_i = len(v) - 1

    v_crop = v[peak_indices[i]:stop_i]
    # get where the voltage is going up
    crop_args = numpy.argwhere(numpy.diff(v_crop) >= 0)[:,-1]
    # the voltage should go up for at least two consecutive points
    crop_arg_arg = numpy.argwhere(numpy.diff(crop_args) == 1)[0][0]
    crop_arg = crop_args[crop_arg_arg]
    end_i = peak_indices[i] + crop_arg + 1
    # the fast ahp is between last peak of burst and the point where voltage is
    ↪going back up
    postburst_fahp.append(numpy.min(v[peak_indices[i]:end_i]))

return postburst_fahp
```

postburst_adp_peak_indices

SpikeEvent : The indices of the small ADP peak after the fast AHP after the end of a burst.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** `postburst_fast_ahp_indices`, `postburst_slow_ahp_indices`
- **Units:** constant
- **Pseudocode:**

```
adp_peak_indices = []
for i, sahpi in enumerate(postburst_sahpi):
```

(continues on next page)

(continued from previous page)

```

if sahpi < postburst_fahpi[i]:
    continue
adppeaki = numpy.argmax(v[postburst_fahpi[i]:sahpi]) + postburst_fahpi[i]
if adppeaki != sahpi - 1:
    adp_peak_indices.append(adppeaki)

```

postburst_adp_peak_values

SpikeEvent : The small ADP peak after the fast AHP after the end of a burst.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `postburst_fast_ahp_indices`, `postburst_slow_ahp_indices`
- **Units:** mV
- **Pseudocode:**

```

adp_peak_values = []
for i, sahpi in enumerate(postburst_sahpi):
    if sahpi < postburst_fahpi[i]:
        continue
    adppeaki = numpy.argmax(v[postburst_fahpi[i]:sahpi]) + postburst_fahpi[i]
    if adppeaki != sahpi - 1:
        adp_peak_values.append(v[adppeaki])

if len(adp_peak_values) == 0:
    return None
return adp_peak_values

```

time_to_postburst_fast_ahp

SpikeEvent : Time to the fast AHP after the end of a burst.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `postburst_fast_ahp_indices`, `burst_end_indices`, `peak_time`
- **Units:** ms
- **Pseudocode:**

```

[t[fahpi] - peak_time[burst_endi[i]] for i, fahpi in enumerate(postburst_fahpi)]

```

time_to_postburst_adp_peak

SpikeEvent : Time to the small ADP peak after the fast AHP after the end of a burst.

This implementation does not assume that every spike belongs to a burst.

The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

- **Required features:** `postburst_adp_peak_indices`, `burst_end_indices`, `peak_time`
- **Units:** ms
- **Pseudocode:**

```
time_to_postburst_adp_peaks = []
n_peaks = len(peak_time)
for i, adppeaki in enumerate(postburst_adppeaki):
    # there are not always an adp peak after each burst
    # so make sure that the burst and adp peak indices are consistent
    k = 0
    while (
        burst_endi[i] + k + 1 < n_peaks and peak_time[burst_endi[i] + k + 1] <
        t[adppeaki]
    ):
        k += 1

    time_to_postburst_adp_peaks.append(t[adppeaki] - peak_time[burst_endi[i] + k])
return time_to_postburst_adp_peaks
```

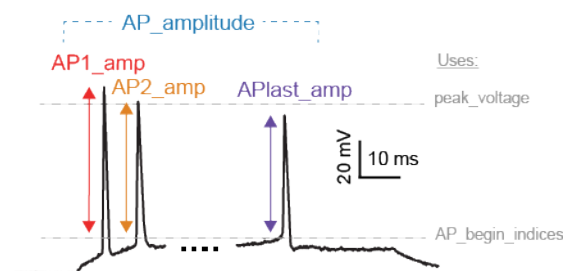
check_ais_initiation

Validation Python efeature : Checks the initiation of action potential in AIS with respect to soma. Returns True if no spike in the soma starts earlier than in the AIS.

Attention! This cannot be used with the `efel.get_feature_values` function. You have to use the `efel.pyfeatures.validation.check_ais_initiation` function, and pass it both the soma trace and the ais trace.

- **Required features:** `AP_begin_time`
- **Units:** constant

3.1.2 Spike shape features



peak_voltage

SpikeShape : The voltages at the maxima of the peaks

- **Required features:** `peak_indices`
- **Units:** mV
- **Pseudocode:**

```
peak_voltage = voltage[peak_indices]
```

AP_height

SpikeShape : Same as peak_voltage: The voltages at the maxima of the peaks

- **Required features:** peak_voltage
- **Units:** mV
- **Pseudocode:**

```
AP_height = peak_voltage
```

AP_amplitude, AP1_amp, AP2_amp, APlast_amp

SpikeShape : The relative height of the action potential from spike onset

- **Required features:** AP_begin_indices, peak_voltage (mV)
- **Units:** mV
- **Pseudocode:**

```
AP_amplitude = peak_voltage - voltage[AP_begin_indices]
AP1_amp = AP_amplitude[0]
AP2_amp = AP_amplitude[1]
APlast_amp = AP_amplitude[-1]
```

mean_AP_amplitude

SpikeShape : The mean of all of the action potential amplitudes

- **Required features:** AP_amplitude (mV)
- **Units:** mV
- **Pseudocode:**

```
mean_AP_amplitude = numpy.mean(AP_amplitude)
```

AP_Amplitude_change

SpikeShape : Difference of the amplitudes of the second and the first action potential divided by the amplitude of the first action potential

- **Required features:** AP_amplitude
- **Units:** constant
- **Pseudocode:**

```
AP_amplitude_change = (AP_amplitude[1:] - AP_amplitude[0]) / AP_amplitude[0]
```

AP_amplitude_from_voltagebase

SpikeShape : The relative height of the action potential from voltage base

- **Required features:** voltage_base, peak_voltage (mV)

- **Units:** mV
- **Pseudocode:**

```
AP_amplitude_from_voltagebase = peak_voltage - voltage_base
```

AP1_peak, AP2_peak

SpikeShape : The peak voltage of the first and second action potentials

- **Required features:** peak_voltage (mV)
- **Units:** mV
- **Pseudocode:**

```
AP1_peak = peak_voltage[0]  
AP2_peak = peak_voltage[1]
```

AP2_AP1_diff

SpikeShape : Difference amplitude of the second to first spike

- **Required features:** AP_amplitude (mV)
- **Units:** mV
- **Pseudocode:**

```
AP2_AP1_diff = AP_amplitude[1] - AP_amplitude[0]
```

AP2_AP1_peak_diff

SpikeShape : Difference peak voltage of the second to first spike

- **Required features:** peak_voltage (mV)
- **Units:** mV
- **Pseudocode:**

```
AP2_AP1_diff = peak_voltage[1] - peak_voltage[0]
```

amp_drop_first_second

SpikeShape : Difference of the amplitude of the first and the second peak

- **Required features:** peak_voltage (mV)
- **Units:** mV
- **Pseudocode:**

```
amp_drop_first_second = peak_voltage[0] - peak_voltage[1]
```

amp_drop_first_last

SpikeShape : Difference of the amplitude of the first and the last peak

- **Required features:** peak_voltage (mV)

- **Units:** mV
- **Pseudocode:**

```
amp_drop_first_last = peak_voltage[0] - peak_voltage[-1]
```

amp_drop_second_last

SpikeShape : Difference of the amplitude of the second and the last peak

- **Required features:** peak_voltage (mV)
- **Units:** mV
- **Pseudocode:**

```
amp_drop_second_last = peak_voltage[1] - peak_voltage[-1]
```

max_amp_difference

SpikeShape : Maximum difference of the height of two subsequent peaks

- **Required features:** peak_voltage (mV)
- **Units:** mV
- **Pseudocode:**

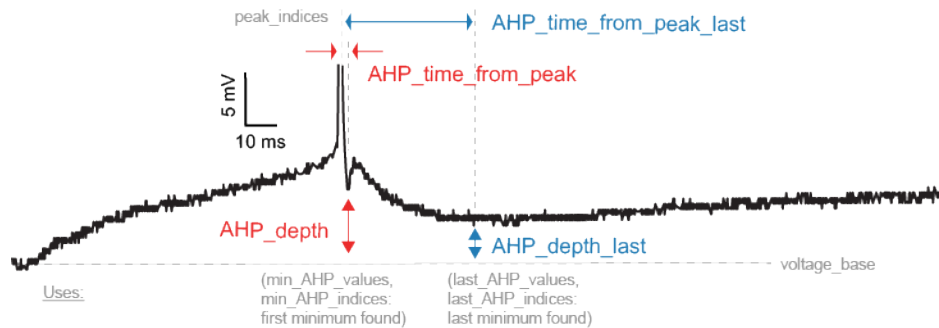
```
max_amp_difference = numpy.max(peak_voltage[:-1] - peak_voltage[1:])
```

AP_amplitude_diff

SpikeShape : Difference of the amplitude of two subsequent peaks

- **Required features:** AP_amplitude (mV)
- **Units:** mV
- **Pseudocode:**

```
AP_amplitude_diff = AP_amplitude[1:] - AP_amplitude[:-1]
```



min_AHP_indices

SpikeShape : Indices of the first after-hyperpolarization of each spike.

- **Required features:** peak_indices
- **Units:** constant

- **Pseudocode:**

```
peak_indices_plus = peak_indices
peak_indices_plus.append(len(voltage) - 1)
for i in range(peak_indices):
    min_AHP_indices.append(numpy.argmin(v[i:i + 1]) + i)
```

min_AHP_values

SpikeShape : Absolute voltage values at the first after-hyperpolarization.

- **Required features:** min_AHP_indices
- **Units:** mV

AHP_depth

SpikeShape : Relative voltage values at the first after-hyperpolarization

- **Required features:** voltage_base (mV), min_AHP_values (mV)
- **Units:** mV
- **Pseudocode:**

```
min_AHP_values = first_min_element(voltage, peak_indices)
AHP_depth = min_AHP_values[:] - voltage_base
```

AHP_depth_abs

SpikeShape : Absolute voltage values at the first after-hyperpolarization. Is the same as min_AHP_values

- **Required features:** min_AHP_values (mV)
- **Units:** mV

AHP_depth_diff

SpikeShape : Difference of subsequent relative voltage values at the first after-hyperpolarization

- **Required features:** AHP_depth (mV)
- **Units:** mV
- **Pseudocode:**

```
AHP_depth_diff = AHP_depth[1:] - AHP_depth[:-1]
```

AHP_depth_from_peak, AHP1_depth_from_peak, AHP2_depth_from_peak

SpikeShape : Voltage difference between AP peaks and first AHP depths

- **Required features:** peak_indices, min_AHP_indices
- **Units:** mV
- **Pseudocode:**

```
AHP_depth_from_peak = v[peak_indices] - v[min_AHP_indices]
AHP1_depth_from_peak = AHP_depth_from_peak[0]
AHP2_depth_from_peak = AHP_depth_from_peak[1]
```

AHP_time_from_peak

SpikeShape : Time between AP peaks and first AHP depths

- **Required features:** peak_indices, min_AHP_values (mV)
- **Units:** ms
- **Pseudocode:**

```
min_AHP_indices = first_min_element(voltage, peak_indices)
AHP_time_from_peak = t[min_AHP_indices[:]] - t[peak_indices[i]]
```

fast_AHP

SpikeShape : Voltage value of the action potential onset relative to the subsequent AHP

Ignores the last spike

- **Required features:** AP_begin_indices, min_AHP_values
- **Units:** mV
- **Pseudocode:**

```
fast_AHP = voltage[AP_begin_indices[:-1]] - voltage[min_AHP_indices[:-1]]
```

fast_AHP_change

SpikeShape : Difference of the fast AHP of the second and the first action potential divided by the fast AHP of the first action potential

- **Required features:** fast_AHP
- **Units:** constant
- **Pseudocode:**

```
fast_AHP_change = (fast_AHP[1:] - fast_AHP[0]) / fast_AHP[0]
```

AHP_depth_abs_slow

SpikeShape : Absolute voltage values at the first after-hyperpolarization starting a given number of ms (default: 5) after the peak

- **Required features:** peak_indices
- **Units:** mV

AHP_depth_slow

SpikeShape : Relative voltage values at the first after-hyperpolarization starting a given number of ms (default: 5) after the peak

- **Required features:** voltage_base (mV), AHP_depth_abs_slow (mV)
- **Units:** mV
- **Pseudocode:**

```
AHP_depth_slow = AHP_depth_abs_slow[:] - voltage_base
```

AHP_slow_time

SpikeShape : Time difference between slow AHP (see AHP_depth_abs_slow) and peak, divided by interspike interval

- **Required features:** AHP_depth_abs_slow
- **Units:** constant

ADP_peak_indices

SpikeShape : Indices of the small afterdepolarization peak

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

strict_stiminterval should be set to True for this feature to behave as expected.

- **Required features:** min_AHP_indices, min_between_peaks_indices
- **Units:** constant
- **Pseudocode:**

```
adp_peak_indices = numpy.array(  
    [numpy.argmax(v[i:j + 1]) for (i, j) in zip(min_AHP_indices, min_v_indices)]  
)
```

ADP_peak_values

SpikeShape : Absolute voltage values of the small afterdepolarization peak

strict_stiminterval should be set to True for this feature to behave as expected.

- **Required features:** min_AHP_indices, min_between_peaks_indices
- **Units:** mV
- **Pseudocode:**

```
adp_peak_values = numpy.array(  
    [numpy.max(v[i:j + 1]) for (i, j) in zip(min_AHP_indices, min_v_indices)]  
)
```

ADP_peak_amplitude

SpikeShape : Amplitude of the small afterdepolarization peak with respect to the fast AHP voltage

strict_stiminterval should be set to True for this feature to behave as expected.

- **Required features:** min_AHP_values, ADP_peak_values
- **Units:** mV
- **Pseudocode:**

```
adp_peak_amplitude = adp_peak_values - min_AHP_values
```


depolarized_base

SpikeShape : Mean voltage between consecutive spikes (from the end of one spike to the beginning of the next one)

- **Required features:** AP_end_indices, AP_begin_indices
- **Units:** mV
- **Pseudocode:**

```

depolarized_base = []
for (start_idx, end_idx) in zip(
    AP_end_indices[:-1], AP_begin_indices[1:]):
    depolarized_base.append(numpy.mean(voltage[start_idx:end_idx]))

```

min_voltage_between_spikes

SpikeShape : Minimal voltage between consecutive spikes

- **Required features:** peak_indices
- **Units:** mV
- **Pseudocode:**

```

min_voltage_between_spikes = []
for peak1, peak2 in zip(peak_indices[:-1], peak_indices[1:]):
    min_voltage_between_spikes.append(numpy.min(voltage[peak1:peak2]))

```

min_between_peaks_indices

SpikeShape : Indices of the minimal voltage between consecutive spikes

The last value of min_between_peaks_values is the minimum between last spike and stimulus end if strict stiminterval is True, or minimum between last spike and last voltage value if strict stiminterval is False

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** peak_indices
- **Units:** constant
- **Pseudocode:**

```

if strict_stiminterval:
    end_idx = numpy.argmax(t >= stim_end)[0][0]
else:
    end_idx = len(time) - 1
peak_indices_plus = peak_indices
peak_indices_plus.append(end_idx)
for i in range(peak_indices):
    min_between_peaks_indices.append(numpy.argmax(v[i:i + 1]) + i)

```

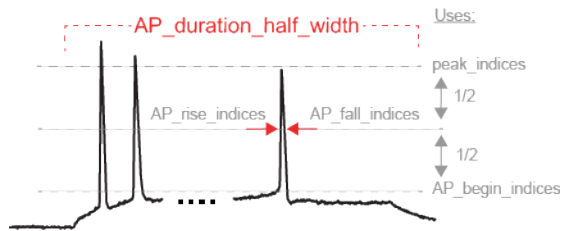
min_between_peaks_values

SpikeShape : Minimal voltage between consecutive spikes

The last value of min_between_peaks_values is the minimum between last spike and stimulus end if strict stiminterval is True, and minimum between last spike and last voltage value if strict stiminterval is False

- **Required features:** min_between_peaks_indices
- **Units:** mV
- **Pseudocode:**

```
min_between_peaks_values = v[min_between_peaks_indices]
```



AP_rise_indices

SpikeShape : indices at the half-height of the spike, in the rising phase of the spike.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** peak_indices, AP_begin_indices
- **Units:** constant
- **Pseudocode:**

```
AP_rise_indices = []
for i in range(len(peak_indices)):
    halfheight = (v[AP_begin_indices[i]] + v[peak_indices[i]] ) / 2.
    diff = abs(v[AP_begin_indices[i]:peak_indices[i]] - halfheight)
    AP_rise_indices.append(numpy.argmin(diff) + AP_begin_indices[i])
```

AP_fall_indices

SpikeShape : indices at the half-height of the spike, in the falling phase of the spike.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** peak_indices, AP_begin_indices, AP_end_indices
- **Units:** constant
- **Pseudocode:**

```
AP_fall_indices = []
for i in range(len(peak_indices)):
    halfheight = (v[peak_indices[i]] + v[AP_end_indices[i]] ) / 2.
```

(continues on next page)

(continued from previous page)

```
diff = abs(v[peak_indices[i]:AP_end_indices[i]] - halfheight)
AP_rise_indices.append(numpy.argmin(diff) + peak_indices[i])
```

AP_duration_half_width

SpikeShape : Width of spike at half spike amplitude, with spike onset as described in AP_begin_time

- **Required features:** AP_rise_indices, AP_fall_indices
- **Units:** ms
- **Pseudocode:**

```
AP_rise_indices = index_before_peak((v(peak_indices) - v(AP_begin_indices)) / 2)
AP_fall_indices = index_after_peak((v(peak_indices) - v(AP_begin_indices)) / 2)
AP_duration_half_width = t(AP_fall_indices) - t(AP_rise_indices)
```

AP_duration_half_width_change

SpikeShape : Difference of the FWHM of the second and the first action potential divided by the FWHM of the first action potential

- **Required features:** AP_duration_half_width
- **Units:** constant
- **Pseudocode:**

```
AP_duration_half_width_change = (
    AP_duration_half_width[1:] - AP_duration_half_width[0]
) / AP_duration_half_width[0]
```

AP_width

SpikeShape : Width of spike at threshold, bounded by minimum AHP

Can use strict_stiminterval compute only for data in stimulus interval.

- **Required features:** peak_indices, min_AHP_indices, threshold
- **Units:** ms
- **Pseudocode:**

```
min_AHP_indices = numpy.concatenate([[stim_start], min_AHP_indices])
for i in range(len(min_AHP_indices)-1):
    onset_index = numpy.where(v[min_AHP_indices[i]:min_AHP_indices[i+1]] >=
    threshold)[0]
    onset_time[i] = t[onset_index]
    offset_index = numpy.where(v[onset_index:min_AHP_indices[i+1]] <=
    threshold)[0]
    AP_width[i] = t(offset_index) - t(onset_index)
```

AP_duration

SpikeShape : Duration of an action potential from onset to offset

- **Required features:** AP_begin_indices, AP_end_indices
- **Units:** ms
- **Pseudocode:**

```
AP_duration = time[AP_end_indices] - time[AP_begin_indices]
```

AP_duration_change

SpikeShape : Difference of the durations of the second and the first action potential divided by the duration of the first action potential

- **Required features:** AP_duration
- **Units:** constant
- **Pseudocode:**

```
AP_duration_change = (AP_duration[1:] - AP_duration[0]) / AP_duration[0]
```

AP_width_between_threshold

SpikeShape : Width of spike at threshold, bounded by minimum between peaks

Can use strict_stiminterval to not use minimum after stimulus end.

- **Required features:** peak_indices, min_between_peaks_indices, threshold
- **Units:** ms
- **Pseudocode:**

```
min_between_peaks_indices = numpy.concatenate([[stim_start], min_between_peaks_
↪indices])
for i in range(len(min_between_peaks_indices)-1):
    onset_index = numpy.where(v[min_between_peaks_indices[i]:min_between_peaks_
↪indices[i+1]] > threshold)[0]
    onset_time[i] = t[onset_index]
    offset_time[i] = t[numpy.where(v[onset_index:min_between_peaks_indices[i+1]] <
↪threshold)[0]]
    AP_width[i] = t(offset_time[i]) - t(onset_time[i])
```

spike_half_width, AP1_width, AP2_width, APlast_width

SpikeShape : Width of spike at half spike amplitude, with the spike amplitude taken as the difference between the minimum between two peaks and the next peak

- **Required features:** peak_indices, min_AHP_indices
- **Units:** ms
- **Pseudocode:**

```

min_AHP_indices = numpy.concatenate([[stim_start], min_AHP_indices])
for i in range(1, len(min_AHP_indices)):
    v_half_width = (v[peak_indices[i-1]] + v[min_AHP_indices[i]]) / 2.
    rise_idx = numpy.where(v[min_AHP_indices[i-1]:peak_indices[i-1]] > v_half_
↪width)[0]
    v_dev = v_half_width - v[rise_idx]
    delta_v = v[rise_idx] - v[rise_idx - 1]
    delta_t = t[rise_idx] - t[rise_idx - 1]
    t_dev_rise = delta_t * v_dev / delta_v

    fall_idx = numpy.where(v[peak_indices[i-1]:min_AHP_indices[i]] < v_half_
↪width)[0]
    v_dev = v_half_width - v[fall_idx]
    delta_v = v[fall_idx] - v[fall_idx - 1]
    delta_t = t[fall_idx] - t[fall_idx - 1]
    t_dev_fall = delta_t * v_dev / delta_v
    spike_half_width[i] = t[fall_idx] + t_dev_fall - t[rise_idx] - t_dev_rise

AP1_width = spike_half_width[0]
AP2_width = spike_half_width[1]
APlast_width = spike_half_width[-1]

```

spike_width2

SpikeShape: Width of spike at half spike amplitude, with the spike onset taken as the maximum of the second derivative of the voltage in the range between the minimum between two peaks and the next peak

- **Required features:** peak_indices, min_AHP_indices
- **Units:** ms
- **Pseudocode:**

```

for i in range(len(min_AHP_indices)):
    dv2 = CentralDiffDerivative(CentralDiffDerivative(v[min_AHP_indices[i]:peak_
↪indices[i + 1]]))
    peak_onset_idx = numpy.argmax(dv2) + min_AHP_indices[i]
    v_half_width = (v[peak_indices[i + 1]] + v[peak_onset_idx]) / 2.

    rise_idx = numpy.where(v[peak_onset_idx:peak_indices[i + 1]] > v_half_width)[0]
    v_dev = v_half_width - v[rise_idx]
    delta_v = v[rise_idx] - v[rise_idx - 1]
    delta_t = t[rise_idx] - t[rise_idx - 1]
    t_dev_rise = delta_t * v_dev / delta_v

    fall_idx = numpy.where(v[peak_indices[i + 1]:] < v_half_width)[0]
    v_dev = v_half_width - v[fall_idx]
    delta_v = v[fall_idx] - v[fall_idx - 1]
    delta_t = t[fall_idx] - t[fall_idx - 1]
    t_dev_fall = delta_t * v_dev / delta_v
    spike_width2[i] = t[fall_idx] + t_dev_fall - t[rise_idx] - t_dev_rise

```

AP_begin_width, AP1_begin_width, AP2_begin_width

SpikeShape : Width of spike at spike start

- **Required features:** min_AHP_indices, AP_begin_indices
- **Units:** ms
- **Pseudocode:**

```
for i in range(len(min_AHP_indices)):
    rise_idx = AP_begin_indices[i]
    fall_idx = numpy.where(v[rise_idx + 1:min_AHP_indices[i]] < v[rise_idx])[0]
    AP_begin_width[i] = t[fall_idx] - t[rise_idx]

AP1_begin_width = AP_begin_width[0]
AP2_begin_width = AP_begin_width[1]
```

AP2_AP1_begin_width_diff

SpikeShape : Difference width of the second to first spike

- **Required features:** AP_begin_width
- **Units:** ms
- **Pseudocode:**

```
AP2_AP1_begin_width_diff = AP_begin_width[1] - AP_begin_width[0]
```

AP_begin_indices

SpikeShape : Indices of spike start.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** min_AHP_indices, peak_indices
- **Units:** constant

AP_end_indices

SpikeShape : Indices of spike end.

Attention! This feature represents indices of the interpolated time series. If you want to use it on time or voltage, make sure that you are using the interpolated output time or voltage feature, and not the time or voltage variable you gave as input.

- **Required features:** peak_indices
- **Units:** constant

AP_begin_voltage, AP1_begin_voltage, AP2_begin_voltage

SpikeShape : Voltage at spike start

- **Required features:** AP_begin_indices
- **Units:** mV
- **Pseudocode:**

```
AP_begin_voltage = v[AP_begin_indices]
AP1_begin_voltage = AP_begin_voltage[0]
AP2_begin_voltage = AP_begin_voltage[1]
```

AP_begin_time

SpikeShape : Time at spike start. Spike start is defined as where the first derivative of the voltage trace is higher than 10 V/s , for at least 5 points

- **Required features:** AP_begin_indices
- **Units:** ms
- **Pseudocode:**

```
AP_begin_time = t[AP_begin_indices]
```

AP_peak_upstroke

SpikeShape : Maximum of rise rate of spike. This feature can be used for AP phase analysis.

- **Required features:** AP_begin_indices, peak_indices
- **Units:** V/s
- **Pseudocode:**

```
ap_peak_upstroke = []
for apbi, pi in zip(ap_begin_indices, peak_indices):
    ap_peak_upstroke.append(numpy.max(dvdt[apbi:pi]))
```

AP_peak_downstroke

SpikeShape : Minimum of fall rate from spike. This feature can be used for AP phase analysis.

- **Required features:** min_AHP_indices, peak_indices
- **Units:** V/s
- **Pseudocode:**

```
ap_peak_downstroke = []
for ahpi, pi in zip(min_ahp_indices, peak_indices):
    ap_peak_downstroke.append(numpy.min(dvdt[pi:ahpi]))
```

AP_rise_time

SpikeShape : Time between the AP threshold and the peak, given a window (default: from 0% to 100% of the AP amplitude)

- **Required features:** AP_begin_indices, peak_indices, AP_amplitude
- **Units:** ms
- **Pseudocode:**

```
rise_times = []
begin_voltages = AP_amps * rise_start_perc + voltage[AP_begin_indices]
end_voltages = AP_amps * rise_end_perc + voltage[AP_begin_indices]

for AP_begin_indice, peak_indice, begin_v, end_v in zip(
    AP_begin_indices, peak_indices, begin_voltages, end_voltages
):
    voltage_window = voltage[AP_begin_indice:peak_indice]

    new_begin_indice = AP_begin_indice + numpy.min(
        numpy.where(voltage_window >= begin_v)[0]
    )
    new_end_indice = AP_begin_indice + numpy.max(
        numpy.where(voltage_window <= end_v)[0]
    )

    rise_times.append(time[new_end_indice] - time[new_begin_indice])
```

AP_fall_time

SpikeShape : Time from action potential maximum to the offset

- **Required features:** AP_end_indices, peak_indices
- **Units:** ms
- **Pseudocode:**

```
AP_fall_time = time[AP_end_indices] - time[peak_indices]
```

AP_rise_rate

SpikeShape : Voltage change rate during the rising phase of the action potential. This feature can be used for AP phase analysis.

- **Required features:** AP_begin_indices, peak_indices
- **Units:** V/s
- **Pseudocode:**

```
AP_rise_rate = (voltage[peak_indices] - voltage[AP_begin_indices]) / (
    time[peak_indices] - time[AP_begin_indices]
)
```

AP_fall_rate

SpikeShape : Voltage change rate during the falling phase of the action potential. This feature can be used for AP phase analysis.

- **Required features:** AP_end_indices, peak_indices
- **Units:** V/s
- **Pseudocode:**


```
AP_fall_rate = (voltage[AP_end_indices] - voltage[peak_indices]) / (
    time[AP_end_indices] - time[peak_indices]
)
```

AP_rise_rate_change

SpikeShape : Difference of the rise rates of the second and the first action potential divided by the rise rate of the first action potential. This feature can be used for AP phase analysis.

- **Required features:** AP_rise_rate_change
- **Units:** constant
- **Pseudocode:**

```
AP_rise_rate_change = (AP_rise_rate[1:] - AP_rise_rate[0]) / AP_rise_rate[0]
```

AP_fall_rate_change

SpikeShape : Difference of the fall rates of the second and the first action potential divided by the fall rate of the first action potential. This feature can be used for AP phase analysis.

- **Required features:** AP_fall_rate_change
- **Units:** constant
- **Pseudocode:**

```
AP_fall_rate_change = (AP_fall_rate[1:] - AP_fall_rate[0]) / AP_fall_rate[0]
```

AP_phaseslope

SpikeShape : Slope of the V, dV/dt phasespace plot at the beginning of every spike (at the point where the derivative crosses the DerivativeThreshold). This feature can be used for AP phase analysis.

- **Required features:** AP_begin_indices
- **Parameters:** AP_phaseslope_range (default=2)
- **Units:** 1/(ms)
- **Pseudocode:**

```
range_max_idx = AP_begin_indices + AP_phaseslope_range
range_min_idx = AP_begin_indices - AP_phaseslope_range
AP_phaseslope = (dvdt[range_max_idx] - dvdt[range_min_idx]) / (v[range_max_idx] -
    ↪ v[range_min_idx])
```

phaseslope_max

Python efeature : Computes the maximum of the phase slope. Attention, this feature is sensitive to interpolation timestep. This feature can be used for AP phase analysis.

- **Required features:** time, voltage
- **Units:** V/s
- **Pseudocode:**

```
phaseslope = numpy.diff(voltage) / numpy.diff(time)
phaseslope_max = numpy.array([numpy.max(phaseslope)])
```

initburst_sahp

Python efeature : Slow AHP voltage after initial burst

The end of the initial burst is detected when the ISIs frequency gets lower than `initburst_freq_threshold`, in Hz. Then the sahп is searched for the interval between `initburst_sahp_start` (in ms) after the last spike of the burst, and `initburst_sahp_end` (in ms) after the last spike of the burst.

- **Required features:** `peak_time`
- **Parameters:** `initburst_freq_threshold` (default=50), `initburst_sahp_start` (default=5), `initburst_sahp_end` (default=100)
- **Units:** mV

initburst_sahp_ssse

Python efeature : Slow AHP voltage from `steady_state_voltage_stimend` after initial burst

- **Required features:** `steady_state_voltage_stimend`, `initburst_sahp`
- **Units:** mV
- **Pseudocode:**

```
numpy.array([initburst_sahp_value[0] - ssse[0]])
```

initburst_sahp_vb

Python efeature : Slow AHP voltage from voltage base after initial burst

- **Required features:** `voltage_base`, `initburst_sahp`
- **Units:** mV
- **Pseudocode:**

```
numpy.array([initburst_sahp_value[0] - voltage_base[0]])
```

bpap_attenuation

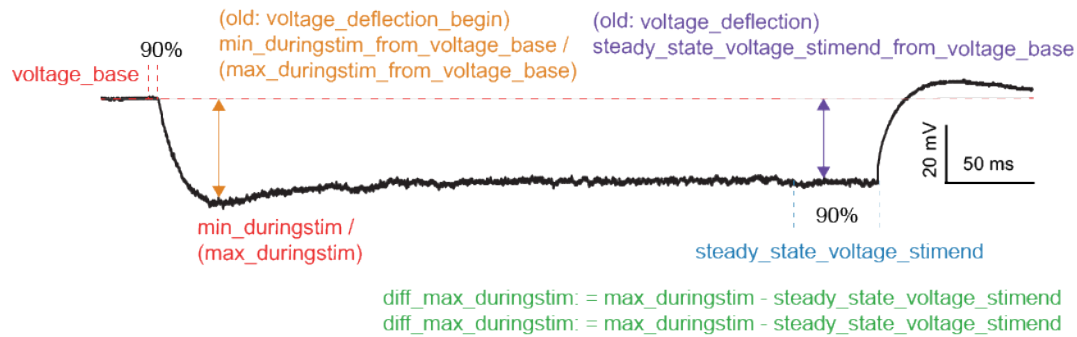
Multitrace Python efeature : Attenuation (ratio of the amplitude of the action potential in the soma and the dendrite) of the backpropagating action potential. The attenuation is computed by first subtracting the resting potential from the voltage traces.

Attention! This cannot be used with the `efel.get_feature_values` function. You have to use the `efel.pyfeatures.multitrace.bpap_attenuation` function, and pass it both the soma trace and the dendrite trace.

- **Required features:** `voltage_base`
- **Units:** constant
- **Pseudocode:**

```
return (numpy.max(v_soma) - vb_soma) / (numpy.max(v_dend) - vb_dend)
```

3.1.3 Subthreshold features



steady_state_voltage_stimend

Subthreshold : The average voltage during the last 10% of the stimulus duration.

- **Required features:** t, V, stim_start, stim_end
- **Units:** mV
- **Pseudocode:**

```
stim_duration = stim_end - stim_start
begin_time = stim_end - 0.1 * stim_duration
end_time = stim_end
steady_state_voltage_stimend = numpy.mean(voltage[numpy.where((t < end_time) & (t >
↪= begin_time))])
```

steady_state_current_stimend

Subthreshold : The average current during the last 10% of the stimulus duration.

- **Required features:** t, I, stim_start, stim_end
- **Units:** nA
- **Pseudocode:**

```
stim_duration = stim_end - stim_start
begin_time = stim_end - 0.1 * stim_duration
end_time = stim_end
steady_state_current_stimend = numpy.mean(current[numpy.where((t < end_time) & (t >
↪= begin_time))])
```

steady_state_hyper

Subthreshold : Steady state voltage during hyperpolarization for 30 data points (after interpolation)

- **Required features:** t, V, stim_start, stim_end
- **Units:** mV
- **Pseudocode:**

```
stim_end_idx = numpy.argmax(time >= stim_end)[0][0]
steady_state_hyper = numpy.mean(voltage[stim_end_idx - 35:stim_end_idx - 5])
```

steady_state_voltage

Subthreshold : The average voltage after the stimulus

- **Required features:** t, V, stim_end
- **Units:** mV
- **Pseudocode:**

```
steady_state_voltage = numpy.mean(voltage[numpy.where((t <= max(t)) & (t > stim_
↪end))])
```

voltage_base

Subthreshold : The average voltage during the last 10% of time before the stimulus.

- **Required features:** t, V, stim_start, stim_end
- **Parameters:** voltage_base_start_perc (default = 0.9), voltage_base_end_perc (default = 1.0)
- **Units:** mV
- **Pseudocode:**

```
voltage_base = numpy.mean(voltage[numpy.where(
    (t >= voltage_base_start_perc * stim_start) &
    (t <= voltage_base_end_perc * stim_start))])
```

current_base

Subthreshold : The average current during the last 10% of time before the stimulus.

- **Required features:** t, I, stim_start, stim_end
- **Parameters:** current_base_start_perc (default = 0.9), current_base_end_perc (default = 1.0), precision_threshold (default = 1e-10), current_base_mode (can be “mean” or “median”, default=“mean”)
- **Units:** nA
- **Pseudocode:**

```
current_slice = I[numpy.where(
    (t >= current_base_start_perc * stim_start) &
    (t <= current_base_end_perc * stim_start))]
if current_base_mode == "mean":
    current_base = numpy.mean(current_slice)
elif current_base_mode == "median":
    current_base = numpy.median(current_slice)
```

time_constant

Subthreshold : The membrane time constant

The extraction of the time constant requires a voltage trace of a cell in a hyper- polarized state. Starting at stim start find the beginning of the exponential decay where the first derivative of V(t) is smaller than -0.005 V/s in 5 subsequent points. The flat subsequent to the exponential decay is defined as the point where the first derivative of the voltage trace is bigger than -0.005 and the mean of the following 70 ms as well. If the voltage trace between the beginning of the decay and the flat includes more than 9 points, fit an exponential decay. Yield the time constant of that decay.

- **Required features:** t, V, stim_start, stim_end

- Units: ms
- Pseudocode:

```

min_derivative = 5e-3
decay_start_min_length = 5 # number of indices
min_length = 10 # number of indices
t_length = 70 # in ms

# get start and middle indices
stim_start_idx = numpy.where(time >= stim_start)[0][0]
# increment stimstartindex to skip a possible transient
stim_start_idx += 10
stim_middle_idx = numpy.where(time >= (stim_start + stim_end) / 2.)[0][0]

# get derivative
t_interval = time[stim_start_idx:stim_middle_idx]
dv = five_point_stencil_derivative(voltage[stim_start_idx:stim_middle_idx])
dt = five_point_stencil_derivative(t_interval)
dvd_t = dv / dt

# find start and end of decay
# has to be over deriv threshold for at least a given number of indices
pass_threshold_idxs = numpy.append(
    -1, numpy.argwhere(dvd_t > -min_derivative).flatten()
)
length_idx = numpy.argwhere(
    numpy.diff(pass_threshold_idxs) > decay_start_min_length
)[0][0]
i_start = pass_threshold_idxs[length_idx] + 1

# find flat (end of decay)
flat_idxs = numpy.argwhere(dvd_t[i_start:] > -min_derivative).flatten()
# for loop is not optimised
# but we expect the 1st few values to be the ones we are looking for
for i in flat_idxs:
    i_flat = i + i_start
    i_flat_stop = numpy.argwhere(
        t_interval >= t_interval[i_flat] + t_length
    )[0][0]
    if numpy.mean(dvd_t[i_flat:i_flat_stop]) > -min_derivative:
        break

dvd_t_decay = dvd_t[i_start:i_flat]
t_decay = time[stim_start_idx + i_start:stim_start_idx + i_flat]
v_decay_tmp = voltage[stim_start_idx + i_start:stim_start_idx + i_flat]
v_decay = abs(v_decay_tmp - voltage[stim_start_idx + i_flat])

if len(dvd_t_decay) < min_length:
    return None

# -- golden search algorithm -- #
from scipy.optimize import minimize_scalar

```

(continues on next page)

(continued from previous page)

```

def numpy_fit(x, t_decay, v_decay):
    new_v_decay = v_decay + x
    log_v_decay = numpy.log(new_v_decay)
    (slope, _), res, _, _, _ = numpy.polyfit(
        t_decay, log_v_decay, 1, full=True
    )
    range = numpy.max(log_v_decay) - numpy.min(log_v_decay)
    return res / (range * range)

max_bound = min_derivative * 1000.
golden_bracket = [0, max_bound]
result = minimize_scalar(
    numpy_fit,
    args=(t_decay, v_decay),
    bracket=golden_bracket,
    method='golden',
)

# -- fit -- #
log_v_decay = numpy.log(v_decay + result.x)
slope, _ = numpy.polyfit(t_decay, log_v_decay, 1)

time_constant = -1. / slope

```

decay_time_constant_after_stim

Subthreshold : The decay time constant of the voltage right after the stimulus

- **Required features:** t, V, stim_start, stim_end
- **Parameters:** decay_start_after_stim (default = 1.0 ms), decay_end_after_stim (default = 10.0 ms)
- **Units:** ms
- **Pseudocode:**

```

interval_indices = numpy.where(
    (time >= interval_start) & (time < interval_end))
stim_start_index = get_index(time, stim_start)
interval_time = time[interval_indices] - stim_end
interval_voltage = abs(
    voltage[interval_indices] -
    voltage[stim_start_index])

# fit
log_interval_voltage = numpy.log(interval_voltage)
slope, _ = numpy.polyfit(interval_time, log_interval_voltage, 1)

tau = -1. / slope

```

multiple_decay_time_constant_after_stim

Subthreshold : When multiple stimuli are applied, this function returns a list of decay time constants each computed on the voltage right after each stimulus.

The settings `multi_stim_start` and `multi_stim_end` are mandatory for this feature to work. Each is a list containing the start and end times of each stimulus present in the current protocol respectively.

- **Required features:** `t`, `V`, `stim_start`, `stim_end`
- **Required settings:** `multi_stim_start`, `multi_stim_end`
- **Parameters:** `decay_start_after_stim` (default = 1.0 ms), `decay_end_after_stim` (default = 10.0 ms)
- **Units:** ms
- **Pseudocode:**

```
multiple_decay_time_constant_after_stim = []
for i in range(len(number_stimuli):
    stim_start = multi_stim_start[i]
    stim_end = multi_stim_end[i]
    multiple_decay_time_constant_after_stim.append(
        decay_time_constant_after_stim(stim_start, stim_end)
    )
```

sag_time_constant

Subthreshold : The decay time constant of the exponential voltage decay from the bottom of the sag to the steady-state.

The start of the decay is taken at the minimum voltage (the bottom of the sag). The end of the decay is taken when the voltage crosses the steady state voltage minus 10% of the sag amplitude. The time constant is the slope of the linear fit to the log of the voltage. The golden search algorithm is not used, since the data is expected to be noisy and adding a parameter in the log ($\log(\text{voltage} + x)$) is likely to increase errors on the fit.

- **Required features:** `t`, `V`, `stim_start`, `stim_end`, `minimum_voltage`, `steady_state_voltage_stimend`, `sag_amplitude`
- **Units:** ms
- **Pseudocode:**

```
# get start decay
start_decay = numpy.argmin(vinterval)

# get end decay
v90 = steady_state_v - 0.1 * sag_ampl
end_decay = numpy.where((tinterval > tinterval[start_decay]) & (vinterval >=
→v90))[0][0]

v_reference = vinterval[end_decay]

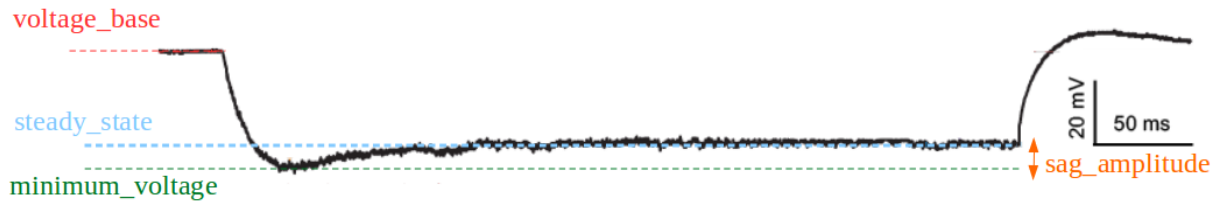
# select t, v in decay interval
interval_indices = numpy.arange(start_decay, end_decay)
interval_time = tinterval[interval_indices]
interval_voltage = abs(vinterval[interval_indices] - v_reference)

# get tau
log_interval_voltage = numpy.log(interval_voltage)
```

(continues on next page)

(continued from previous page)

```
slope, _ = numpy.polyfit(interval_time, log_interval_voltage, 1)
tau = abs(1. / slope)
```



sag_amplitude

Subthreshold : The difference between the minimal voltage and the steady state at stimend

- **Required features:** t, V, stim_start, stim_end, steady_state_voltage_stimend, minimum_voltage, voltage_deflection_stim_ssse
- **Parameters:**
- **Units:** mV
- **Pseudocode:**

```
if (voltage_deflection_stim_ssse <= 0):
    sag_amplitude = steady_state_voltage_stimend - minimum_voltage
else:
    sag_amplitude = None
```

sag_ratio1

Subthreshold : The ratio between the sag amplitude and the maximal sag extend from voltage base

- **Required features:** t, V, stim_start, stim_end, sag_amplitude, voltage_base, minimum_voltage
- **Parameters:**
- **Units:** constant
- **Pseudocode:**

```
if voltage_base != minimum_voltage:
    sag_ratio1 = sag_amplitude / (voltage_base - minimum_voltage)
else:
    sag_ratio1 = None
```

sag_ratio2

Subthreshold : The ratio between the maximal extends of sag from steady state and voltage base

- **Required features:** t, V, stim_start, stim_end, steady_state_voltage_stimend, voltage_base, minimum_voltage
- **Parameters:**
- **Units:** constant
- **Pseudocode:**


```

if voltage_base != minimum_voltage:
    sag_ratio2 = (voltage_base - steady_state_voltage_stimend) / (voltage_base -
    ↪ minimum_voltage)
else:
    sag_ratio2 = None

```

ohmic_input_resistance

Subthreshold : The ratio between the voltage deflection and stimulus current

- **Required features:** t, V, stim_start, stim_end, voltage_deflection
- **Parameters:** stimulus_current
- **Units:** M
- **Pseudocode:**

```
ohmic_input_resistance = voltage_deflection / stimulus_current
```

ohmic_input_resistance_vb_ssse

Subthreshold : The ratio between the voltage deflection (between voltage base and steady-state voltage at stimend) and stimulus current

- **Required features:** t, V, stim_start, stim_end, voltage_deflection_vb_ssse
- **Parameters:** stimulus_current
- **Units:** M
- **Pseudocode:**

```
ohmic_input_resistance_vb_ssse = voltage_deflection_vb_ssse / stimulus_current
```

voltage_deflection_vb_ssse

Subthreshold : The voltage deflection between voltage base and steady-state voltage at stimend

The voltage base used is the average voltage during the last 10% of time before the stimulus and the steady state voltage at stimend used is the average voltage during the last 10% of the stimulus duration.

- **Required features:** t, V, stim_start, stim_end, voltage_base, steady_state_voltage_stimend
- **Units:** mV
- **Pseudocode:**

```
voltage_deflection_vb_ssse = steady_state_voltage_stimend - voltage_base
```

voltage_deflection

Subthreshold : The voltage deflection between voltage base and steady-state voltage at stimend

The voltage base used is the average voltage during all of the time before the stimulus and the steady state voltage at stimend used is the average voltage of the five values before the last five values before the end of the stimulus duration.

- **Required features:** t, V, stim_start, stim_end
- **Units:** mV

- **Pseudocode:**

```
voltage_base = numpy.mean(V[t < stim_start])
stim_end_idx = numpy.where(t >= stim_end)[0][0]
steady_state_voltage_stimend = numpy.mean(V[stim_end_idx-10:stim_end_idx-5])
voltage_deflection = steady_state_voltage_stimend - voltage_base
```

voltage_deflection_begin

Subthreshold : The voltage deflection between voltage base and steady-state voltage soon after stimulation start

The voltage base used is the average voltage during all of the time before the stimulus and the steady state voltage used is the average voltage taken from 5% to 15% of the stimulus duration.

- **Required features:** t, V, stim_start, stim_end
- **Units:** mV
- **Pseudocode:**

```
voltage_base = numpy.mean(V[t < stim_start])
tstart = stim_start + 0.05 * (stim_end - stim_start)
tend = stim_start + 0.15 * (stim_end - stim_start)
condition = numpy.all((tstart < t, t < tend), axis=0)
steady_state_voltage_stimend = numpy.mean(V[condition])
voltage_deflection = steady_state_voltage_stimend - voltage_base
```

voltage_after_stim

Subthreshold : The mean voltage after the stimulus in (stim_end + 25%*end_period, stim_end + 75%*end_period)

- **Required features:** t, V, stim_end
- **Units:** mV
- **Pseudocode:**

```
tstart = stim_end + (t[-1] - stimEnd) * 0.25
tend = stim_end + (t[-1] - stimEnd) * 0.75
condition = numpy.all((tstart < t, t < tend), axis=0)
voltage_after_stim = numpy.mean(V[condition])
```

minimum_voltage

Subthreshold : The minimum of the voltage during the stimulus

- **Required features:** t, V, stim_start, stim_end
- **Units:** mV
- **Pseudocode:**

```
minimum_voltage = min(voltage[numpy.where((t >= stim_start) & (t <= stim_end))])
```

maximum_voltage

Subthreshold : The maximum of the voltage during the stimulus

- **Required features:** t, V, stim_start, stim_end
- **Units:** mV
- **Pseudocode:**

```
maximum_voltage = max(voltage[numpy.where((t >= stim_start) & (t <= stim_end))])
```

maximum_voltage_from_voltagebase

Subthreshold : Difference between maximum voltage during stimulus and voltage base

- **Required features:** maximum_voltage, voltage_base
- **Units:** mV
- **Pseudocode:**

```
maximum_voltage_from_voltagebase = maximum_voltage - voltage_base
```

depol_block_bool

Python efeature : Check for a depolarization block. Returns 1 if there is a depolarization block or a hyperpolarization block, and returns 0 otherwise.

A depolarization block is detected when the voltage stays higher than the mean of AP_begin_voltage for longer than 50 ms.

A hyperpolarization block is detected when, after stimulus start, the voltage stays below -75 mV for longer than 50 ms.

- **Required features:** AP_begin_voltage
- **Units:** constant

impedance

Python efeature : Computes the impedance given a ZAP current input and its voltage response. It will return the frequency at which the impedance is maximal, in the range (0, impedance_max_freq] Hz, with impedance_max_freq being a setting with 50.0 as a default value.

- **Required features:** current, spike_count, voltage_base, current_base
- **Units:** Hz
- **Pseudocode:**

```
normalized_voltage = voltage_trace - voltage_base
normalized_current = current_trace - current_base
if spike_count < 1: # if there is no spikes in ZAP
    fft_volt = numpy.fft.fft(normalized_voltage)
    fft_cur = numpy.fft.fft(normalized_current)
    if any(fft_cur) == 0:
        return None
    # convert dt from ms to s to have freq in Hz
    freq = numpy.fft.fftfreq(len(normalized_voltage), d=dt / 1000.)
    Z = fft_volt / fft_cur
    norm_Z = abs(Z) / max(abs(Z))
```

(continues on next page)

(continued from previous page)

```

    select_idx = numpy.swapaxes(numpy.argmax((freq > 0) & (freq <= impedance_max_
    ↪ freq)), 0, 1)[0]
    smooth_Z = gaussian_filter1d(norm_Z[select_idx], 10)
    ind_max = numpy.argmax(smooth_Z)
    return freq[ind_max]
else:
    return None

```

activation_time_constant

Python `efeature`: Time constant for an ion channel activation trace. Fits for `stim_start` to trace maximum interval as $A - B * \exp(-t/\tau)$.

Attention! For *voltage clamp* data, user should pass the current response as voltage to `efel`. See voltage clamp example for more details.

- **Required features:** time, voltage, `stim_start`, `stim_end`
- **Units:** ms
- **Pseudocode:**

```

def exp_fit(t, tau, A0, A1) -> np.ndarray | float:
    return A0 + A1 * np.exp(-t / tau)

# isolate stimulus interval
stim_start_idx = np.flatnonzero(time >= stim_start)[0]
stim_end_idx = np.flatnonzero(time >= stim_end)[0]
time_interval = time[stim_start_idx:stim_end_idx]
voltage_interval = voltage[stim_start_idx:stim_end_idx]

# keep trace going from stim_start to voltage max
max_idx = np.argmax(voltage_interval)
time_interval = time_interval[:max_idx + 1]
voltage_interval = voltage_interval[:max_idx + 1]

# correct time so that it starts from 0
time_interval -= time_interval[0]

# fit
popt, _ = curve_fit(
    exp_fit,
    time_interval,
    voltage_interval,
    p0=(1., voltage_interval[-1], voltage_interval[0] - voltage_interval[-1]),
    bounds=((0, -np.inf, -np.inf), (np.inf, np.inf, 0)), # positive tau, negative_
    ↪ A1
    nan_policy="omit",
)
time_constant = np.array([abs(popt[0])])

```

deactivation_time_constant

Python efeature : Time constant for an ion channel deactivation trace. Fits for stim_start to stim_end as $A + B * \exp(-t/\tau)$.

Attention! For *voltage clamp* data, user should pass the current response as voltage to efel. See voltage clamp example for more details.

- **Required features:** time, voltage, stim_start, stim_end
- **Units:** ms
- **Pseudocode:**

```
def exp_fit(t, tau, A0, A1) -> np.ndarray | float:
    return A0 + A1 * np.exp(-t / tau)

# isolate stimulus interval
interval_indices = np.where((time >= stim_start) & (time < stim_end))
time_interval = time[interval_indices]
voltage_interval = voltage[interval_indices]

# correct time so that it starts from 0
time_interval -= time_interval[0]

# fit
popt, _ = curve_fit(
    exp_fit,
    time_interval,
    voltage_interval,
    p0=(1., voltage_interval[-1], max(0, voltage_interval[0] - voltage_interval[-
→1])),
    bounds=((0, -np.inf, 0), np.inf), # positive tau, positive A1
    nan_policy="omit",
)
time_constant = np.array([abs(popt[0])])
```

inactivation_time_constant

Python efeature : Time constant for an ion channel inactivation trace. Fits for trace maximum to stim end interval as $A + B * \exp(-t/\tau)$. Depends on inactivation_tc_end_skip setting, which removes a given number of data points at the end of the trace, right before stim_end. This is useful to remove artifacts that would bias the fit. Default is 10 data points.

Attention! For *voltage clamp* data, user should pass the current response as voltage to efel. See voltage clamp example for more details.

- **Required features:** time, voltage, stim_start, stim_end, inactivation_tc_end_skip (default = 10)
- **Units:** ms
- **Pseudocode:**

```
def exp_fit(t, tau, A0, A1) -> np.ndarray | float:
    return A0 + A1 * np.exp(-t / tau)

# isolate stimulus interval
stim_start_idx = np.flatnonzero(time >= stim_start)[0]
```

(continues on next page)

(continued from previous page)

```

stim_end_idx = np.flatnonzero(time >= stim_end)[0]
time_interval = time[stim_start_idx:stim_end_idx - end_skip]
voltage_interval = voltage[stim_start_idx:stim_end_idx - end_skip]

# keep trace going from voltage max to stim end
# remove end of trace to remove artifacts due to stimulus change
max_idx = np.argmax(voltage_interval)
time_interval = time_interval[max_idx:]
voltage_interval = voltage_interval[max_idx:]

# correct time so that it starts from 0
time_interval -= time_interval[0]

# fit
popt, _ = curve_fit(
    exp_fit,
    time_interval,
    voltage_interval,
    p0=(1., voltage_interval[-1], voltage_interval[0] - voltage_interval[-1]),
    bounds=((0, -np.inf, 0), np.inf), # positive tau, positive A1
    nan_policy="omit",
)
time_constant = np.array([abs(popt[0])])

```

3.1.4 Extracellular features

Extracellular features can be calculated for data from a microelectrode array (MEA). These features were written by Alessio Buccino and are described in [Buccino et al., 2024](#). The features can be either absolute, computed for each channel separately, or relative, computed with respect to the channel with the largest extracellular signal amplitude.

Attention! These features cannot be extracted with the usual `get_feature_values` function. They have to be extracted using the `efel.pyfeatures.extrafeats` module. To see how to use it, have a look at the [Extracellular Features Extraction for MEA Data](#) examples.

peak_to_valley

Extracellular (absolute): time in seconds between the negative and positive peaks.

If the negative peak precedes the positive one, the value of the feature is positive. Conversely, when the positive peak precedes the negative one, the value is negative. It takes an array of MEA recordings and the sampling frequency as input.

- **Input:** waveforms(numpy.ndarray (num_waveforms x num_samples)), sampling_frequency (float: rate at which the waveforms are sampled (Hz))
- **Required features:** None
- **Units:** s
- **Pseudocode:**

```

trough_idx, peak_idx = _get_trough_and_peak_idx(waveforms)
ptv = (peak_idx - trough_idx) * (1 / sampling_frequency)
ptv[ptv == 0] = np.nan

```

halfwidth

Extracellular (absolute): waveform width in seconds.

Width of waveform at half of its amplitude in seconds. If the positive peak precedes the negative one, the value is negative. This procedure helps to maximize the shape information carried by the feature value.

- **Input:** waveforms, sampling_frequency
- **Required features:** None
- **Units:** s
- **Pseudocode:**

```
trough_idx, peak_idx = _get_trough_and_peak_idx(waveforms)
ptratio = np.empty(trough_idx.shape[0])
ptratio[:] = np.nan
for i in range(waveforms.shape[0]):
    if peak_idx[i] == 0 and trough_idx[i] == 0:
        continue
    ptratio[i] = np.abs(waveforms[i, peak_idx[i]] /
                       waveforms[i, trough_idx[i]])
return ptratio
```

repolarization_slope

Extracellular (absolute): dV/dT of the action potential between the negative peak and the baseline.

After reaching its maximum depolarization, the neuronal potential will recover. The repolarization slope is defined as the dV/dT of the action potential between the negative peak and the baseline. Optionally the function returns also the indices per waveform where the potential crosses the baseline.

- **Input:** waveforms, sampling_frequency
- **Required features:** None
- **Units:** V/s
- **Pseudocode:**

```
trough_idx, peak_idx = _get_trough_and_peak_idx(waveforms)

rslope = np.empty(waveforms.shape[0])
rslope[:] = np.nan
return_to_base_idx = np.empty(waveforms.shape[0], dtype=np.int_)
return_to_base_idx[:] = 0

time = np.arange(0, waveforms.shape[1]) * (1 / sampling_frequency) # in s
for i in range(waveforms.shape[0]):
    if trough_idx[i] == 0:
        continue

    rtn_idx = np.where(waveforms[i, trough_idx[i]:] >= 0)[0]
    if len(rtn_idx) == 0:
        continue

    return_to_base_idx[i] = (
```

(continues on next page)

(continued from previous page)

```

        rtn_idx[0] + trough_idx[i]
    ) # first time after trough, where waveform is at baseline

    if return_to_base_idx[i] - trough_idx[i] < 3:
        continue
    slope = _get_slope(
        time[trough_idx[i]:return_to_base_idx[i]],
        waveforms[i, trough_idx[i]:return_to_base_idx[i]]
    )
    rslope[i] = slope[0]

if not return_idx:
    return rslope
else:
    return rslope, return_to_base_idx

```

recovery_slope

Extracellular (absolute): After depolarization, the neuron repolarizes until the signal peaks. The recovery slope is the slope of the action potential after the peak, returning to the baseline in dV/dT . The slope is computed within a user-defined window after the peak (default = 0.7 ms).

- **Input:** waveforms, sampling_frequency, window (float: length after peak wherein to compute recovery slope (ms))
- **Required features:** None
- **Units:** V/s
- **Pseudocode:**

```

_, peak_idx = _get_trough_and_peak_idx(waveforms)
rslope = np.empty(waveforms.shape[0])
rslope[:] = np.nan

time = np.arange(0, waveforms.shape[1]) * (1 / sampling_frequency) # in s

for i in range(waveforms.shape[0]):
    if peak_idx[i] in [0, waveforms.shape[1]]:
        continue
    max_idx = int(peak_idx[i] + ((window / 1000) * sampling_frequency))
    max_idx = np.min([max_idx, waveforms.shape[1]])

    if len(time[peak_idx[i]:max_idx]) < 3:
        continue
    slope = _get_slope(
        time[peak_idx[i]:max_idx], waveforms[i, peak_idx[i]:max_idx]
    )
    rslope[i] = slope[0]

return rslope

```


neg_peak_relative

Extracellular (relative): The relative amplitude of the negative peak with respect to the negative signal peak of the channel with the largest amplitude. For the largest-amplitude channel, this feature has a value of 1.

- **Input:** waveforms
- **Required features:** None
- **Units:** constant
- **Pseudocode:**

```
fun = np.min
peak_amp = np.abs(fun(waveforms))
relative_peaks = np.abs(fun(waveforms, 1)) / peak_amp

return relative_peaks
```

pos_peak_relative

Extracellular (relative): The relative amplitude of the positive signal peak with respect to the positive signal peak of the channel with the largest amplitude. For the largest-amplitude channel, this feature has a value of 1.

- **Input:** waveforms
- **Required features:** None
- **Units:** constant
- **Pseudocode:**

```
fun = np.max
peak_amp = np.abs(fun(waveforms))
relative_peaks = np.abs(fun(waveforms, 1)) / peak_amp

return relative_peaks
```

neg_peak_diff

Extracellular (relative): The time difference between the negative signal peak with respect to the negative signal peak of the channel with the largest amplitude. For the largest-amplitude channel, this feature has a value of 0. Note that values can also be negative if the respective negative signal peak occurs earlier than the negative signal peak on the largest-amplitude channel.

- **Input:** waveforms, sampling_frequency
- **Required features:** None
- **Units:** s
- **Pseudocode:**

```
argfun = np.argmin

peak_chan = np.unravel_index(argfun(waveforms), waveforms.shape)[0]
peak_time = argfun(waveforms[peak_chan])
relative_peak_times = (argfun(waveforms, 1) - peak_time) / fs

return relative_peak_times
```

pos_peak_diff

Extracellular (relative): The time difference between the positive peak with respect to the occurrence of the positive peak of the channel with the largest amplitude. For the largest-amplitude channel, this feature has a value of 0. Note that values can also be negative if the respective positive signal peak occurs earlier than the positive signal peak on the largest-amplitude channel.

- **Input:** waveforms, sampling_frequency
- **Required features:** None
- **Units:** s
- **Pseudocode:**

```
argfun = np.argmax

peak_chan = np.unravel_index(argfun(waveforms), waveforms.shape)[0]
peak_time = argfun(waveforms[peak_chan])
relative_peak_times = (argfun(waveforms, 1) - peak_time) / fs

return relative_peak_times
```

neg_image

Extracellular (relative): Voltage values at the time of the negative signal peak on the largest-amplitude channel. The values are normalized by the negative signal-amplitude value on the largest-amplitude channel.

- **Input:** waveforms
- **Required features:** None
- **Units:** constant
- **Pseudocode:**

```
funarg = np.argmin
fun = np.min

peak_channel, peak_time = np.unravel_index(
    funarg(waveforms), waveforms.shape
)
relative_peaks = waveforms[:, peak_time] / fun(waveforms[peak_channel])

return relative_peaks
```

pos_image

Extracellular (relative): The voltage values at the time of the positive signal peak on the largest-amplitude channel. The values are normalized by the positive signal-amplitude value on the largest-amplitude channel.

- **Input:** waveforms
- **Required features:** None
- **Units:** constant
- **Pseudocode:**

```
funarg = np.argmax
fun = np.max

peak_channel, peak_time = np.unravel_index(
    funarg(waveforms), waveforms.shape
)
relative_peaks = waveforms[:, peak_time] / fun(waveforms[peak_channel])

return relative_peaks
```


PYTHON API

Copyright (c) 2015, EPFL/Blue Brain Project

This file is part of eFEL <<https://github.com/BlueBrain/eFEL>>

This library is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License version 3.0 as published by the Free Software Foundation.

This library is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public License along with this library; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

4.1 Submodules

<i>api</i>	eFEL Python API functions.
<i>io</i>	
<i>pyfeatures.cppfeature_access</i>	Module containing access functions to C++ features for Python features.
<i>pyfeatures.isi</i>	Features that are depending on the inter-spike intervals.
<i>pyfeatures.extrafeats</i>	Extracellular features functions
<i>pyfeatures.multitrace</i>	Contains the features that are computed using multiple traces.
<i>pyfeatures.pyfeatures</i>	
<i>pyfeatures.validation</i>	Contains scientific validation methods on input signals.
<i>units</i>	Module to get units of efeatures.
<i>settings</i>	efel Settings class

4.1.1 efel.api

eFEL Python API functions.

This module provides the user-facing Python API of eFEL. The convenience functions defined here call the underlying 'cppcore' library to hide the lower level API from the user.

Copyright (c) 2015, EPFL/Blue Brain Project

This file is part of eFEL <<https://github.com/BlueBrain/eFEL>>

This library is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License version 3.0 as published by the Free Software Foundation.

This library is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU Lesser General Public License for more details.

You should have received a copy of the GNU Lesser General Public License along with this library; if not, write to the Free Software Foundation, Inc., 51 Franklin Street, Fifth Floor, Boston, MA 02110-1301 USA.

Functions

FeatureNameExists(feature_name)		
<i>feature_name_exists</i> (feature_name)		Returns True if the feature name exists in eFEL, False otherwise.
getDependencyFileLocation()		
getDistance(trace, featureName, mean, std[, ...])		
getFeatureNames()		
getFeatureValues(traces, featureNames[, ...])		
getMeanFeatureValues(traces, featureNames[, ...])		
<i>get_dependency_file_location</i> ()		Gets the location of the Dependency file.
<i>get_distance</i> (trace, feature_name, mean, std)		Calculate distance value for a list of traces.
<i>get_feature_names</i> ()		Return a list with the name of all the available features
<i>get_feature_values</i> (traces, feature_names[, ...])		Calculate feature values for a list of traces.
<i>get_mean_feature_values</i> (traces, feature_names)		Convenience function that returns mean values from <i>get_feature_values</i> ()
<i>get_py_feature</i> (feature_name)		Return values of the given feature name.
<i>get_settings</i> ()		Returns the current settings of eFEL.
<i>register_feature</i> (feature_function)		Register a new feature.
<i>reset</i> ()		Resets the efel settings to their default values.
setDependencyFileLocation(location)		
setDerivativeThreshold(newDerivativeThreshold)		
setDoubleSetting(setting_name, new_value)		
setIntSetting(setting_name, new_value)		
setStrSetting(setting_name, new_value)		
setThreshold(newThreshold)		
<i>set_dependency_file_location</i> (location)		Sets the location of the Dependency file.
<i>set_derivative_threshold</i> (...)		Set the threshold for the derivative for detecting the spike onset.
<i>set_double_setting</i> (setting_name, new_value)		Set a certain double setting to a new value.
<i>set_int_setting</i> (setting_name, new_value)		Set a certain integer setting to a new value.
<i>set_setting</i> (setting_name, new_value)		Set a certain setting to a new value.
<i>set_str_setting</i> (setting_name, new_value)		Set a certain string setting to a new value.
<i>set_threshold</i> (new_threshold)		Set the spike detection threshold in the eFEL, default - 20.0

`efel.api.feature_name_exists(feature_name)`

Returns True if the feature name exists in eFEL, False otherwise.

Parameters

feature_name (str)

Return type

bool

`efel.api.get_dependency_file_location()`

Gets the location of the Dependency file.

Return type

str

Returns

Path to the location of a Dependency file.

`efel.api.get_distance(trace, feature_name, mean, std, trace_check=True, error_dist=250)`

Calculate distance value for a list of traces.

Parameters

- **trace** (dict) – Trace dict that represents one trace. The dict should have the following keys: 'T', 'V', 'stim_start', 'stim_end'
- **feature_name** (str) – Name of the the features for which to calculate the distance
- **mean** (float) – Mean to calculate the distance from
- **std** (float) – Std to scale the distance with
- **trace_check** (bool) – Let the library check if there are spikes outside of stimulus interval, default is True
- **error_dist** (float) – Distance returned when error, default is 250

Return type

float

Returns

The absolute number of standard deviation the feature is away from the mean. In case of anomalous results a value of 'error_dist' standard deviations is returned. This can happen if: a feature generates an error, there are spikes outside of the stimulus interval, the feature returns a NaN, etc.

`efel.api.get_feature_names()`

Return a list with the name of all the available features

Return type

list[str]

Returns

A list that contains all the feature names available in the eFEL. These names can be used in the feature_names argument of e.g. get_feature_values()

`efel.api.get_feature_values(traces, feature_names, parallel_map=None, return_list=True,
raise_warnings=True)`

Calculate feature values for a list of traces.

This function is the core of eFEL API. A list of traces (in the form of dictionaries) is passed as argument, together with a list of feature names.

The return value consists of a list of dictionaries, one for each input trace. The keys in the dictionaries are the names of the calculated features, the corresponding values are lists with the feature values. Beware that every feature returns an array of values. E.g. AP_amplitude will return a list with the amplitude of every action potential.

Parameters

- **traces** (list[dict]) – Every trace dict represents one trace. The dict should have the following keys: ‘T’, ‘V’, ‘stim_start’, ‘stim_end’
- **feature_names** (list[str]) – List with the names of the features to be calculated on all the traces.
- **parallel_map** (Optional[Callable]) – Map function to parallelise over the traces. Default is the serial map() function
- **return_list** (bool) – By default the function returns a list of dicts. This optional argument can disable this, so that the result of the parallel_map() is returned. Can be useful for performance reasons when an iterator is preferred.
- **raise_warnings** (bool) – Raise warning when efel c++ returns an error

Return type

Union[list, Iterator]

Returns

For every input trace a feature value dict is returned (in the same order). The dict contains the keys of ‘feature_names’, every key contains a numpy array with the feature values returned by the C++ efel code. The value is None if an error occurred during the calculation of the feature.

`efel.api.get_mean_feature_values(traces, feature_names, raise_warnings=True)`

Convenience function that returns mean values from get_feature_values()

Instead of return a list of values for every feature as get_feature_values() does, this function returns per trace one value for every feature, namely the mean value.

Parameters

- **traces** (list[dict]) – Every trace dict represents one trace. The dict should have the following keys: ‘T’, ‘V’, ‘stim_start’, ‘stim_end’
- **feature_names** (list[str]) – List with the names of the features to be calculated on all the traces.
- **raise_warnings** (bool) – Raise warning when efel c++ returns an error

Return type

list[dict]

Returns

For every input trace a feature value dict is returned (in the same order). The dict contains the keys of ‘feature_names’, every key contains the mean of the array that is returned by get_feature_values() The value is None if an error occurred during the calculation of the feature, or if the feature value array was empty.

`efel.api.get_py_feature(feature_name)`

Return values of the given feature name.

Parameters

feature_name (str)

Return type

ndarray | None

`efel.api.get_settings()`

Returns the current settings of eFEL.

Return type*Settings*

`efel.api.register_feature(feature_function)`

Register a new feature.

Parameters

feature_function (Callable)

`efel.api.reset()`

Resets the efel settings to their default values. see `efel.Settings()`

`efel.api.set_dependency_file_location(location)`

Sets the location of the Dependency file.

eFEL uses ‘Dependency’ files to let the user define versions of features to use. The installation directory of eFEL contains a default ‘DependencyV5.txt’ file. Unless users want to change this file, it is not necessary to call this function. Modifying the Dependency file can be useful in debugging.

Parameters

location (str | Path) – Path to the location of a Dependency file.

Raises

FileNotFoundError – If the path to the dependency file doesn’t exist.

Return type

None

`efel.api.set_derivative_threshold(new_derivative_threshold)`

Set the threshold for the derivative for detecting the spike onset.

Some features use a threshold on dV/dt to calculate the beginning of an action potential. This function allows you to set this threshold.

Parameters

new_derivative_threshold (float) – The new derivative threshold value (in the same units as the traces, e.g. mV/ms).

Return type

None

`efel.api.set_double_setting(setting_name, new_value)`

Set a certain double setting to a new value.

Parameters

- **setting_name** (str)
- **new_value** (float)

Return type

None

`efel.api.set_int_setting(setting_name, new_value)`

Set a certain integer setting to a new value.

Parameters

- **setting_name** (str)
- **new_value** (int)

Return type

None

`efel.api.set_setting(setting_name, new_value)`

Set a certain setting to a new value.

Parameters

- **setting_name** (str) – Name of the setting to change.
- **new_value** (int | float | str) – New value for the setting.

Return type

None

`efel.api.set_str_setting(setting_name, new_value)`

Set a certain string setting to a new value.

Parameters

- **setting_name** (str)
- **new_value** (str)

Return type

None

`efel.api.set_threshold(new_threshold)`

Set the spike detection threshold in the eFEL, default -20.0

Parameters

new_threshold (float) – The new spike detection threshold value (in the same units as the traces, e.g. mV).

Return type

None

4.1.2 efel.io

Functions

<code>extract_stim_times_from_neo_data(blocks, ...)</code>	Seeks for the <code>stim_start</code> and <code>stim_end</code> parameters inside the Neo data.
<code>load_ascii_input(file_path[, delimiter])</code>	Loads electrophysiology data from an ASCII file.
<code>load_neo_file(file_name[, stim_start, stim_end])</code>	Loads a data file using neo and converts it for eFEL readability.
<code>save_feature_to_csv(feature_values, filename)</code>	Save feature values as a CSV file.
<code>save_feature_to_json(feature_values, filename)</code>	Save feature values as a JSON file.

`efel.io.extract_stim_times_from_neo_data(blocks, stim_start, stim_end)`

Seeks for the `stim_start` and `stim_end` parameters inside the Neo data.

Parameters

- **blocks** (*Neo object blocks*) – Description of what blocks represents.
- **stim_start** (*numerical value or None*) – Start time of the stimulation in milliseconds. If not available, None should be used.
- **stim_end** (*numerical value or None*) – End time of the stimulation in milliseconds. If not available, None should be used.

Returns

A tuple containing:

- `stim_start` (numerical value or None): Start time of the stimulation in milliseconds.
- `stim_end` (numerical value or None): End time of the stimulation in milliseconds.

Return type

tuple

Notes

- `Epoch.name` should be one of “stim”, “stimulus”, “stimulation”, “current_injection”.
- First `Event.name` should be “stim_start”, “stimulus_start”, “stimulation_start”, “current_injection_start”.
- Second `Event.name` should be one of “stim_end”, “stimulus_end”, “stimulation_end”, “current_injection_end”.

`efel.io.load_ascii_input(file_path, delimiter='')`

Loads electrophysiology data from an ASCII file.

Returns: A tuple containing two numpy arrays, one for time and one for voltage.

Parameters

- **file_path** (Path | str)
- **delimiter** (str)

Return type

tuple[ndarray, ndarray]

`efel.io.load_neo_file(file_name, stim_start=None, stim_end=None, **kwargs)`

Loads a data file using neo and converts it for eFEL readability.

Parameters

- **file_name** (*string*) – Path to the Dependency file location.
- **stim_start** (*numerical value, optional*) – Start time in ms. Optional if an Epoch or two Events are in the file.
- **stim_end** (*numerical value, optional*) – End time in ms. Optional if an Epoch or two Events are in the file.
- ****kwargs** – Additional arguments for the `read()` method of Neo IO class.

Returns**Segments containing traces, formatted as**

[Segments_1, Segments_2, ..., Segments_n], where each Segments_i is [Traces_1, Traces_2, ..., Traces_n].

Return type

list of Segments

Notes

- `Epoch.name` should be “stim”, “stimulus”, “stimulation”, “current_injection”.
- First `Event.name`: “stim_start”, “stimulus_start”, “stimulation_start”, “current_injection_start”.
- Second `Event.name`: “stim_end”, “stimulus_end”, “stimulation_end”, “current_injection_end”.

`efel.io.save_feature_to_csv(feature_values, filename)`

Save feature values as a CSV file.

`efel.io.save_feature_to_json(feature_values, filename)`

Save feature values as a JSON file.

4.1.3 efel.pyfeatures.cppfeature_access

Module containing access functions to C++ features for Python features.

Functions

<code>get_cpp_feature(feature_name[, raise_warnings])</code>	Return value of feature implemented in cpp.
--	---

`efel.pyfeatures.cppfeature_access.get_cpp_feature(feature_name, raise_warnings=False)`

Return value of feature implemented in cpp.

Parameters

feature_name (str)

Return type

ndarray | None

4.1.4 efel.pyfeatures.isi

Features that are depending on the inter-spike intervals.

Functions

<code>ISI_CV()</code>	Coefficient of variation of ISIs.
<code>ISI_log_slope()</code>	The slope of a linear fit to a loglog plot of the ISI values.
<code>ISI_log_slope_skip()</code>	The slope of a linear fit to a loglog plot of the ISI values, but not taking into account the first ISI values.
<code>ISI_semiplot_slope()</code>	The slope of a linear fit to a semilog plot of the ISI values.
<code>ISI_values()</code>	Get all ISIs, inter-spike intervals.
<code>ISIs()</code>	Get all ISIs, inter-spike intervals.
<code>burst_ISI_indices()</code>	Calculate burst ISI indices based on burst factor and ISI values.
<code>burst_mean_freq()</code>	Calculate the mean frequency of bursts.
<code>initburst_sahp()</code>	SlowAHP voltage after initial burst.
<code>interburst_voltage()</code>	The voltage average in between two bursts.
<code>inv_ISI_values()</code>	Calculate the inverse of ISI values.
<code>inv_fifth_ISI()</code>	Calculate the inverse of the fifth ISI.
<code>inv_first_ISI()</code>	Calculate the inverse of the first ISI.
<code>inv_fourth_ISI()</code>	Calculate the inverse of the fourth ISI.
<code>inv_last_ISI()</code>	Calculate the inverse of the last ISI.
<code>inv_second_ISI()</code>	Calculate the inverse of the second ISI.
<code>inv_third_ISI()</code>	Calculate the inverse of the third ISI.
<code>irregularity_index()</code>	Calculate the irregularity index of ISI values.
<code>single_burst_ratio()</code>	Calculates the single burst ratio.
<code>strict_burst_number()</code>	Calculate the strict burst number.

`efel.pyfeatures.isi.ISI_CV()`

Coefficient of variation of ISIs.

If the `ignore_first_ISI` flag is set, the first ISI will be ignored.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.ISI_log_slope()`

The slope of a linear fit to a loglog plot of the ISI values.

If the `ignore_first_ISI` flag is set, the first ISI will be ignored.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.ISI_log_slope_skip()`

The slope of a linear fit to a loglog plot of the ISI values, but not taking into account the first ISI values.

Uses the `spike_skipf` and `max_spike_skip` settings to determine how many ISIs to skip. .

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.ISI_semilog_slope()`

The slope of a linear fit to a semilog plot of the ISI values.

If the `ignore_first_ISI` flag is set, the first ISI will be ignored.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.ISI_values()`

Get all ISIs, inter-spike intervals.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.ISIs()`

Get all ISIs, inter-spike intervals.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.burst_ISI_indices()`

Calculate burst ISI indices based on burst factor and ISI values.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.burst_mean_freq()`

Calculate the mean frequency of bursts.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.initburst_sahp()`

SlowAHP voltage after initial burst.

Return type

`ndarray` | `None`

`efel.pyfeatures.isi.interburst_voltage()`

The voltage average in between two bursts.

Return type

ndarray | None

`efel.pyfeatures.isi.inv_ISI_values()`

Calculate the inverse of ISI values.

Return type

ndarray | None

`efel.pyfeatures.isi.inv_fifth_ISI()`

Calculate the inverse of the fifth ISI.

Return type

ndarray | None

`efel.pyfeatures.isi.inv_first_ISI()`

Calculate the inverse of the first ISI.

Return type

ndarray | None

`efel.pyfeatures.isi.inv_fourth_ISI()`

Calculate the inverse of the fourth ISI.

Return type

ndarray | None

`efel.pyfeatures.isi.inv_last_ISI()`

Calculate the inverse of the last ISI.

Return type

ndarray | None

`efel.pyfeatures.isi.inv_second_ISI()`

Calculate the inverse of the second ISI.

Return type

ndarray | None

`efel.pyfeatures.isi.inv_third_ISI()`

Calculate the inverse of the third ISI.

Return type

ndarray | None

`efel.pyfeatures.isi.irregularity_index()`

Calculate the irregularity index of ISI values.

If the `ignore_first_ISI` flag is set, the first ISI will be ignored.

Return type

ndarray | None

`efel.pyfeatures.isi.single_burst_ratio()`

Calculates the single burst ratio.

The ratio is the length of the first ISI over the average of the rest. If the `ignore_first_ISI` flag is set, the first ISI will be ignored.

Return type

ndarray | None

`efel.pyfeatures.isi.strict_burst_number()`

Calculate the strict burst number.

This implementation does not assume that every spike belongs to a burst. The first spike is ignored by default. This can be changed by setting `ignore_first_ISI` to 0.

The burst detection can be fine-tuned by changing the setting `strict_burst_factor`. Default value is 2.0.

Return type

ndarray

4.1.5 efel.pyfeatures.extrafeats

Extracellular features functions

Functions

<code>calculate_features</code> (waveforms, sampling_frequency)	Calculate features for all waveforms
<code>halfwidth</code> (waveforms, sampling_frequency[, ...])	Width of waveform at half of its amplitude.
<code>peak_image</code> (waveforms[, sign])	Normalized amplitude at the time of peak minimum or maximum.
<code>peak_time_diff</code> (waveforms, fs[, sign])	Peak time differences with respect to channel with largest amplitude.
<code>peak_to_valley</code> (waveforms, sampling_frequency)	Time between trough and peak.
<code>peak_trough_ratio</code> (waveforms)	Normalized ratio of peak height over trough depth
<code>recovery_slope</code> (waveforms, ...)	Return the recovery slope of input waveforms.
<code>relative_amplitude</code> (waveforms[, sign])	Normalized amplitude with respect to channel with largest amplitude.
<code>repolarization_slope</code> (waveforms, ...[, ...])	Return slope of repolarization period between trough and baseline

`efel.pyfeatures.extrafeats.calculate_features(waveforms, sampling_frequency, upsample=None, feature_names=None, recovery_slope_window=0.7)`

Calculate features for all waveforms

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute features for
- **sampling_frequency** – float rate at which the waveforms are sampled (Hz)
- **feature_names** – list or None (if None, compute all) features to compute
- **recovery_slope_window** – float window length in ms after peak wherein recovery slope is computed

Returns**Dictionary with computed metrics.**

Keys are the metric names, values are the computed features

Return type

dict (num_waveforms x num_metrics)

`efel.pyfeatures.extrafeats.halfwidth(waveforms, sampling_frequency, return_idx=False)`

Width of waveform at half of its amplitude. If the peak precedes the trough, halfwidth is negative.

Computes the width of the waveform peak at half its height

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute features for
- **sampling_frequency** – float rate at which the waveforms are sampled (Hz)
- **return_idx** – bool if true, also returns index of threshold crossing before and index of threshold crossing after peak

Returns

Halfwidth of the waveforms or (Halfwidth of the waveforms, index_cross_pre_peak, index_cross_post_peak)

Return type

np.ndarray or (np.ndarray, np.ndarray, np.ndarray)

`efel.pyfeatures.extrafeats.peak_image(waveforms, sign='negative')`

Normalized amplitude at the time of peak minimum or maximum.

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute features for
- **sign** – str “positive” | “negative”

Returns

Peak images for the waveforms

Return type

np.ndarray

`efel.pyfeatures.extrafeats.peak_time_diff(waveforms, fs, sign='negative')`

Peak time differences with respect to channel with largest amplitude.

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute features for
- **fs** – float Sampling rate in Hz
- **sign** – str “positive” | “negative”

Returns

Peak time differences for the waveforms

Return type

np.ndarray

`efel.pyfeatures.extrafeats.peak_to_valley(waveforms, sampling_frequency)`

Time between trough and peak. If the peak precedes the trough, peak_to_valley is negative.

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute feature for
- **sampling_frequency** – float rate at which the waveforms are sampled (Hz)

Returns

peak_to_valley in seconds

Return type

np.ndarray (num_waveforms)

`efel.pyfeatures.extrafeats.peak_trough_ratio(waveforms)`

Normalized ratio of peak height over trough depth

Assumes baseline is 0

Parameters

waveforms – numpy.ndarray (num_waveforms x num_samples) waveforms to compute feature for

Returns

Peak to trough ratio

Return type

np.ndarray (num_waveforms)

`efel.pyfeatures.extrafeats.recovery_slope(waveforms, sampling_frequency, window)`

Return the recovery slope of input waveforms. After repolarization, the neuron hyperpolarizes until it peaks. The recovery slope is the slope of the action potential after the peak, returning to the baseline in dV/dT. The slope is computed within a user-defined window after the peak.

Takes a numpy array of waveforms and returns an array with recovery slopes per waveform.

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute features for
- **sampling_frequency** – float rate at which the waveforms are sampled (Hz)
- **window** – float length after peak wherein to compute recovery slope (ms)

Returns

Recovery slope of the waveforms

Return type

np.ndarray

`efel.pyfeatures.extrafeats.relative_amplitude(waveforms, sign='negative')`

Normalized amplitude with respect to channel with largest amplitude.

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute features for
- **sign** – str “positive” | “negative”

Returns

Relative amplitudes for the waveforms

Return type

np.ndarray

`efel.pyfeatures.extrafeats.repolarization_slope(waveforms, sampling_frequency, return_idx=False)`

Return slope of repolarization period between trough and baseline

After reaching its maximum polarization, the neuron potential will recover. The repolarization slope is defined as the dV/dT of the action potential between trough and baseline.

Optionally the function returns also the indices per waveform where the potential crosses baseline.

Parameters

- **waveforms** – numpy.ndarray (num_waveforms x num_samples) waveforms to compute features for
- **sampling_frequency** – float rate at which the waveforms are sampled (Hz)
- **return_idx** – bool if true, also returns index of threshold crossing before and index of threshold crossing after peak

Returns**Repolarization slope of the**

waveforms or (Repolarization slope of the waveforms, return to base index)

Return type

np.ndarray or (np.ndarray, np.ndarray)

4.1.6 efel.pyfeatures.multitrace

Contains the features that are computed using multiple traces.

Functions

<code>bpap_attenuation(soma_trace, dendrite_trace)</code>	Computes the attenuation of backpropagating action potential.
---	---

`efel.pyfeatures.multitrace.bpap_attenuation(soma_trace, dendrite_trace)`

Computes the attenuation of backpropagating action potential.

Backpropagating action potential is the action potential that is initiated in the soma and propagates to the dendrite. The attenuation is the ratio of the amplitude of the action potential in the soma and the dendrite. The attenuation is computed by first subtracting the resting potential from the voltage traces.

Parameters

- **soma_trace** (dict)
- **dendrite_trace** (dict)

Return type

float

4.1.7 efel.pyfeatures.pyfeatures

Functions

<code>Spikecount()</code>	
<code>Spikecount_stimint()</code>	
<code>activation_time_constant()</code>	Time constant for an ion channel activation trace.
<code>burst_number()</code>	The number of bursts.
<code>current()</code>	Get current trace
<code>deactivation_time_constant()</code>	Time constant for an ion channel deactivation trace.
<code>depol_block()</code>	Check for a depolarization block
<code>depol_block_bool()</code>	Wrapper around the <code>depol_block</code> feature.
<code>exp_fit(t, tau, A0, A1)</code>	Exponential function used in exponential fitting.
<code>impedance()</code>	
<code>inactivation_time_constant()</code>	Time constant for an ion channel inactivation trace.
<code>initburst_sahp_ssse()</code>	SlowAHP voltage from steady_state_voltage_stimend after initial burst
<code>initburst_sahp_vb()</code>	SlowAHP voltage from voltage base after initial burst
<code>phaseslope_max()</code>	Calculate the maximum phase slope.
<code>spike_count()</code>	Get spike count.
<code>spike_count_stimint()</code>	Get spike count within stimulus interval.
<code>spikes_in_burst1_burst2_diff()</code>	Calculate the diff between the spikes in 1st and 2nd bursts
<code>spikes_in_burst1_burstlast_diff()</code>	Calculate the diff between the spikes in 1st and last bursts
<code>spikes_per_burst()</code>	Calculate the number of spikes per burst
<code>spikes_per_burst_diff()</code>	Calculate the diff between the spikes in each burst and the next one
<code>time()</code>	Get time trace.
<code>trace_check()</code>	Returns <code>np.array([0])</code> if there are no spikes outside stimulus boundaries.
<code>voltage()</code>	Get voltage trace.

`efel.pyfeatures.pyfeatures.activation_time_constant()`

Time constant for an ion channel activation trace. Fits for `stim_start` to trace maximum interval as $A - B * \exp(-t/\tau)$.

Return type

`ndarray` | `None`

`efel.pyfeatures.pyfeatures.burst_number()`

The number of bursts.

Return type

`ndarray`

`efel.pyfeatures.pyfeatures.current()`

Get current trace

`efel.pyfeatures.pyfeatures.deactivation_time_constant()`

Time constant for an ion channel deactivation trace. Fits for `stim_start` to `stim_end` as $A + B * \exp(-t/\tau)$.

Return type

`ndarray` | `None`

`efel.pyfeatures.pyfeatures.depol_block()`

Check for a depolarization block

`efel.pyfeatures.pyfeatures.depol_block_bool()`

Wrapper around the `depol_block` feature. Returns [1] if `depol_block` is None, [0] otherwise.

`efel.pyfeatures.pyfeatures.exp_fit(t, tau, A0, A1)`

Exponential function used in exponential fitting.

Parameters

- ***t*** (*ndarray or float*) – time series
- ***tau*** (*float*) – time constant
- ***A0*** (*float*) – constant added to the exponential
- ***A1*** (*float*) – constant multiplying the exponential

Return type

`ndarray | float`

`efel.pyfeatures.pyfeatures.inactivation_time_constant()`

Time constant for an ion channel inactivation trace. Fits for trace maximum to stim end interval as $A + B * \exp(-t/\tau)$.

Return type

`ndarray | None`

`efel.pyfeatures.pyfeatures.initburst_sahp_ssse()`

SlowAHP voltage from steady_state_voltage_stimend after initial burst

`efel.pyfeatures.pyfeatures.initburst_sahp_vb()`

SlowAHP voltage from voltage base after initial burst

`efel.pyfeatures.pyfeatures.phaseslope_max()`

Calculate the maximum phase slope.

Return type

`ndarray | None`

`efel.pyfeatures.pyfeatures.spike_count()`

Get spike count.

Return type

`ndarray`

`efel.pyfeatures.pyfeatures.spike_count_stimint()`

Get spike count within stimulus interval.

Return type

`ndarray`

`efel.pyfeatures.pyfeatures.spikes_in_burst1_burst2_diff()`

Calculate the diff between the spikes in 1st and 2nd bursts

`efel.pyfeatures.pyfeatures.spikes_in_burst1_burstlast_diff()`

Calculate the diff between the spikes in 1st and last bursts

`efel.pyfeatures.pyfeatures.spikes_per_burst()`

Calculate the number of spikes per burst

`efel.pyfeatures.pyfeatures.spikes_per_burst_diff()`

Calculate the diff between the spikes in each burst and the next one

`efel.pyfeatures.pyfeatures.time()`

Get time trace.

Return type

ndarray | None

`efel.pyfeatures.pyfeatures.trace_check()`

Returns np.array([0]) if there are no spikes outside stimulus boundaries.

Returns None upon failure.

Return type

ndarray | None

`efel.pyfeatures.pyfeatures.voltage()`

Get voltage trace.

Return type

ndarray | None

4.1.8 efel.pyfeatures.validation

Contains scientific validation methods on input signals.

Functions

<code>check_ais_initiation(soma_trace, ais_trace)</code>	Checks the initiation of action potential in AIS with respect to soma.
--	--

`efel.pyfeatures.validation.check_ais_initiation(soma_trace, ais_trace)`

Checks the initiation of action potential in AIS with respect to soma.

Parameters

- **soma_trace** (dict)
- **ais_trace** (dict)

Return type

bool

4.1.9 efel.units

Module to get units of efeatures.

Functions

<code>get_unit(feature_name)</code>	Get the unit of a feature.
-------------------------------------	----------------------------

`efel.units.get_unit(feature_name)`

Get the unit of a feature.

Parameters

feature_name (str)

Return type
str

4.1.10 efel.settings

efel Settings class

Classes

<i>Settings</i> ([Threshold, DerivativeThreshold, ...])	eFEL settings class.
---	----------------------

```
class efel.settings.Settings(Threshold=-20.0, DerivativeThreshold=10.0,
                             DownDerivativeThreshold=-12.0,
                             dependencyfile_path='/home/docs/checkouts/readthedocs.org/user_builds/efel/envs/latest/lib/python3.6/site-packages/efel/DependencyV5.txt', spike_skipf=0.1, max_spike_skip=2,
                             interp_step=0.1, burst_factor=1.5, strict_burst_factor=2.0,
                             voltage_base_start_perc=0.9, voltage_base_end_perc=1.0,
                             current_base_start_perc=0.9, current_base_end_perc=1.0,
                             rise_start_perc=0.0, rise_end_perc=1.0, initial_perc=0.1,
                             min_spike_height=20.0, strict_stiminterval=False,
                             initburst_freq_threshold=50, initburst_sahp_start=5,
                             initburst_sahp_end=100, DerivativeWindow=3, voltage_base_mode='mean',
                             current_base_mode='mean', precision_threshold=1e-10, sahп_start=5.0,
                             ignore_first_ISI=True, impedance_max_freq=50.0, AP_phaseslope_range=2,
                             inactivation_tc_end_skip=10)
```

eFEL settings class.

Threshold

Spike detection threshold (default: -20.0).

Type
float

DerivativeThreshold

Threshold value for derivative calculations (default: 10.0).

Type
float

DownDerivativeThreshold

Threshold value for downward derivative calculations (default: -12.0).

Type
float

dependencyfile_path

Path to the dependency file (default: 'DependencyV5.txt').

Type
str

spike_skipf

Fraction of spikes to skip (default: 0.1).

Type
float

max_spike_skip

Maximum number of spikes to skip (default: 2).

Type

int

interp_step

Interpolation step (default: 0.1).

Type

float

burst_factor

Burst factor (default: 1.5).

Type

float

strict_burst_factor

Strict burst factor (default: 2.0).

Type

float

voltage_base_start_perc

Voltage base start percentage (default: 0.9).

Type

float

voltage_base_end_perc

Voltage base end percentage (default: 1.0).

Type

float

current_base_start_perc

Current base start percentage (default: 0.9).

Type

float

current_base_end_perc

Current base end percentage (default: 1.0).

Type

float

rise_start_perc

Rise start percentage (default: 0.0).

Type

float

rise_end_perc

Rise end percentage (default: 1.0).

Type

float

initial_perc

Initial percentage (default: 0.1).

Type

float

min_spike_height

Minimum spike height (default: 20.0).

Type

float

strict_stiminterval

Strict stimulus interval (default: False).

Type

bool

initburst_freq_threshold

Initial burst frequency threshold (default: 50).

Type

int

initburst_sahp_start

Initial burst SAHP start (default: 5).

Type

int

initburst_sahp_end

Initial burst SAHP end (default: 100).

Type

int

DerivativeWindow

Derivative window (default: 3).

Type

int

voltage_base_mode

Voltage base mode (default: “mean”).

Type

str

current_base_mode

Current base mode (default: “mean”).

Type

str

precision_threshold

Precision threshold (default: 1e-10).

Type

float

sahp_start

SAHP start (default: 5.0).

Type

float

ignore_first_ISI

Ignore first ISI (default: True).

Type

bool

impedance_max_freq

Impedance maximum frequency (default: 50.0).

Type

float

inactivation_tc_end_skip

number of data points to skip before stim end for inactivation_time_constant feature (default: 10).

Type

int

Parameters

- **Threshold** (float)
- **DerivativeThreshold** (float)
- **DownDerivativeThreshold** (float)
- **dependencyfile_path** (str)
- **spike_skipf** (float)
- **max_spike_skip** (int)
- **interp_step** (float)
- **burst_factor** (float)
- **strict_burst_factor** (float)
- **voltage_base_start_perc** (float)
- **voltage_base_end_perc** (float)
- **current_base_start_perc** (float)
- **current_base_end_perc** (float)
- **rise_start_perc** (float)
- **rise_end_perc** (float)
- **initial_perc** (float)
- **min_spike_height** (float)
- **strict_stiminterval** (bool)
- **initburst_freq_threshold** (int)
- **initburst_sahp_start** (int)
- **initburst_sahp_end** (int)

- **DerivativeWindow** (int)
- **voltage_base_mode** (str)
- **current_base_mode** (str)
- **precision_threshold** (float)
- **sahp_start** (float)
- **ignore_first_ISI** (bool)
- **impedance_max_freq** (float)
- **AP_phaseslope_range** (int)
- **inactivation_tc_end_skip** (int)

reset_to_default()

Reset settings to their default values

set_setting(*setting_name*, *new_value*)

Set a certain setting to a new value.

Parameters

- **setting_name** (*str*) – Name of the setting to be modified.
- **new_value** (*Union[int, float, str, bool]*) – New value for the setting.

Raises

- **ValueError** – If the value is of the wrong type.
- **FileNotFoundError** – If the path to the dependency file does not exist
- **(for 'dependencyfile_path' setting).** –

Return type

None

CHANGELOG

All notable changes to this project will be documented in this file.

The format is based on [Keep a Changelog](#), and this project adheres to [Semantic Versioning](#).

5.1 5.7.11 - 2024-11

- Removed pdf documentation file. Reason: There was no substantial information in the pdf that were not already present in the official doc.
- added an example for using settings, and added this example to the docs
- added the (already implemented) following features to docs:
 - check_ais_initiation
 - bpap_attenuation
 - peak_indices
 - AP_rise_indices
 - AP_end_indices
 - AP_fall_indices
 - min_AHP_indices
 - AP_begin_indices
 - min_between_peaks_indices
 - burst_begin_indices
 - burst_end_indices
 - ADP_peak_indices
 - interburst_min_indices
 - postburst_min_indices
 - postburst_slow_ahp_indices
 - postburst_fast_ahp_indices
 - postburst_adp_peak_indices
 - interburst_15percent_indices
 - interburst_20percent_indices
 - interburst_25percent_indices

- interburst_30percent_indices
- interburst_40percent_indices
- interburst_60percent_indices
- burst_ISI_indices

5.2 5.7.9 - 2024-09

- Reduces redundant dependency tree initializations in cppcore, improving feature extraction performance by up to 2.5x. Ensures getFeatureValues avoids unnecessary resets; use efel.reset to reload dependencies when needed.

5.3 5.6.7 - 2024-08

- Fixed impedance feature (frequency index was inconsistent with smooth Z)

5.4 5.7.1 - 2024-07

- When stimulus starts at 0 for impedance (which can happen often), use steady_state_voltage_stimend to get holding voltage, and equivalent for holding current. Since at the end of the stimulus, we expect the current and voltage to vary rapidly around the holding value, this is a good enough proxy to get it.
- implemented test_steady_state_current_stimend, along with documentation and test
- fixed current interpolation
- fixed stim_end value in impedance test data file

5.5 5.7.0 - 2024-07

- Adding extracellular features: *“peak_to_valley”*, *“halfwidth”*, *“peak_trough_ratio”*, *“repolarization_slope”*, *“recovery_slope”*, *“neg_peak_relative”*, *“pos_peak_relative”*, *“neg_peak_diff”*, *“pos_peak_diff”*, *“neg_image”* and *“pos_image”*
- Add example for extracellular features
- Documentation update to mention features that can be used for phase analysis: *“AP_phaseslope”*, *“phaseslope_max”*, *“AP_fall_rate”*, *“AP_fall_rate_change”*, *“AP_peak_downstroke”*, *“AP_peak_upstroke”*, *“AP_rise_rate”* and *“AP_rise_rate_change”*

5.6 5.6.29 - 2024-06

- Added AP_phaseslope_range to efel.settings as int with default value of 2

5.7 5.6.27 - 2024-05

- Fixed edge case (1 spike and no min_AHP_indices dependency) in AP_begin_indices

5.8 5.6.20 - 2024-05

- Refactored LibV1, LibV2, LibV3, LibV5 into more meaningful categories: SpikeEvent, SpikeShape and Sub-threshold.

5.9 5.6.6 - 2024-04

- Adding AP_height to documentation

5.10 5.6.0 - 2024-02

- Reduce 3 alternative implementations to get ISIs into 1.
- “all_ISI_values” is recommended, “ISI_values” and “ISIs” are deprecated.
- The features depending on “ISI_values” are moved to Python and now they depend on “all_ISI_values”.
- BUGFIX: single_burst_ratio, irregularity_index, burst_mean_freq, interburst_voltage features were ignoring the first two ISIs when the ignore_first_ISI was set.
- Added new feature: inv_ISI_values that computes and returns all of the inverse isi values.

5.11 5.5.5 - 2024-01

- Type annotate api.py’s functions.
- Deprecate camel case function names in api.py.
- Start using same requirements_docs.txt in readthedocs and tox.
- Enable autodoc and typehints in the API documentation.
- Fix docstring errors in the io module.
- Add changelog to the documentation.

5.12 [5.5.4] - 2024-01

- New feature: phaseslope_max

5.13 5.5.3 - 2024-01

- Add type stub for cppcore module to make Python recognise the C++ functions’ arguments and return values.

5.14 5.5.0 - 2024-01

5.14.1 C++ changes

- AP_end_indices, AP_rise_time, AP_fall_time, AP_rise_rate, AP_fall_rate do not take into account peaks before stim_start anymore.
- New test and test data for spontaneous firing case. The data is provided by github user SzaBoglarka using cell <https://modeldb.science/114047>.

5.15 5.4.0 - 2024-01

5.15.1 C++ changes

- New C++ function *getFeatures* replaced *getVec*.
- *getFeatures* automatically handles failures & distinguishes empty results from failures.
- Centralized error handling in *getFeatures* shortens the code by removing repetitions.
- C++ features' access is restricted. Read-only references are marked *const*.
- Removed wildcard features from C++ API. Use of Python is encouraged for that purpose.

5.15.2 Python changes

- *bpap_attenuation* feature is added to the Python API.
- *Spikecount*, *Spikecount_stimint*, *burst_number*, *strict_burst_number* and *trace_check* features migrated to Python from C++.
- *check_ais_initiation* is added to the Python API.

DEVELOPER'S GUIDE

Contents

- *Developer's Guide*
 - *Requirements*
 - *Forking and cloning the git repository*
 - *Makefile*
 - *Adding a new eFeature*
 - * *Picking a name*
 - * *Creating a branch*
 - * *Implementation*
 - * *Updating relevant files*
 - * *Adding a test*
 - * *Add documentation*
 - * *Pull request*

6.1 Requirements

As a developer you will need some extra requirements

- To get the latest source code: [Git](#)
- To run the tests: [Pytest](#)
- To build the documentation: [Sphinx](#), and `pdflatex` (e.g. from [Mactex](#))

6.2 Forking and cloning the git repository

To make changes to the eFEL, one first needs to fork the eFEL:

<https://help.github.com/articles/fork-a-repo/>

Then one creates a local clone of the git repository on your computer:

```
git clone https://github.com/yourgithubusername/eFEL.git
```

After changes are made, they should be pushed back to your github account. Then a pull request can be created:

```
https://help.github.com/articles/using-pull-requests/
```

6.3 Makefile

To simplify certain tasks for developers, a Makefile is provided in the root of the eFEL project. This Makefile has the following targets

- **install**: installs the eFEL using pip from the working directory
- **test**: run the installation and all the tests
- **doc**: build the sphinx and latex documentation
- **clean**: clean up the build directories
- **pypi**: run test target and upload to pypi
- **push**: clean the build, update the version from the git hash, install eFEL, run the tests, build the doc, and push the documentation and source to github

6.4 Adding a new eFeature

Adding a new eFeature requires several steps.

6.4.1 Picking a name

Try to be specific in the name of the eFeature, because in the future you or somebody else might want to develop an eFeature with slightly different behavior. Don't be afraid to use long names, e.g. 'min_voltage_between_spikes' is perfectly ok.

6.4.2 Creating a branch

Create a git branch with the name of the new eFeature:

```
git checkout -b your_efeaturename
```

6.4.3 Implementation

All the eFeatures in the eFEL are coded in C++. Thanks to an [eFeatures dependency settings file](#), several implementation of the same eFeature name can coexist. E.g. [this](#) is the file with the implementations of all 'V5' features. You can implement the new eFeature by extending one of the current LibV* files, or by creating your own. You might want to consider starting the implementation by writing a test for the eFeature (see below for instruction on how to do that).

6.4.4 Updating relevant files

Apart from the implementation in the LibV*.cpp file, other files have to be changed to accomodate the new eFeature

- efel/cppcore/LibV5.h: Declare your feature
- efel/DependencyV5.txt: Add your eFeature and its dependencies to this file
- efel/cppcore/FillFptrTable.cpp: Add a reference to the eFeature in the relevant table

- `efel/cppcore/cfeature.cpp`: Add the type of the eFeature
- `AUTHORS.txt`: If your name isn't there yet, add yourself to the authors list
- `efel/units/units.json`: Add the units of the eFeature to the API

You can confirm everything compiles correctly by executing:

```
make test
```

6.4.5 Adding a test

Most eFeatures are fairly easy to implement in Python, so it is advised to first write a Python implementation of your eFeature, and to add a test to it. Then, while you are implementing the code in C++ you can easily compare the results to the test.

The tests of the individual eFeatures are [here](#). Just add your own test by defining a new function `'test_yourfeature()'`.

Some test data is available [at this link](#), but you can of course add your own traces.

The easiest way to run the tests is by executing:

```
make test
```

6.4.6 Add documentation

Add the documentation of the new eFeature to this file:

<https://github.com/BlueBrain/eFEL/blob/master/docs/source/eFeatures.rst>

Please provide some pseudo-Python code for the eFeature.

The documentation can be built by:

```
make doc
```

It can be viewed by opening:

```
docs/build/html/index.html
```

To build the documentation, `pdflatex` has to be present on the system. On a Mac this can be installed using [Mactex](#). On Ubuntu one can use:

```
sudo apt-get install texlive-latex-base texlive-latex-extra xzdec
tlmgr install helvetic
```

6.4.7 Pull request

When all the above steps were successful, you can push the new eFeature branch to your github repository:

```
git commit -a
git push origin your_efeaturename
```

Finally create a pull request:

<https://help.github.com/articles/using-pull-requests/>

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

e

- `efel`, [121](#)
- `efel.api`, [121](#)
- `efel.io`, [127](#)
- `efel.pyfeatures.cppfeature_access`, [129](#)
- `efel.pyfeatures.extrafeats`, [132](#)
- `efel.pyfeatures.isi`, [129](#)
- `efel.pyfeatures.multitrace`, [135](#)
- `efel.pyfeatures.pyfeatures`, [135](#)
- `efel.pyfeatures.validation`, [138](#)
- `efel.settings`, [139](#)
- `efel.units`, [138](#)

A

`activation_time_constant()` (in module `efel.pyfeatures.pyfeatures`), 136

B

`bpap_attenuation()` (in module `efel.pyfeatures.multitrace`), 135

`burst_factor` (`efel.settings.Settings` attribute), 140

`burst_ISI_indices()` (in module `efel.pyfeatures.isi`), 130

`burst_mean_freq()` (in module `efel.pyfeatures.isi`), 130

`burst_number()` (in module `efel.pyfeatures.pyfeatures`), 136

C

`calculate_features()` (in module `efel.pyfeatures.extrafeats`), 132

`check_ais_initiation()` (in module `efel.pyfeatures.validation`), 138

`current()` (in module `efel.pyfeatures.pyfeatures`), 136

`current_base_end_perc` (`efel.settings.Settings` attribute), 140

`current_base_mode` (`efel.settings.Settings` attribute), 141

`current_base_start_perc` (`efel.settings.Settings` attribute), 140

D

`deactivation_time_constant()` (in module `efel.pyfeatures.pyfeatures`), 136

`dependencyfile_path` (`efel.settings.Settings` attribute), 139

`depol_block()` (in module `efel.pyfeatures.pyfeatures`), 136

`depol_block_bool()` (in module `efel.pyfeatures.pyfeatures`), 137

`DerivativeThreshold` (`efel.settings.Settings` attribute), 139

`DerivativeWindow` (`efel.settings.Settings` attribute), 141

`DownDerivativeThreshold` (`efel.settings.Settings` attribute), 139

E

`efel`
module, 121

`efel.api`
module, 121

`efel.io`
module, 127

`efel.pyfeatures.cppfeature_access`
module, 129

`efel.pyfeatures.extrafeats`
module, 132

`efel.pyfeatures.isi`
module, 129

`efel.pyfeatures.multitrace`
module, 135

`efel.pyfeatures.pyfeatures`
module, 135

`efel.pyfeatures.validation`
module, 138

`efel.settings`
module, 139

`efel.units`
module, 138

`exp_fit()` (in module `efel.pyfeatures.pyfeatures`), 137

`extract_stim_times_from_neo_data()` (in module `efel.io`), 127

F

`feature_name_exists()` (in module `efel.api`), 123

G

`get_cpp_feature()` (in module `efel.pyfeatures.cppfeature_access`), 129

`get_dependency_file_location()` (in module `efel.api`), 124

`get_distance()` (in module `efel.api`), 124

`get_feature_names()` (in module `efel.api`), 124

`get_feature_values()` (in module `efel.api`), 124

`get_mean_feature_values()` (in module `efel.api`), 125

`get_py_feature()` (in module `efel.api`), 125

`get_settings()` (in module `efel.api`), 125

`get_unit()` (in module `efel.units`), 138

H

`halfwidth()` (in module `efel.pyfeatures.extrafeats`), 132

I

`ignore_first_ISI` (`efel.settings.Settings` attribute), 142

`impedance_max_freq` (`efel.settings.Settings` attribute), 142

`inactivation_tc_end_skip` (`efel.settings.Settings` attribute), 142

`inactivation_time_constant()` (in module `efel.pyfeatures.pyfeatures`), 137

`initburst_freq_threshold` (`efel.settings.Settings` attribute), 141

`initburst_sahp()` (in module `efel.pyfeatures.isi`), 130

`initburst_sahp_end` (`efel.settings.Settings` attribute), 141

`initburst_sahp_ssse()` (in module `efel.pyfeatures.pyfeatures`), 137

`initburst_sahp_start` (`efel.settings.Settings` attribute), 141

`initburst_sahp_vb()` (in module `efel.pyfeatures.pyfeatures`), 137

`initial_perc` (`efel.settings.Settings` attribute), 140

`interburst_voltage()` (in module `efel.pyfeatures.isi`), 130

`interp_step` (`efel.settings.Settings` attribute), 140

`inv_fifth_ISI()` (in module `efel.pyfeatures.isi`), 131

`inv_first_ISI()` (in module `efel.pyfeatures.isi`), 131

`inv_fourth_ISI()` (in module `efel.pyfeatures.isi`), 131

`inv_ISI_values()` (in module `efel.pyfeatures.isi`), 131

`inv_last_ISI()` (in module `efel.pyfeatures.isi`), 131

`inv_second_ISI()` (in module `efel.pyfeatures.isi`), 131

`inv_third_ISI()` (in module `efel.pyfeatures.isi`), 131

`irregularity_index()` (in module `efel.pyfeatures.isi`), 131

`ISI_CV()` (in module `efel.pyfeatures.isi`), 129

`ISI_log_slope()` (in module `efel.pyfeatures.isi`), 130

`ISI_log_slope_skip()` (in module `efel.pyfeatures.isi`), 130

`ISI_semilog_slope()` (in module `efel.pyfeatures.isi`), 130

`ISI_values()` (in module `efel.pyfeatures.isi`), 130

`ISIs()` (in module `efel.pyfeatures.isi`), 130

L

`load_ascii_input()` (in module `efel.io`), 128

`load_neo_file()` (in module `efel.io`), 128

M

`max_spike_skip` (`efel.settings.Settings` attribute), 139

`min_spike_height` (`efel.settings.Settings` attribute), 141

module

`efel`, 121

`efel.api`, 121

`efel.io`, 127

`efel.pyfeatures.cppfeature_access`, 129

`efel.pyfeatures.extrafeats`, 132

`efel.pyfeatures.isi`, 129

`efel.pyfeatures.multitrace`, 135

`efel.pyfeatures.pyfeatures`, 135

`efel.pyfeatures.validation`, 138

`efel.settings`, 139

`efel.units`, 138

P

`peak_image()` (in module `efel.pyfeatures.extrafeats`), 133

`peak_time_diff()` (in module `efel.pyfeatures.extrafeats`), 133

`peak_to_valley()` (in module `efel.pyfeatures.extrafeats`), 133

`peak_trough_ratio()` (in module `efel.pyfeatures.extrafeats`), 134

`phaseslope_max()` (in module `efel.pyfeatures.pyfeatures`), 137

`precision_threshold` (`efel.settings.Settings` attribute), 141

R

`recovery_slope()` (in module `efel.pyfeatures.extrafeats`), 134

`register_feature()` (in module `efel.api`), 125

`relative_amplitude()` (in module `efel.pyfeatures.extrafeats`), 134

`repolarization_slope()` (in module `efel.pyfeatures.extrafeats`), 134

`reset()` (in module `efel.api`), 126

`reset_to_default()` (`efel.settings.Settings` method), 143

`rise_end_perc` (`efel.settings.Settings` attribute), 140

`rise_start_perc` (`efel.settings.Settings` attribute), 140

S

`sahp_start` (`efel.settings.Settings` attribute), 141

`save_feature_to_csv()` (in module `efel.io`), 128

`save_feature_to_json()` (in module `efel.io`), 129

`set_dependency_file_location()` (in module `efel.api`), 126

`set_derivative_threshold()` (in module `efel.api`), 126

`set_double_setting()` (in module `efel.api`), 126

`set_int_setting()` (in module `efel.api`), 126

`set_setting()` (`efel.settings.Settings` method), 143

`set_setting()` (in module `efel.api`), 126

`set_str_setting()` (in module `efel.api`), 127

`set_threshold()` (in module `efel.api`), 127

`Settings` (class in `efel.settings`), 139

[single_burst_ratio\(\)](#) (in module *efel.pyfeatures.isi*),
[131](#)
[spike_count\(\)](#) (in module *efel.pyfeatures.pyfeatures*),
[137](#)
[spike_count_stimint\(\)](#) (in module
efel.pyfeatures.pyfeatures), [137](#)
[spike_skipf](#) (*efel.settings.Settings* attribute), [139](#)
[spikes_in_burst1_burst2_diff\(\)](#) (in module
efel.pyfeatures.pyfeatures), [137](#)
[spikes_in_burst1_burstlast_diff\(\)](#) (in module
efel.pyfeatures.pyfeatures), [137](#)
[spikes_per_burst\(\)](#) (in module
efel.pyfeatures.pyfeatures), [137](#)
[spikes_per_burst_diff\(\)](#) (in module
efel.pyfeatures.pyfeatures), [137](#)
[strict_burst_factor](#) (*efel.settings.Settings* attribute),
[140](#)
[strict_burst_number\(\)](#) (in module
efel.pyfeatures.isi), [132](#)
[strict_stiminterval](#) (*efel.settings.Settings* attribute),
[141](#)

T

[Threshold](#) (*efel.settings.Settings* attribute), [139](#)
[time\(\)](#) (in module *efel.pyfeatures.pyfeatures*), [138](#)
[trace_check\(\)](#) (in module *efel.pyfeatures.pyfeatures*),
[138](#)

V

[voltage\(\)](#) (in module *efel.pyfeatures.pyfeatures*), [138](#)
[voltage_base_end_perc](#) (*efel.settings.Settings* at-
tribute), [140](#)
[voltage_base_mode](#) (*efel.settings.Settings* attribute),
[141](#)
[voltage_base_start_perc](#) (*efel.settings.Settings* at-
tribute), [140](#)